

VECTOR

APL: the Language and its Application

- APL94 Conference Prospectus 6
- A First Look at APL*PLUS III 60
- Cannon and Smillie on Easter 67
- ASLGREG with Predictions 81
- Allan Gay on VSAM Migration 133

... plus

- 16-page Educational Supplement 17

*The Journal of the
British APL Association*



A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.10 No.3 January 1994

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL*PLUS/PC, APL*PLUS II, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the Vector TrueType font, available free from Vector Production).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1993-94

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£20	1	1
(Supplement for Airmail, not needed for Europe)	£8		
UK Corporate Membership	£100	10	5
Overseas Corporate	£155	10	
Sustaining	£430	50	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates are £250 per full page, £125 for half-page or less (there is a £75 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 04393-385.

CONTENTS

		Page
Editorial	Anthony Camacho	3
APL NEWS		
Quick Reference Diary		5
APL94: The Language and its Application Conference Prospectus		6
News from Sustaining Members	Gill Smith	9
The Education Vector	Alan Mayer	17
REVIEWS SECTION		
APL Product Guide — Updates	Gill Smith	33
GDDME by Lingo Allegro — Update	Adrian Smith	35
RECENT MEETINGS		
APL93 Conference Evaluation Survey	Marc Griffiths	37
APLomb: the view through Quad-shaped spectacles	Ian A. Clark	41
BAA-GUI		
Dyalog APL Session	Jonathan Barman	51
APL*PLUS II Session	Anthony Camacho	57
APL*PLUS III — a First Look	Adrian Smith	60
GENERAL ARTICLES		
When is Easter?	Ray Cannon	67
A Note on the Easter Algorithm in J	Keith Smillie	78
ASLGREG: Proposed Additional Features		
Predictions with Confidence Limits	Ellis Morgan	81
The N-Queens Problem in 1 line	R.G. Selfridge	94
TECHNICAL SECTION		
Hackers' Corner: Minding your .INIs	Adrian Smith	96
At Play with J	Eugene McDonnell	100
Loops in APL2	H Hausmann	106
A Control Operator	Denis Samson	113
The Ultimate TURING Proof	G.A. Langlet	124
A VSAM Simulation for APL*PlusII/386	Allan Gay	133
Index to Advertisers		143



SOLITON ASSOCIATES

SHARP APL: ***the high-performance choice***

SHARP APL is superior in the rapid development of mission-critical applications which meet the ever-changing demands of our customers. Users profit from its many benefits, including robustness in handling shared file multi-user applications plus its recently introduced UNIX-MVS/MVS-UNIX connectivity.



SOLITON PROVIDES:

- Superior productivity
- Powerful cooperative processing
- High performance DB2 interface
- Applications for end-users and programmers
- Responsive support services

For more information, phone or fax:

SOLITON ASSOCIATES LIMITED

44 Victoria Street, Suite 2100
Toronto, Ontario, Canada M5C 1Y2

Tel: (416) 364-9355 Fax: (416) 364-6159

In Europe

Soliton Associates Limited of Canada
Box 8230, 1005 AE Amsterdam
The Netherlands

Tel: 31 20 570-8733 Fax: 31 20 570-8758

In the U.S.

Soliton Associates Incorporated
1100 University Avenue, Suite 111
Rochester, New York, USA 14607

Tel: (716) 256-6466 Fax: (716) 256-6469

Editorial

by Anthony Camacho

Reduction in Membership Dues for People Outside the British Isles

The Committee recommends that we should charge people outside the UK only enough to cover extra mailing costs. This will reduce dues for almost all our European and American members. The new rates will be proposed at an Extraordinary General Meeting to be held on 4th February 1994 so that they may apply for the 1994-5 membership year. At the same EGM some other changes to our constitution will be proposed to bring it into conformity to the BCS rules (the date of the AGM), to allow the Committee to vary the membership year and to simplify administration (additional signatories for cheques).

British APL Association Outstanding Achievement Award 1994

Nominations are invited for presentation at the 1994 AGM. It is a personal award — not to be given to a team of people. We think there isn't much point in giving the award to a large team who then would get so little each they all agree to give it to a deserving charity. The award is for an achievement — not necessarily for a life's work. The winner holds the trophy for the year and gets a cheque for £500 to spend as he or she likes. We want it to be a pleasant surprise to win it. Previous winners have been Roger Hui for the J interpreter, Paul Chapman for the I-APL interpreter, John Scholes for the Dyalog interpreter; anyone would think that the only achievement possible in the APL world was writing an interpreter. Show us that isn't so. No-one is barred from nomination (non-members are eligible too).

British APL Association Committee for 1994-5

Here is your chance to enter public service and do some good to the world. This year the Chairman, Treasurer, Publicity Officer and Education Officer have completed three years in their posts and must resign. We have no Activities Officer because Richard Weber had to resign. Every post is open for election; we would be very glad indeed to see some competition. Please send nominations (ensure the nominee agrees) to the Secretary, Duncan Pearson, 143 Hull Road, York YO1 3JX or email <100265.1564@compuserve.com>.

The Journal of Array Processing Languages and Algorithms

This is the Editor's choice of name for what we have been calling The APL Review. People said the old title would mislead as there are publications with Review in their titles which contain no original material but only criticisms of other work. The new title is long enough to impress and can be abbreviated to Jnl, APL & A. The Editor tells me he is serious about including any array processing language or algorithm and would like to see material in, for example, K or Nial. Please send papers to Jonathan Barman, Hilltop House, East Garston, Newbury, Berks RG16 7HD, UK or email <100116.1030@compuserve.com>. We are glad to report that unsolicited papers are already being received. This proves, what we always maintained, that there was plenty of high class academic work being done which is not being offered to Quote-Quad or Vector because they are not fully peer-refereed journals.

APL in Business: Swansea 18-22 July 1994

The deadline for papers has been extended. By the time this Vector appears you will be running rather close to the deadline. If you want to present a paper and have not yet told us, it is now urgent to contact Dr Alan Sykes, the Program Chair with the details. His address is European Business Management School, Swansea University, Singleton Park, Swansea SA2 8PP, UK or you can email <A.M.Sykes@swansea.ac.uk>.

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association
c/o Gill Smith
Brook House, Gilling East,
YORK YO6 4JJ**

Price in UK: £12 per complete volume (4 issues);
£14 (overseas), £20 (airmail) including Postage.

Quick Reference Diary 1994

Date	Venue	Event
February 4th	IEE	BAA Extraordinary General Meeting
March 6-8	Phoenix, Arizona	Symposium on Applied Computing APL-Track Chair: Chris Lee (Manugistics) Contact Chris Lee at (301) 984-5117
March 25th	IEE	BAA Meeting
May 27th	IEE	BAA Annual General Meeting
July 18-22	Swansea	APL in Business
Sept 11-15	Antwerp	APL94
October 28th	IEE	BAA Meeting: Vendor Forum

British APL Association meetings are normally held in the IEE, Savoy Place.
Nearest tube outlets: Temple or Embankment.

APL Training Courses for 1994

If you would like to have your courses or seminars listed in Vector, please contact Gill Smith with the details.

Dates for Future Issues of VECTOR

	Vol.10	Vol.11	Vol.11
	No.4	No.1	No.2
Copy date	4th March 94	3rd June 94	2nd Sept 94
Ad booking	11th March 94	10th June 94	9th Sept 94
Ad Copy	18th March 94	17th June 94	16th Sept 94
Distribution	April 94	July 94	October 94

APL94: The Language and its Application

Conference Overview

The theme of the conference will focus on APL — the language, its environment, its practice, and its applications. A list of application topics covered could be (being not exhaustive):

APL and Education	Biology	Finance	Mathematics
Astronomy	Economics	Binary Algebra	Medicine
O.R.	Numerical Calculus	Engineering	Decision Support
Statistics	Process Control	Expert Systems	Physics
Production Control	A.I.	Chemistry	Fuzzy Sets
Actuary			

APL is a living language, continuously extended and adapted to the evolution of information technology. On the other hand, we must convince the outside world that APL is the most powerful and most productive programming language, by showing its facilities in practice, illustrated by successful applications. We think that applications are important for the outside world. We will attract people to APL if we offer them solutions to their problems: we need running applications to show them.

Conference Site

The APL94 International Conference will take place from 11th to 15th September 1994 at the Government House, Antwerp, Belgium. The conference site is in the centre of the city. Hotel reservations available at registration will include the Plaza Hotel and the Waldorf Hotel, both at about ten minutes' walk from the conference site, and the Arcade Hotel. Antwerp is very easy to reach by air, car or by train.

Papers and Posters

Papers will be presented on APL — the language, its environment, its practice, and its applications. There will also be presentations involving related languages, such as Nial and J. The papers will be printed in Proceedings provided to the participants.

Posters give the opportunity to publish later breaking ideas, to announce preliminary results, and to state a claim for first publication. They allow a one-on-one dialogue between presenter and attendees, and permit deeper discussion. There is no deadline for presenting a poster, but if you send a 100 to 120 word abstract and a one or two page summary to the Program Chairman before 31st March 1994, the abstract will be published in the Invitations and Final Program, and the summary will be distributed to all participants of the conference. Posters will be published after the conference in one of the leading APL Journals. We should note that the posters have more importance every year.

Workshops

Workshops allow interested parties to discuss specific issues in a small and informal forum. A text, eventually addended by personal notes of potential participants and a bibliography on the subject, will be distributed to the participants of the workshop and will be used as basis for further discussion. At a panel, to be held before the closing session, the conveners will report the discussions made and decisions taken at their workshops. A short post-conference report on each workshop will be published in one of the leading APL Journals.

Tutorials

On Sunday and Monday 11/12 September, tutorials will be given in two parallel tracks:

1. An Introduction to APL & Extended APL
2. An Introduction to the new language J

The tutorials — except for the lunch — are free of charge for the participants and are open to non-participants to the conference. Other subjects have been suggested. If you wish to organize an additional tutorial, please send a detailed proposal, outline, brief description of topics, intended audience, and resume(s) of presenter(s) to the Program Chairman (Deadline: 31 March 1994)

Vendor Forums

Forums allow vendors to present detailed presentations of their product development and plans. On the other hand, vendor forums are giving the participants the opportunity to express their expectations for the future. If your company or organization is interested to present a forum, please contact the Conference Chairman before 31 January 1994.

Conference Chairman:

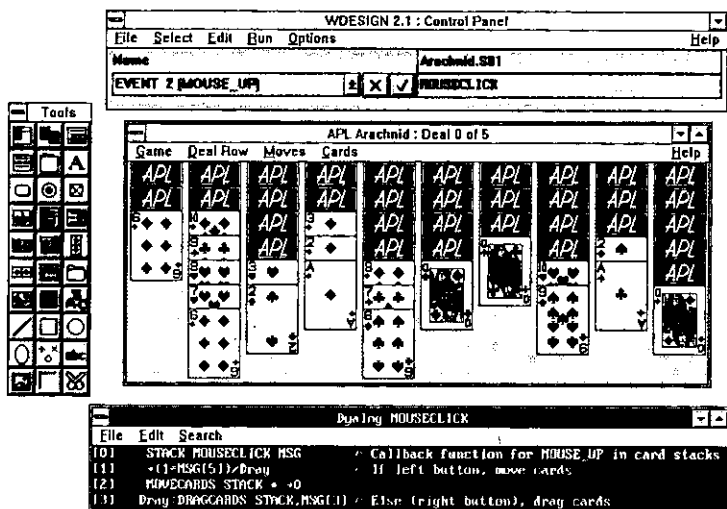
Joseph De Kerf
Rooienberg 72
B-2570 Duffel - Belgium
tel: (32)15-314724, fax: (32) 3-2160689
e-mail: tiapl94@autoctrl.rug.ac.be

Program Chairman:

Alain Delmotte
Avenue du Marathon 6
B-1348 Louvain-la-Neuve - Belgium
tel (home): (32)10-451192
fax: (32)10-451714
e-mail: apl94pgm@ires.ucl.ac.be



The DEFINITIVE APL for WINDOWS™



- Produces snappy, professional-looking Windows applications.
- FREE Run-Time.
- Powerful, elegant and easy-to-use GUI support.
- Complete device-independent graphics, including bitmaps, icons.
- Built-in support for drag-drop.
- WDESIGN workspace for interactive GUI development.
- DDE support through Shared Variables.
- Direct access to Windows clipboard.
- On-the-fly calls to DLLs (eg Oracle, Microsoft SQL Server) using ☐NA.
- No workspace limitations.
- Supports all Windows printers with TrueType APL font.
- Workspaces portable to OSF/Motif, Windows NT and OS/2.

DYADIC

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 OAL,
United Kingdom. Tel: (0256) 811125 Fax: (0256) 811130 Email: sales@dyadic.co.uk.

News from Sustaining Members

Compiled by Gill Smith

Insight Systems

As a new Sustaining Member, we would like to start off by expressing our thanks to the BAA for providing us with this excellent platform from which to express our continued commitment to APL.

Insight Systems was formed in 1990 from the remains of I.P.Sharp Copenhagen. We suddenly found ourselves with no mainframe to logon to, and spent the first couple of years searching for good development tools on workstations with sufficient algorithmic punch for the type of applications we were used to. Needless to say, we did not find anything until the APL vendors finally got their act together.

The next search was for connectivity tools which would enable us to get at corporate data from the development tools of our choice, whether they be Excel, C++, Smalltalk or APL. To our delight, we discovered that the product which we liked best — SequeLink Client/Server MiddleWare™ — did not have a Danish distributor.

Besides marketing SequeLink to the general Data Processing Public in Denmark, we have created a product called SQAPL. The "SQAPL Client" gives APL applications access to all SequeLink-supported services (which includes practically all major SQL databases) via just about any network protocol. Our "SQAPL Server" product allows APL systems to function as SequeLink Servers. With the SQAPL Server, you can use a Unix APL Server to provide improved performance and security to Windows or OS/2 applications. Client and Server systems need not use the same APL interpreter — you can write a front-end in APL/W or APL*PLUS II under Windows and call APL2/6000 applications if you wish.

To non-APL SequeLink Clients such as Visual Basic, C++ or Excel, the SQAPL Server looks like any other server — with a "non-standard" dialect of SQL. Data is input to APL in the form of "bind variables", and returned in tabular form.

Since we announced SQAPL at APL93, we have distributed SQAPL products to customers using Dyalog APL and APL*PLUS II under Windows, OS/2 and Unix, to connect to Dyalog APL, Ingres, Oracle (6 and 7), RDB, OS/2 DBN, DB2/2 and DB2/370 over TCP/IP, SPX, DecNET, and NetBIOS. SQAPL is also available for APL2/2 and APL2/6000, as soon as these products become generally available.

The development of SQAPL has resulted in some tools for the delivery of APL tools on multiple platforms. We can now generate C-coded tools or utilities capable of handling APL arrays from (or converting between internal forms of) Dyalog APL, APL*PLUS II or APL2 (and we plan to add APL/68000 and Sharp APL), packaged for any of these systems under Windows, OS/2 or Unix. Please give us a call if you have an idea which needs this type of technology.

Manugistics Inc

The phones at Manugistics have been ringing all day long since we announced plans for the new APL*PLUS III for Windows, due for release in Spring 1994. We're very encouraged to hear from so many clients who have expressed excitement and approval of the design plans we announced for APL*PLUS III. Many of you may have had the opportunity to preview the product during James Wheeler's "seven cities in 10 days road show" in Europe this past December where feedback from seminar attendees was overwhelmingly positive.

Here are some highlights of the upcoming release:

- APL*PLUS III will be built using the Win32 API, Microsoft's official facility for supporting 32-bit applications under Windows. This ensures that the applications you build will also run on Windows NT and the future "Chicago" version of Windows. Win32s will be packaged with the product.
- APL*PLUS III features a pure Windows-based session manager and full-screen editor that follows Windows style conventions. This consistent graphical user interface means that in addition to having a standard Windows look-and-feel, Windows users will be able to very quickly become productive, probably without even opening the manuals.
- *Free run-time!*
- The APL-based APLGUI workspace in the current version of APL*PLUS II is being replaced with facilities that are written in C and built into the interpreter for APL*PLUS III. This means that the toolkit itself will be smaller, simpler, and fast. Applications written using the APLGUI toolkit in APL*PLUS II will be upwardly compatible with APL*PLUS III/Windows.

Those of you who program in other languages besides APL will be glad to know APL*PLUS III will include control structures similar to those in C for easier code readability and maintenance. Even some hard-core APL purists who have seen how we're implementing control structures admit they will be of value.

Other features in APL*PLUS III include on-line Help, improved DDE and DLL interfaces, and the APL*PLUS Debugger, repackaged with a Windows user interface.

Anyone purchasing APL*PLUS II between now and the time APL*PLUS III is available will receive a coupon for a free migrate to the Windows product when it is released. Beta systems of APL*PLUS III are available through European dealers.

For those of you running DOS applications only, we've also announced that an unbundled version of APL*PLUS II for DOS is now available. This version runs under both DOS and Windows, but will not include the Windows interface and APLGUI toolkit. This represents an opportunity for those of you who do not need a true Windows APL to get the full power of APL*PLUS II at a much lower price (check with your local dealer for pricing).

MicroAPL Limited

MicroAPL is announcing APL Level II for the Apple Macintosh with PowerPC at MacWorld in San Francisco on January 5 1994. I am sure by now that most of you will have read about the new generation of Macintosh computers based on the PowerPC RISC processor which are scheduled for release in the first half of 1994. These computers will, for the first time, bring the power of high-end RISC workstations to the personal computer market.

The new interpreter has been translated to run native on the PowerPC using PortAsm, MicroAPL's 680x0 to RISC assembly-language translation tool. APL Level II is based on and 100% compatible with APL68000 Level II, which was originally written in 68000 assembler to ensure maximum performance. PortAsm allows us to efficiently translate this code onto the PowerPC architecture while still maintaining a single set of APL interpreter source code. Indeed the compatibility between our Apple Macintosh version and the PowerPC version is so great that even auxiliary processors written in 68000 assembler will work without translation! This is achieved by automatically triggering the 68000 emulator which Apple have designed into the Macintosh for the PowerPC.

As with all our second-generation APLs, APL Level II is designed for maximum compatibility with IBM's mainframe APL2 product. APL Level II workspaces and APL files share the same binary format as those of our APL68000 products.

Features of APL Level II include:

Close conformance to IBM's APL2	Full Mac user interface
Nested and mixed arrays	Easy access to Mac toolbox
Selective specification	APL GUI event handling
User-defined operators	Multi-tasking
N-wise reduction	Sophisticated session manager
Axis on scalar functions	Multiple independent edit windows
Overlays (packages)	Direct import from other APLs
Powerful file system	Free runtime licences available

APL for PowerPC will be available simultaneously with the commercial launch of the first PowerPC Macintoshes in early 1994. It comes complete with a collection of support workspaces and complete documentation.

MicroAPL's development staff have also been at work on the RISC System/6000 and the result is the completion of our APL68000/X to C—ISAM interface. Based around the Informix implementation of C—ISAM, this product allows the APL programmer to create multi-key files where the keys themselves may be made up of several sub-parts. Access to files can be by exact key match, generic key match, first record and read forward, or last record and read backwards. It is supplied with a support workspace and full documentation.

As part of our continuing commitment to quality we have been enhancing our in-house APL testing suite. We can now perform over 200,000 automatic tests on new interpreters (or new releases of existing interpreters) to ensure that they are performing correctly. This is achieved by comparing known results with the results from the new interpreter. APL statements and their results are transferred onto the testing environment then the APL statements are executed by the new interpreter and the new results are compared with the originals.

Dyadic Systems Ltd

During the last quarter of '93, details of the "X/Open Common Desktop Environment" (CDE), which was announced earlier in the year, began to emerge. As a result, Dyadic postponed its plans for a December release of Dyalog APL for Motif and a parallel upgrade to Dyalog APL/W for Windows.

CDE is part of a major new initiative which is backed by all the leading Unix vendors to provide a common application programming interface and a standard desktop environment for Unix workstations. CDE is based on OSF/Motif with additional software and tools contributed by members of the consortium. For example, the desktop is developed from Hewlett Packard's VUE and the inter-client communications is based on Sun's ToolTalk. CDE is intended not only to

provide standardisation for developers and users alike, but also to deliver all of functionality and ease-of-use features of Microsoft Windows. Indeed, one way of looking at CDE is that it is the Unix answer to Windows.

Although the decision to re-target the Motif version of Dyalog APL for CDE has delayed the product considerably (CDE itself will not be available until mid '94), it will bring major benefits to APL users. In particular, CDE will enable Dyadic to deliver 100% APL/W compatibility in areas where this is impossible to achieve with Motif alone. For example, CDE provides very "Windows-like" printer support and font management facilities that are entirely absent from Motif itself. In addition, CDE's inter-application communication facilities are equivalent to Windows DDE and will permit the full implementation of shared variables in the Unix version of Dyalog APL.

Dyalog APL for CDE will be provided as a free upgrade to existing Dyalog APL/X users who have a current Support and Maintenance Agreement. To allow customers to begin to migrate existing APL/W and APL/X applications to the new product, an interim version for "raw" Motif is available on IBM RS/6000 and Sun SPARC systems. This version lacks certain features such as the PRINTER object and shared variables, and is supplied with the Dyalog APL/W manuals rather than the final CDE documentation. Nevertheless, this version is otherwise complete and will run most APL/W applications (with the restrictions noted above) unchanged. Customers wishing to take advantage of this offer should contact Dyadic or their local Dyalog APL distributor.

Although there will be several months delay before CDE is available from IBM, Sun Microsystems etc. the Dyadic development team will not be idle. Rather than producing a parallel (with Motif) end '93 release of Dyalog APL/W as originally planned, the company intends instead to develop an even richer new version of the product. In addition to the features announced at APL93, this version will include major object-oriented extensions to the APL language and many exciting enhancements to the already extensive GUI support facilities.

In response to the growing number of successful Dyalog APL/W products, Dyadic is planning to produce a third-party software catalogue ("Outer Products"). This will be distributed to existing customers and shipped with each new copy of the software. To obtain an entry in the catalogue, please contact Peter Donnelly at Dyadic.

Soliton Associates Limited

Version 4.02 of SHARP APL for UNIX has been well received by customers, and Soliton's mainframe customer base is showing increasing interest in porting large-scale multi-user SHARP APL applications from IBM MVS mainframes to the UNIX environment. Significant cost savings can be obtained by running such applications under UNIX. Where required, Soliton's NSVP and MVSLink products provide access from UNIX to MVS data and services.

Current development plans for SHARP APL for UNIX include enhanced support for very large SHARP APL applications on SMP (symmetrical multi-processor) UNIX platforms.

An Application Programming Interface (API) for Soliton's Network Shared Variable Processor (NSVP) has recently been announced. The NSVP API supports access from non-SHARP APL software both to SHARP APL and to SHARP APL Auxiliary Processors. The NSVP accessed by the API can be either local or remote, thus the NSVP can be on the same computer as the calling software or on a network-connected computer.

The NSVP API will support the client-server architecture, typically with non-SHARP APL clients accessing SHARP APL or SHARP APL Auxiliary Processor servers. Initially, the NSVP API will be available under UNIX, with support for UNIX-MVS connectivity. This will allow, for example, access from another vendor's APL for UNIX to Soliton's DB2 Auxiliary Processor for MVS. Most of Soliton's major auxiliary processors for MVS can now be installed stand-alone (thus without SHARP APL for MVS) and accessed via NSVP from network-connected UNIX and MVS computers. Support for the NSVP API in additional environments, including Microsoft Windows, is under consideration.

Kestrel Consulting

First of all, let me wish all Vector Members a Happy New Year! At the time of going to press it is December and far from things slowing up in preparation for the festive season, the phone lines have never been hotter! Our American contracts in particular have been coming through fast and furious and we are, as ever, on the lookout for APL or SMALLTALK professionals with a taste for the good life.

As a result of our constant dealings with America, we now have an office in Los Angeles which enables us to provide an even better service for anyone wishing to work in the USA. Kestrel takes care of the Visa application and costs, pays for a return flight to the USA and foots the bill for the initial accommodation and

hire car expenses. All you need to do is provide our clients with a glowing telephone interview!

If you fancy a change of scene but not a trip across the Atlantic, don't despair! We are in contact with many APL sites in this country and are sure to be able to offer you something of interest.

Dutch APL Users Group

The Dutch APL Users group organised an APL day in The Hague in close co-operation with Share Europe. A short summary of the speakers and their main points follows.

- Theo Zwart of OASIS chaired the day and started with a general introduction.
- Louis van Geldrop of Pecoma kicked off with an evocative statement about 'The Good, the Bad and the Ugly of APL'. It appeared that the ugly stands for the ugly APL characters, the bad was for the suppliers (insufficient standardisation, APL 2 late) and the good were (of course) the APL programmers.
- David Eastwood of Micro-APL talked about 'The Advantage of APL in Application Development'. He raised many interesting points in defining the characteristics of 4GLs in general and where to fit in APL. Unfortunately, the tight schedule did not allow for much discussion. David finished with a small APL.68000 demo on the Macintosh.
- Jim Brown of IBM talked about 'Inside the Human Brain with Parallel Processing'. He showed how you could feed another processor with APL instructions; you can not only transmit data and APL expressions (and get them executed and replied to) but even send APL functions by sending a transfer-file-function. Jim also elaborated about the possibilities of assigning several computations (for example, 128 sections of a brain-slice entomogram) to several processors, or to have a PC session manager for user-interface and a mainframe computer for computation.
- Peter Donnelly of Dyadic Systems demonstrated his event-driven paradigm for GUI and other objects (graphics in particular). Peter showed the 4 basic GUI quad functions `WCreate` `WSet` `WGet` and `WName`, how properties of objects could be set (positional as well as by keyword), deleted and copied (... well, ... next week).
- Paul Lewi from Janssen Pharmaceutical spoke about 'APL as Tool of Thought; an Instrument of Problem Formulation'. "Not solving, that is less interesting than problem formulation", he said. Paul elaborated upon the virtue of APL's symbols and of simplification. He showed several examples of APL equalities which are obvious in APL's symbols but are not obvious, or may even be impossible in a 3GL. He elaborated about the virtue of being able to simplify (thereby losing details, of course) like Mercator's projection as done in factor analysis. Paul showed several examples of the use of factor analysis. Your

acceptance of this tool depends on your being a Platonist (the shadows give you all you can know about reality) or Aristotelian (what you see is all there is to see).

- Benoit Paquin of Soliton Associates talked about 'Sharp APL for the Nineties'. First he recapped the history of Soliton (I.P. Sharp, Reuters, Soliton; gigantic databases, data, APL). He stressed the strong expertise of Soliton in networking and communication between very different computers. This was also illustrated by means of Soliton's NSVP (Network Shared Variable Processor) where APL on an IBM mainframe under MVS corresponds to APL (SAX actually) on a Unix machine. Benoit invited other vendors to work together more closely and share each others' strengths (a point also raised earlier by van Geldrop).
- Jim Brown talked about 'Network Programming with Sockets: A Comparison of C and APL2'. Dressed in his socks, Jim said he was wearing a suit and a tie and wanted to talk about sockets (not socks). So he talked about protocols. Jim showed how to communicate by means of the TCP/IP protocol to other machines by AP119. He compared the elaborate way of doing that in C with the comprehensive way of doing the same in APL (mainly due to the need for extensive declarations in C). It seemed wasteful to repeat the character string 'TCPIP' in each command but Jim explained that this was 'a feature', because it would enable them to communicate via other protocols too (if necessary).
- James Wheeler of Manugistics was unable to come but sent his slides. So in his place Theo Zwart presented them. They mostly stressed that Manugistics is still working to improve the Windows version of APL*PLUS (and they will drop DOS support to some extent). A full Windows version was finally announced, called APL*PLUS III. APL will also have the feature that it can be called from another language, like C, by DLL. Manugistics are also still working at a Windows version for Statgraphics. A positive message was the announcement of a free run-time version in the future.
- Nancy Wheeler of IBM talked about 'APL2 for OS/2 and APL2 Connections'. Like Benoit she recapped history, but now IBM's view, in particular about the APs. Nancy reported that IBM is working towards more consistency in the use of APs so a user could port his applications between different machines without bothering about differences in AP syntax. Nancy also presented a short wish-list of top-priorities (but apparently still under discussion) including porting to other, non-IBM-machines, Windows NT, Windows? and (after some prompting from the audience) the APL compiler. Finally, she and Jim gave a demonstration of several features of the OS/2 version of APL2: easy printer support, colour font, keyboard handling and the use of name association.
- Although it had become very late, Bernard Smoor evoked a lively but short discussion about Vendor co-operation.

Leiden, 4 November 1993
Eke van Batenburg

THE EDUCATION VECTOR

January 1994

Editor Alan Mayer

This Education Vector has been reprinted from VECTOR Vol.10 No.3. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Anthony Camacho, 11 Auburn Rd, Redland, BRISTOL, BS6 6LS Tel: 0272-730036.

Contents

Editorial	Alan Mayer	18
Letter	Kenneth Iverson	19
Editorial Comment	Alan Mayer	
Defined Functions for the Calculation of Determinants	Joseph De Kerf	21
Taylor Arithmetic	Walter Spunde	23
J-Ottings 2	Norman Thomson	29

Dr. Alan Mayer
European Business Management School
University College of Swansea
Singleton Park
Swansea SA2 8PP Wales, UK

Ysgol Rheolaeth Busnes Ewropeaidd
Parc Singleton,
Abertawe SA2 8PP

Tel: 0792-205678 Ext.4274 Fax: 0792-295626 Email: A.D.MAYER@SWANSEA.AC.UK

Editorial

Happy New Year and welcome once again to Education Vector. 1994 sees the tenth anniversary of the first edition of Vector in May 1984, when the British APL Association made the "quantum leap" from a simple newsletter to the professionally produced (and regularly appearing!) journal that is now read throughout the world. Education Vector first appeared two years later, in October 1986, under the editorship of Norman Thomson — who has maintained a close (and valued) interest through the years, and is now contributing articles on J, one of which appears in this edition.

During 1994 the British APL Association plans to introduce another journal, dedicated to papers on original, serious applications of APL, J and other array-orientated languages. The journal was announced last year as "The APL Review", but this title came in for some criticism at APL93, and a more formal title has been adopted: "*Journal of Array Processing Languages and Algorithms*". All papers will be fully-refereed and edited by an international team of APL experts in a wide variety of fields, such as statistics, econometrics, engineering, actuarial applications, simulation, operational research and so on. People who use APL professionally can learn from each other, even when their fields of study or application are very different. The new journal aims to draw all of these fields together and provide a vehicle for inter-disciplinary communication.

Linked with the launch of the journal is the special International Workshop "*APL in Business*" at the European Business Management School, Swansea University, in July. Jointly sponsored by the British APL Association and EBMS, under coordinators Adrian Smith (Chairman) and Alan Sykes (Programme Chair), this will use the common medium of APL to draw together the inter-related themes of Finance, Operational Research and Statistics.

In September (11th to 15th), the Annual International Conference, APL94, takes place in Antwerp, and authors are invited to submit papers on APL — the language, its environment, its practice and its applications. The Programme Chairman is Alain Delmotte, Avenue du Marathon 6, B-1348 Louvain-la-Neuve, Belgium.

In this edition of Education Vector, I am pleased once again to include a substantial contribution from Walter Spunde, who is no stranger to our readers, and one from Joseph De Kerf, who is the Conference Chairman of APL94.

We are always keen to hear of local initiatives to promote APL. A "first" to my knowledge is an APL Class advertised by Bill McLean (also well-known to our

readers) through the University of Glasgow Adult Continuing Education scheme. Well done Bill! Perhaps there are others out there who could promote APL in this way.

The British APL Association is always looking for new members. Application forms are available from Rowena Small, 8 Cardigan Road, London, E3 5HU. UK subscription is currently £12 (students £6).

Letter and Editorial Comment

From: Kenneth E. Iverson, Toronto, Ontario, Canada.

I was pleased to see that discussion of J is beginning to appear in the Education Vector. J has a variety of ways of composing functions to produce new functions (using hooks and forks as well as several conjunctions) in addition to the well-known composition of mathematics; I believe that a simple unified introduction to the forms of composition would be helpful to readers of Education Vector. I therefore suggest that you invite submission of such an article.

Note on J for beginners by the Editor

Norman Thomson made a mistake in J-ottings 1. It is only too easy to do and as Editor I have to admit that it was my responsibility to spot the error and correct it. I failed and will probably fail again. I find J very difficult.

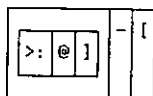
Unlike Kenneth E Iverson, I feel that what we offer as a correct expression should be correct. Ken is unworried about an error, saying "If it doesn't work then the reader will have to find out why and the effort will be good for him." I wonder whether the complete absence of complaints on my desk about the error is because nobody spotted it or because everyone is too kind. The helpful suggestion Ken made in his letter, reproduced here, was that we should commission an article to explain composition.

We have done so. We hope to be able to publish it in April. Experienced APLers too easily think they have the right mindset to understand J. J is a dialect of APL or at least they have a common aim — to enable algorithms to be expressed briefly and without ambiguity but almost everyone has difficulties when they begin to learn J. Even Donald McIntyre admitted to coming close to despair before he could write J expressions which did what he wanted.

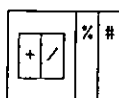
There is no single text which we can advocate as the best way to learn J and even the articles in Vector are imperfect as the early ones were written in early

versions of J which have been superseded. Some expressions have changed meaning. Even the Dictionary of J has occasional omissions as Gene McDonnell remarks (see *At Play with J* in this issue of Vector).

A valuable aid to a learner who wants to see how an expression will be evaluated is to enter the expression without arguments. For example when you enter `>:@]-` the system displays its interpretation of this: `>:@]-`

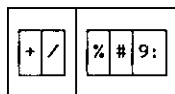


This shows that the effect is `( >: @ ] ) - [` as Norman Thomson says. Another of Norman Thomson's expressions is: `+/%#`



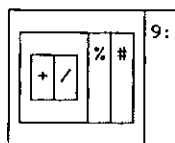
But when you give it an argument its interpretation changes, unless you have inserted parentheses round the expression shown.

Of course if you supply an argument the result is no longer the interpretation of the expression but the result of the expression applied to the argument so you have to use a little cunning to see the effect: you supply an argument in the form of a verb which has a suitable argument as a result `+/%#9:`



[Note: the verbs `0:` to `9:` were added to J in version 6.]

The way to get the result you want is to enter parentheses: `(+/%#)9:`



APL Defined Functions for the Calculation of Determinants

by Joseph De Kerf

In a lot of scientific and statistical problems, we need the solution of a system of linear equations. In APL such systems may be solved directly with the dyadic form of the domino. In addition, if there are more equations than unknowns, the method of least squares is applied.

In some applications however, we need to calculate a determinant, for instance as a byproduct of the solution of such a system of linear equations. In SHARP APL and in J, the dyadic form of the inner product is extended to its monadic form $f.gA$, such that for instance $+.*A$ and $-.*A$ return the permanent and the determinant of the right argument A respectively. However, this extension was not implemented in any other current dialect of APL. In the meantime, most APL users who need the value of the determinant of a matrix are referred to a defined function.

Consulting the literature to find an appropriate defined function for the calculation of the value of the determinant of a numeric square matrix, we imposed *a priori* the following conditions:

1. The function should work for every numeric square matrix. The result for an empty numeric zero-by-zero matrix should be 1.
2. The function should be index origin independent, working not only for index origin 1, but also for index origin 0.
3. The function should return a scalar, not a one-element vector, neither a one-by-one matrix.
4. If the argument is not a numeric square matrix, no value may be returned as explicit result. Eventually, either an error report is signalled, or an appropriate message is displayed.

More than twenty defined functions published in the literature were examined. None of the investigated functions satisfied the stated requirements. As such, we propose two new defined functions which satisfy those requirements, as far as tested at least. Both functions conform to ISO APL. It is supposed that, either the APL implementation used does not support nested nor heterogeneous arrays or,

if the implementation supports nested and heterogeneous arrays, then the arguments are simple and homogeneous.

```
[0] D←DET1 M;N;I;IO
[1] →((2≠ρρM)∨(1≠/ρM)∨(1↑,M)=≠1↑,M)/0
[2] →(2>N)/0,D←(+1↑,M)+0=N+1+ρM
[3] D←NρI+IO+1
[4] LAB:D[I]+DET1 M[1+;N;(I≠N)/N]
[5] →(N>I+I+1)/LAB
[6] D←-/M[1;]*D
```

DET1 calculates the determinant by recursion, based on minor expansion. It is proposed to be used in the classroom, as a demonstration of such minor expansion. In addition, it is a very illustrative example of the mechanism and possibilities offered by recursion.

```
[0] D←DET2 M;N;I;IO
[1] →((2≠ρρM)∨(1≠/ρM)∨(1↑,M)=≠1↑,M)/0
[2] →(1=1↑ρM)/0,D←+/1↑,M
[3] →(0=1↑ρM)/0,D←IO+1
[4] LAB:M[I,1;]+M[1,I+([/|M])⋮[/|/|M;]]
[5] M[;J,1]+M[;1,J+([|M[1;]]⋮[/|/|M[1;]])]
[6] D←D×M[1;1]×-11*+/1≠I,J
[7] M←(1 1 +M)-(1+M[;1])×1+M[1;]÷M[1;1]
[8] →(1≤1+ρM)/LAB
```

DET2 calculates the determinant by elimination, with as pivot in each cycle the maximum of the magnitude of the elements of the current active matrix. It is proposed, not only to be used in the classroom as a demonstration of the elimination method, but also as a subfunction for practical applications in which the value of a determinant is needed.

Finally, if the APL implementation used supports nested and heterogeneous arrays, and we do not restrict the arguments to be simple and homogeneous, for APL2-line implementations e.g., line 1 in both functions could be replaced by:

```
[1] →((2≤M)∨(2≠ρρM)∨(1≠/ρM)∨∨((εM),1↑εM)ε⊂AV)/0
```

A detailed report of this investigation, including details about the behaviour of the defined functions from the literature examined, has been published in the APL-CAM Journal, Vol.14, No.3, 16 July 1992, pp 417-488. Copies of this report are available at the address of the author:

Joseph De Kerf, Rooienberg 72,
B-2570 Duffel, BELGIUM.
Tel.(32) 15-314742.

Taylor Arithmetic

by Walter G. Spunde spunde@usq.edu.au

Introduction

In the APL93 conference paper[4], it was shown how a differentiation arithmetic for *function samples* could be developed using two functions *By* and *Times* for calculating the samples of linear combinations and products of functions with known samples. A function *sample* was defined as a vector of vectors holding the values of the function and its derivatives up to a given order at a set of sampling point of the function's domain. The set of derivative values at any particular point may be used to write a truncated Taylor series for the function in the neighbourhood of that point, thus providing an analytic expression for the function in some neighbourhood of the point.

$$f(x) \approx \sum_{r=0}^n \frac{f^{(r)}(x_0)}{r!} (x - x_0)^r = \sum_{r=0}^n a_r (x - x_0)^r$$

Following Rall's development [3] and Neidinger's [2] suggestion that the Taylor coefficients, $\{a_r\}$ be used in computation rather than the derivative $\{f^{(r)}(x_0)\}$ values, a function sample could be re-defined as a vector of vectors holding the Taylor coefficients for the function at a set of sampling points. This article demonstrates the simplicity of developing such a Taylor arithmetic and applying it to the solution of differential equations. A calculus course that develops skill with differentiation arithmetic and Taylor series has its reward in being able to produce series solutions for systems of non-linear differential equations without any further tools of analysis.

The Differentiation Arithmetic

Function *By* remains the same for Taylor samples as for derivative samples:

```
T←M By Tns;n
n←⌊ρ⌋, "Tns ⋄ Tns←n+⌈Tns ⋄ T←÷+/m×Tns
```

The product of two Taylor samples is simply the polynomial product truncated to the order of the lowest order sample.

```
T←T1 Times T2;n;⌊io
⌊io←0 ⋄ n←(ρ,T1)⌊ρ,T2
T←+((⌊n)∘.≤⌊n)×(⌊n)∘.ϕ(n†,T1)∘.×n†,T2
```

We say that a function, f , is *known* if, given any sample, U , for a function, $u(x)$, the sample for $f(u(x))$ can be computed. For example, the *square* function is known, since the sample for the square of any function can be found from U Times U ; the polynomial $2+3x-4x^2$ is known, since the sample for $2+3u-4u^2$ can be computed from $2 \ 3 \ -4$ By One U (U Times U), where $One = 1 \ 0 \ 0 \dots 0$. Samples for the identity and constant functions of order n are given by

$$\begin{array}{ll} T \leftarrow n \text{ Constant } a & z \leftarrow n \text{ Identity } v; m \\ T \leftarrow (c, a), n \rho c(\rho, a) \rho 0 & m \leftarrow \rho v \leftarrow, v \\ & z \leftarrow (c v), (c m \rho 1), (n-1) \rho c m \rho 0 \end{array}$$

When a function, $y = f(x)$, satisfies a differential equation of the form

$$\frac{dy}{dx} = g(x)$$

where g is a *known* function with sample G , the chain rule for differentiation may be used to find the sample for $f(u(x))$ for any *known* function u ; that is f is *known*.

$$\begin{array}{l} T \leftarrow CFTI \ U; \square i 0; n \\ \square i 0 \leftarrow 1 \diamond \rightarrow (1 \neq n \leftarrow \rho, U) / L1 \diamond T \leftarrow f, U \diamond \rightarrow 0 \\ L1: T \leftarrow (f \ 1 \uparrow U), (G \text{ Times } D \ U) \div 1 n - 1 \end{array}$$

Here $1 \uparrow U$ are the values of function u at the sampling points, and the function D gives the Taylor sample for the derivative of u .

$$\begin{array}{l} T \leftarrow D \ U; \square i 0 \\ \square i 0 \leftarrow 0 \diamond T \leftarrow 1 \uparrow U \times 1 \rho, U \end{array}$$

Whereas for a derivative sample we had only to drop the first term of the sample to obtain the sample for the derivative of a function, for a Taylor sample, each coefficient must be suitably adjusted since the r^{th} Taylor coefficient is related to the r^{th} derivative by

$$a_r = \frac{1}{r!} \frac{d^r u}{dx^r}$$

Since the function *Times* returns a sample of the same order as the smaller of the orders of the two samples in its arguments, it is clear that even if G is only of zeroth order then the result of executing line $L1$ in *CFTI* will produce a function sample of order 1. Repeated execution of this line can therefore be used to find a sample whose order is the same as that of U . Thus when a function, $y=f(x)$, satisfies a differential equation of the form

$$\frac{dy}{dx} = g(x, y)$$

where g is a known function, the sample G can be computed to 0th order provided that the function values of y are computable (or arbitrarily assigned) at the sample points. The function *CFTN* can be used as a prototype for defining functions that make *known* all the elementary functions of mathematics, and any desired special functions as well.

```
T←CFTN U;□io;n;T0
□io←1 ◊ +(1+n+ρ,U)/L0 ◊ T+f,U ◊ →0
L0: T+T0+f 1+U
→(n=ρT)/0 ◊ T+T0,(G Times D U)÷:ρT ◊ →1/c
```

Replacing f by $\div$ and G by $-T \text{ Times } T$ will produce a function for calculating the samples of reciprocals (thus providing a way for calculating samples for quotients). Using 30 for f and $1 \ 1 \text{ By One } (T \text{ Times } T)$ for G will calculate samples for the tangent of any function. For functions such as the arctangent, where f is replaced by -30 and G by *Reciprocal 1 1 By One (U Times U)* we may use function *CFTI* since the repeated executions of the final line are not required.

Solving Differential Equations

To find the n^{th} order sample for the integral of a function, $f(x)$, whose Taylor sample at x can be computed by executing the character string F , composed of known functions of X , where $X+n \text{ Identity } x$, one need only execute (with $\square io \leftarrow 1$) $Y \leftarrow Y0, (n \div \pm F) \div ! : n$ where $Y0$ is the 0th order sample of (arbitrary) initial values of the function at x points.

When the function y satisfies a differential equation of the form $y' = f(x,y)$ the string F contains terms in Y . If Y is of order $k-1$, then executing F gives a Taylor sample for y' of order $k-1$. The last term of this sample is $y^{(k-1)}/(k-1)!$ or $y^{(k)}/(k-1)!$, which, if divided by k , could be catenated to Y to give a k^{th} order sample for y . Thus, repeated execution of the line

```
Y←Y, (~1±F)÷k
```

will produce Taylor series solutions of the differential equation about the sampling points to any order. Series solutions for differential equations can be obtained from function *TDE*.

```
Y←F TDE Data;k;n;X;Y;Y0
□io←1 ◊ k+0 ◊ n+Data[1]
X+n Identity 2>Data ◊ Y←Y0←~1+Data
→(2==Data)/L0 ◊ Y←Y0←Y
L0: →('Y'εF)/L1 ◊ Y←Y0,(n±F)÷!:n ◊ →0
L1: →(n≤k+k+1)/0 ◊ Y←Y, (~1±F)÷k ◊ →L1
```

As an example we find a family of solutions for the logistic equation

$$\frac{dy}{dx} = y(1-y)$$

The precise solutions can be found analytically to be

$$y(x) = \frac{1}{1 + ae^{-x}}$$

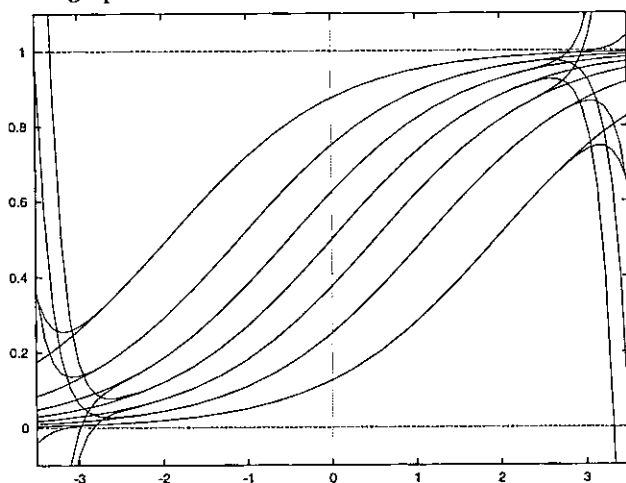
Given non-zero initial values y_0 at $x = 0$, these solutions can be computed from

$$x \leftarrow 100 \text{ Divs } \frac{1}{n} \text{ } a \leftarrow (1/y_0) - 1 \text{ } es \leftarrow 1 + (-x) \cdot a$$

where $n \text{ Divs } I$ is a function that divides an interval, I , into n equal parts. First we find the values of the polynomial approximations to the solutions of degree 8, using a polynomial evaluating function, $Poly$, with

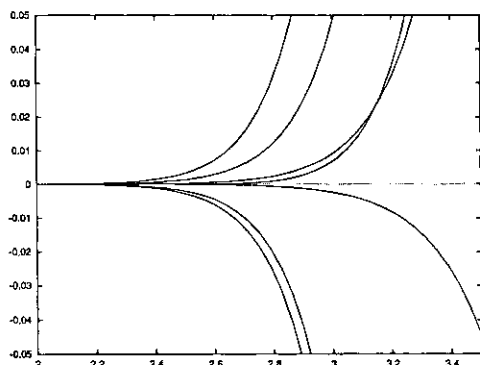
$$\begin{aligned} One &\leftarrow 20 \text{ Constant } 1 \text{ } degree \leftarrow 8 \text{ } de \leftarrow Y \text{ Times } \frac{1}{n} \text{ By One } Y \\ x_0 &\leftarrow 8 \rho_0 \text{ } y_0 \leftarrow 7 \text{ Divs } 0.125 \text{ } 1 \\ ps &\leftarrow \psi(, f \text{ } de \text{ TDE degree } x_0 \text{ } y_0) \text{ Poly } cx \end{aligned}$$

The largest difference between the values for exact (es) and approximate (ps) solutions was 0.00005768, over the $(-1, 1)$ range. Over a range of $(-2, 2)$ the largest error was 0.01117 and errors were readily apparent on a graph. Recalculating the approximate solutions with 20 degree polynomials we found the largest error to be 0.00003448 over the $(-2, 2)$ range, with the errors becoming apparent on a graph when $x > 2.25$



Solutions and Polynomial Approximations for $y'=y(1-y)$

The errors only become significant at the ends of the interval of usefulness and the accuracy of the results within the interval is much greater



Errors in Approximate Solutions for $y'=y(1-y)$

Speeding up the Computations

The function *CFTN* above recalculates all lower order terms in a sample with every repetition of its final line through application of the *Times* function. This can be avoided by computing only the leading term of the polynomial product in this line.

```
T←CFT U;n
→(1+n+ρ,U)/L1 ♦ T+fn ,U ♦ →0
L1: T+fn 1+U
→(n=ρT)/0 ♦ T+T,+(/((φ(ρT)+D U)×G))÷ρT ♦ →□lc
```

The function listing for the modified function, *CFT*, may not display the simplicity of the chain rule as clearly, but it does give much faster execution times. Using Neidinger's [1] test function (*X Times Reciprocal Sin X Times Reciprocal Ln Arctan Exp X* average execution times for calculating a 10th order sample at the point $x=1$ were reduced from 2.58 to 1.43 seconds, and for a 25th order sample from 27.53 secs to 11.97 secs.

The speed of execution of the *Times* function was improved by re-writing it as

```
T←T1 Times2 T2;n;k
k←1 ♦ n+(ρ,T1)lρ,T2
T←(1+T1)×1+T2
→(n<k+k+1)/0 ♦ T+T,+(/((φk+T1)×k+T2 ♦ →□lc
```

A third version, avoiding the loop with the *each* operator, proved to be consistently better.

```
T←T1 Times3 T2;⊞io;n
⊞io+1 ⊙ n+(ρ,T1)⊞ρ,T2
T←⊙+/“(⊞n)⊙“⊞T1)⊙“(⊞n)⊙“⊞T2
```

Average execution times for each of the three versions of the *Times* function for the different inputs are shown below for the computation of both *X Times X* and *(X Times Reciprocal Sin X) Times Reciprocal Ln Arctan Exp X*

Secs	10-1		25-1	
<i>Times</i>	0.055	1.429	0.280	11.95
<i>Times2</i>	0.033	1.275	0.126	7.82
<i>Times3</i>	0.027	1.242	0.110	7.22

	10-25		25-25	
<i>Times</i>	0.093	3.049	0.478	28.23
<i>Times2</i>	0.060	2.165	0.280	14.74
<i>Times3</i>	0.055	2.082	0.258	14.22

Computing the test function as *X Times Reciprocal Sin X Times Ln Arctan Exp X* a 10th order sample at $x=1$ took 1.011 secs (using *Times3*) which is comparable to the time taken by *Mathematica* for this computation. Calculating the values of the tenth derivative of the test function at 100 points took 3.85 secs compared to the 17.5 secs in [4] and was over a second faster than with analogous functions for derivative samples.

Walter G. Spunde,
Department of Mathematics, University of Southern Queensland,
Toowoomba, Australia, 4350

References

- [1] R.D. Neidinger, "Automatic Differentiation and APL", College Mathematics Journal, 20, 1989, 238-251.
- [2] R.D. Neidinger, "Differential Equations are Recurrence Relations", Quote Quad, 23, 1992, 165-169.
- [3] L.B. Rall, *Automatic Differentiation Techniques and Applications*, Lecture Notes in Computer Science, No. 120, Springer-Verlag, Berlin-Heidelberg — New York, 1981.
- [4] W.G. Spunde "Point-wise Calculus", Quote Quad, 24, 1993, 259-266.

J-ottings 2

by Norman Thomson

First, a recantation! In the previous J-ottings I fell into a trap (rapidly pointed out to me by Donald McIntyre) of confusing the *nouns* *R* and *L* in the APL expression $1+R-L$ with the J *verbs* $\lceil$ and $\rfloor$ which mean in their dyadic form *take* the left and right arguments respectively, and monadically *same*, that is *take* the argument unchanged. The sentence $4>:\rfloor-\lceil 5$ is interpreted exactly as in APL, that is three monadic verbs are executed in succession right to left followed by a dyadic one $>:$ meaning *is greater than or equal*. The intermediate stages of evaluation are therefore

```
4>:\rfloor-5
4>:\rfloor_5
4>:_5
1
```

This leads me to the point where the previous J-ottings left off, namely with the thesis that difficulties in learning J arise from the need to understand some profoundly important concepts at a fundamental level. One of the most important is that of verb trains. Hook and fork are the simplest examples of *verb trains* and embody rules to which there is nothing directly comparable in APL. A hook is a pair of successive verbs, unsymmetrical in the sense that the right verb is necessarily monadic and the left necessarily dyadic. If no left argument is present, the right argument doubles as left argument.

In the case of fork the outer verbs are executed before the inner one which means that a fork is conceptually an elementary tree structure.

The inclusion of parentheses in the sentence above, that is

```
4(>:\rfloor-\lceil)5
```

defines a train of four verbs. As in APL, the parentheses cause the evaluation of their contents *prior* to involvement with their arguments. The functions on their own are combined - a hook or a fork is produced from a train and becomes a single verb. In this case they first (working from right to left) form a fork $\rfloor-\lceil$ and then the verb $>:$ combines with the fork-verb to form a hook.

Dealing first with the fork, this has a right argument only, and so the left argument (note that a fork must have a left argument) is supplied by repeating the right argument, and so the verb sequence $\rfloor-\lceil$ evaluates to (right argument -

right argument), that is zeros in the shape of the right argument. Thus the sentence above is interpreted (executing the verbs from right to left)

```

    apply the fork to give 0,
    then apply the hook which asks the question:
      is 4 greater than or equal to 0?,
    answer 1

```

So the expression ignores everything about the right argument (5) except its shape. The same effect is achieved by defining the explicit verb

```
f1=.>:]-[
```

which thus returns the answer to the question "is the left argument positive or 0?" As Donald points out, this is more expeditiously answered with the defined hook

```
f1=.>:0:
```

thereby introducing the verb 0: which delivers the *value* 0.

J also allows the sequential composition of verbs which is explicitly denoted by the conjunction @ (atop). Again there is no directly comparable primitive form in APL. If sequential composition is introduced prior to the fork, viz:

```
f2=.>:@]-[
```

Since conjunctions are evaluated before hooks and forks, this is equivalent to

```
f2=.(>:@)]-[
```

The above may give some insight into the richness of the J language and of the subtleties of the interpretation of J expressions.

J has the facility for tacit definition. This is an extra in the way that some APLs have Direct Definition built in. With tacit definition you may define a verb train, give it a name and its right and (possibly) left argument are not shown. The arguments to the (possibly) left and right of the defined name are then inserted to the left and right of the defined expression. So you can define the mean of a vector as

```
mean =. +/%#
```

then if, for example, $w = .1 \ 4 \ 2 \ 2$, both `mean w` and `(+/%#)w` have the same value namely 2.25. On the other hand if the parentheses are not present the

result of $+/\%w$ is the same as $+/\div pW$ namely 0.25. A very convenient and appealing hook in this context is $-mean$:

```
(-mean)w
_1.25 1.75 _0.25 _0.25
```

which, applying the hook rule, means take the mean away from every item of vector w . (Donald McIntyre in "AMENDMENT – A Change for the Better" (Vector 9.3) describes how this can be extended elegantly to deal with nouns of higher rank.)

Forks can also be used in tacit as well as explicit form. Mean itself qualifies in this regard; so does $(>./-<./)$ for range of a vector.

(A gentle test for the reader at this stage: what does the monadic hook $--$ mean?)

I consider it is helpful to develop an understanding of J by distinguishing between similar sentences, for example consider the result of applying the following tacit verbs:

```
(+/-!)0 1 2 3 4
(+/@!)0 1 2 3 4
(+/@(-!))0 1 2 3 4
```

Round brackets on the whole aid understanding, so consider the final expression first which is "+ insert" atop in sequence the hook $(-!)$. Since there is no right argument the value of the hook is each of the five numbers minus its factorial, that is $_1 0 0 _3 _{20}$. Adding these gives the value $_24$.

In the second expression the removal of the inner parentheses changes the nature of the verb to a hook whose right component is $!$ and whose left component is the verb sequence $" +/ "$ following $-."$ A composition like this, that is two verbs joined by a conjunction, behaves as a single verb entity. Since the rank of the rightmost verb is 0 (in APL terms it is a scalar function) the composed verb also assumes rank 0, that is it is applied on an item by item basis. This is an instance of the rule of "Inherited Rank" (see "Mastering J" by Donald McIntyre in APL91 Proceedings).

Finally remove the $@$ as in the first sentence and what remains is a fork in which the outer verbs are evaluated first so that the sentence is equivalent to $(+ / 0 1 2 3 4) - ! 0 1 2 3 4$ that is $9 9 8 4 _{14}$

Now try repeating this replacing $!$ with $\%$ - this should at the least acquaint you with the symbol $_$ standing for minus infinity!

In conclusion consider the definition of a verb `ss` which describes how to obtain the sum of squares of a vector about its mean. First use the hook `-mean`, then in sequence "square" and "add insert":

```
ss=./@*:@(-mean)
ss v
4.75
```

This is indeed what is wanted, but it is a good idea to test understanding by considering the meanings of some different but similar forms:

```
ss1=.(+/@*:@)(-mean)
ss1 v
4.75
```

This means apply a doubly composed verb "(sum atop square) atop mean deviation" which gives the same result as `ss`. Now consider

```
ss2=./@*:@-mean
1.5625 3.0625 0.0625 0.0625
```

The priority rule for conjunctions means that `ss2` is the hook `((+/@*:@)-)mean`. `mean` acts monadically and delivers its result to the doubly composed verb `((+/@*:@)-)` which acts dyadically with the original argument as left argument and the mean as right argument. Following the Inheritance Rule, the verb `(+/@*:@)` inherits rank 0 from the minus and so is equivalent to `*`. `ss2` is thus a rank 0 verb and its result of `ss2` is the squares of the negation of the mean deviations.

The parentheses around `(-mean)` are thus critical for a correct "sums of squares" definition in order that a rank of 1 is inherited by the later stages of square and sum. More generally the fundamental nature of verb rank gives rise to one of the most important differences between J and APL, and deserves the strongest emphasis.

Ed's comment: The answer to Norman's gentle test is that the expression `--`, if monadic, means subtract from the right argument the negation of the right argument, so the effect is to double each item in the right argument as in the two examples below.

```
(--) 1.5
0 2 4 6 8
(--) 2 3$ 1.5
0 2 4
6 8 0
```

APL Product Guide — Updates

compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

Pressure on space occasionally prevents us from printing the complete guide, however updates will always be listed. We do depend on the alacrity of vendors to keep us informed about their products. Anyone who is not included in the Guide should contact me to get their free entry — see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL-based Packages
- APL Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 04393-385.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Insight Systems	APL*PLUS/PC	poa	APL systems marketed and supported ...
	Dyalog APL	poa	from: Dyalog, Manugistics, IBM
	APL2	poa	under: Windows, OS/2 and Unix

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Insight Systems	IUTILS/XP	20-95	Cross-platform utility library including simple OS calls (DIR, COPY, DEL, RENAME) and DATE functions. For APL*PLUS II, APL2 and Dyalog APL under Windows, OS/2 and Unix.
	ASI	95	APL Spreadsheet interface, "Device-Independent" spreadsheet driver supporting Excel, 123 and Quattro-Pro for Dyalog APL/W
	WinCom	95	Asynchronous comms package for Dyalog APL/W
	S2D,22D,X2X	poa	Advanced APL syntax analysis and conversion packages from Sharp and APL2 to Dyalog, and between any two APLs
	SQAPL Client	poa	Interface from APL*PLUS II, APL2 and Dyalog (Windows, OS/2 or Unix) to most SQL databases over most networks.
	SQAPL Server	poa	Makes APL*PLUS II, APL2 or Dyalog APL (Unix) available as SequeLink servers. Can be called from SQAPL clients or other applications such as Excel, C++, Smalltalk, Visual Basic.

APL CONSULTANCY

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dogon Research	Consultancy	poa	APL Systems consultancy, design, implementation, support, documentation and maintenance. All dialects with special emphasis on APL2 and Dyalog APL/W.
Insight Systems	Consultancy	poa	Experts in APL conversions between any combination of: APL*PLUS, APL2, Dyalog APL and Sharp APL. We are also experienced right-sizers, comfortable with networks and relational databases (that also means when NOT to use SQL) and client/server development in APL, C and Visual Basic.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
Japan APL Assoc.	Tokyo, Japan			
Melbourne APL Grp	Melbourne, Australia			
SE APL Users Grp				

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
APL Club Germany	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: (49) 6227-2003
Dogon Research	Dick Bowman	2 Dean Gardens, London E17 3QP Tel: 081-520-8334 Email: bowman@apl.demon.co.uk
Insight Systems ApS	Morten Kromberg	Nordre Strandvej 119A, DK-3150 Hellebæk, Denmark Tel: (45) 42 10 70 22 Fax: (45) 42 10 75 74
Japan APL Association		23-2-302 Hiromichi, Adachi-ku, Tokyo 120, Japan
Melbourne APL Group	Harvey Davies	CSIRO Div Atm Res, Private Bag No.1, Mordialloc, Victoria 3195, Australia
SE APL Users Group	John Manges	991 Creekdale Drive, Clarkston, GA30021 USA

GDDME by Lingo Allegro — Update

reviewed by Adrian Smith

This is effectively an addendum to the 'first look' in Vector 10.2. GDDME is an independent task which communicates with Dyalog APL/W via shared variables. It provides a very thorough emulation of IBM's mainframe *'Graphical Data Display Manager'* as accessed via AP126 from APL2 and VS APL.

The initial version managed the emulation very solidly, but had a number of interesting deficiencies on the interface front! These all appear to have been fixed at release 2.2 (Dec 93), and the product has acquired some useful extra functionality along the way. I feel it can now be unreservedly recommended as a simple way of lifting a mainframe application and dropping it more or less unchanged on a Windows platform.

Bugs Fixed or Worked Around

The problem with non-interlocked shared variables is basically a deficiency of Windows DDE as opposed to mainframe $\square SVO$ — I never ever bother to refer to a mainframe GDDM variable after an assignment unless there is some possibility of failure. This speeds things up no end, but unfortunately is not tenable with $\square SVO$ on the PC. Accordingly, the rule is now to reference the variable after every assignment, and this is well documented in the GDDME manuals. Here is an amended test function, which works fine:

```

V xx;fmt;nmod;rc
[1]  A Simple GDDME test (X is shared)
[2]  A Make a simple format and set up screen ...
[3]  fmt←2 5p1 4 10 1 24,2 8 10 5 12
[4]  X←(402,(pfmt),,fmt)'' ⋄ rc←X
[5]  X←(407 1 4,407 2 3)'' ⋄ rc←X
[6]  A Fill top field with green underscores ...
[7]  X←(424 1 24)(24p'_'') ⋄ rc←X
[8]  A ... and lower field with red dots
[9]  X←(424 2 60)(60p'.'') ⋄ rc←X
[10] A Set Cursor and Hold it ...
[11] X←(430 1 1 1)'' ⋄ rc←X
[12] X←(,101)'' ⋄ nmod←X ⋄ 'Read result was: '(5+nmod)
[13] A Query modified fields ...
[14] nmod←ϕnmod
[15] X←(420,nmod)'' ⋄ 'Flds to read: '(5+X)
V

```

The problems with the imprisoned mouse have been fixed, as has the function-key mapping — shift+F1 reports F13, which I think is the best choice. You now have a choice of colour schemes (set on startup with a *b=n* parameter) which should cover most requirements. My old *DRAW* and *FSM* code all looks quite acceptable with no modification to the colours.

GDDME now uses the MSDOS-CP437 fonts, which are bitmapped at a variety of sizes. This gives a much-improved readability, particularly at the smaller sizes.

Improvements

The main improvement is support for screens other than 24 by 80. GDDME accepts *r=nn* and *c=nn* parameters on startup, and correctly reports the chosen screen size with *FSQDEV*. It will run up to 132 columns and 43 rows at a very respectable speed (although a decent PC screen resolution is essential to get the benefit).

The other promised improvement is an implementation as a Windows DLL rather than an independent task. This should make it go a little faster, and I assume it will be as easily called from *APL*PLUS III* as from *Dyalog/W*. I do not yet have this version to check.

Remaining Limitations

One small thing to check is whether you use the line-draw characters in your mainframe screen layouts. Nestlé certainly do (to put nice boxes around fields), and we will need to convert these to standard ASCII characters as they are not in the MS DOS font.

Apart from this, my only reservation is on the graphics front: should GDDME emulate the GDDM line-styles and shadings? At the moment a request for line-5 (GDDM code for long-dash) gives an invisible line (the Windows interpretation). Similarly the excellent GDDM dog-tooth tweed patterns render as rather tacky Windows hatchings. Perhaps it might be more work for the emulation, but the quality of the finished products would probably be worth it.

Summary

There is not a lot wrong with this product if you need to shift a mainframe system in a hurry. Even in its shared-variable implementation it feels faster than any mainframe ever did, and with DLL calls it should absolutely fly. Definitely recommended.

Analysis of the APL93 Conference Evaluation Survey

by Marc Griffiths

Internet: marcg@utcc.utoronto.ca

CompuServe: 76260,3314

Introduction

This report is an edited summary analysis of the evaluation survey conducted at the APL93 Conference. The conference was sponsored by the Association for Computing Machinery's Special Interest Group on APL (ACM SIGAPL) and held in Toronto, Canada from 15 to 19 August 1993. Eighty-one questionnaires were returned and analysed. This represents about 25% of an estimated 320 delegates who attended the conference. While this is a good response rate, the sample size is too small for meaningful cross-correlations.

In measuring the success or otherwise of the conference as a whole, bear in mind that two reports appeared in July 1993 which cited an average drop in attendance at conferences in Canada of 20% to 25% compared to a year earlier.

Summary of Analysis

1. Almost three-quarters of the respondents constituted repeat business. Of the newcomers, word of mouth was most frequently cited as the method through which they first learned of the conference. More than one respondent noted a desire for more outsiders to be brought in to the conference, both as attendees and speakers.
2. Over one-fifth of the respondents praised the technical programme, but an almost equal number were dissatisfied with it. Most of the latter want more papers, panels, workshops and tutorials dealing with (commercial) application design and development, and computing topics such as object-oriented programming and graphical user interfaces.
3. Many comments indicated the importance of providing adequate facilities for social and professional interaction outside the planned social events. These manifested themselves as requests for space and time for informal interaction among the delegates between sessions through the day, and for gatherings in the evenings. In addition, several respondents asked for space and equipment to allow attendees to experiment with software which had been discussed in a presentation.

4. The vendor exhibits and forums were among those parts of the conference which elicited the most positive reactions from the largest number of respondents.
5. Cancelled sessions were repeatedly mentioned as a source of annoyance. Some respondents had come to APL93 specifically to attend these tutorials, papers, etc. The feelings of frustration and disappointment were exacerbated further by the failure of the conference organizers to notify delegates of the cancellations in time for an alternative session to be selected. Indeed, in the case of the APL Help Facilities panel, no one knew that the moderator had failed to appear until the audience had assembled in the room.
6. Respondents remarked on shortcomings in the conduct of speakers and panel moderators. These included poor preparation, lax timekeeping and failure to appear.
7. A significant number of respondents encountered difficulties in finding rooms on the conference site. Floor plans of the appropriate buildings and improved signs in corridors and stairwells as well as outside each of the rooms would have remedied this problem.
8. The complaints about some of the events (vendor forums, exhibitors' reception and the City Hall reception) were directed at the amount and type of food. There always appeared to be too little of all kinds, including both vegetarian and non-vegetarian. The two lunch-time vendor forums presented their own problems, as the food was intended for the forum attendees only, but other conference delegates did not hesitate to help themselves to occasionally generous portions. Other respondents suggested that coffee and pastries or muffins should be available for half an hour prior to the start of each day's presentations; that lunch be provided; and that the conference fee include lunch.
9. Some respondents complained about various aspects of the pricing — too high, not in the currency of the host country, and too few things included as part of the registration fee.

Method

A copy of the conference evaluation questionnaire was included in each registration kit. Additional copies were available at the information desk for delegates who wanted to fill in a questionnaire but had forgotten to bring it with them. There was no control mechanism to prevent the return of more than one questionnaire by an individual, but there is no evidence that anyone submitted multiple copies.

The responses to questions 1 to 5 and question 7 were coded and input to a spreadsheet program and simple analyses were carried out. Responses which

did not comply with the instructions on the questionnaire were deemed invalid and excluded from the analysis. For example, questions 2, 5 and 7 asked the respondent to select one from several choices. Where the respondent selected more than one, none of the choices were included in the analysis.

Some questions were open-ended. Analysis of the replies to these was limited to drawing up a list of categories in order to group the replies, and to classifying the responses according to the categories. The categories themselves were inferred from the replies; another person looking at the responses might well have used different categories or allocated the replies differently. Question 6 (Briefly describe how you use APL.) was too general to yield meaningful results beyond the fact that a broad spectrum of people attended the conference. If the questionnaire is used again, this question should be reworded so as to avoid the problems noted above. Bear in mind that (i) not all conference attendees are users, and (ii) it may be desirable to distinguish between APL and J. The large number of comments supplied in response to questions 8 (What did you like most about the conference?) and 9 (What did you like least about the conference?) necessitated the omission of much of the detail. For questions 10 (How would you improve this conference?) and 11 (What suggestions do you have for future conferences?) fewer comments were made for each section, so many of these have been provided verbatim within the detailed section of this report. There is little or no detailed discussion of the results of questions 8 to 11.

Principal Findings

1. Almost 60% of the respondents have access to the Internet. [Q1]
2. Over two-thirds (68.4%) of the respondents first heard about APL93 through attendance at APL92 or by direct mail. [Q2]
3. Almost three-quarters (74.1%) of the respondents had attended a previous ACM SIGAPL sponsored conference. [Q3]
4. Over two-thirds (68.8%) of the attendees came to the conference primarily for one of three reasons: (i) to learn about new applications of APL or J, or their application in a specific discipline (36%); (ii) to meet friends or colleagues (20.3%); (iii) to learn about the APL or J languages (12.5%). [Q5]
5. The most popular parts of the programme were those associated with the vendors — the vendor exhibits, the exhibitors' reception, and the vendor forums. The least popular were the birds-of-a-feather (BOF) sessions, workshops and panels. [Q7]

6. The Casa Loma banquet, the tutorials and the vendor exhibits were the three most favourably rated parts of the programme. The exhibitors' reception, the Art Gallery and the panels fared most poorly in the ratings. [Q7]
7. Respondents offered 93 comments on what they liked most at the conference. These centred on the people and the atmosphere, the technical programme and comments pertaining to the friendliness of the people and the atmosphere dominated the replies (30.1%). The Programme Committee had felt that the quality of the papers was very high, and most of the positive comments about the programme supported this (22.6%). Three respondents cited Timo Laurmaa's paper, "Undocumented Features of APL." Several attendees praised the tutorials very highly, and one was pleased by the "emphasis and interest in J." The vendor exhibits were also praised (11.8%). [Q8]
Paper presentations, plenary sessions, and tutorials all featured as important aspects of the conference in terms of popularity, although the overall assessment of the papers was less than good (2.14). The reasons for this may be accounted for by comments in Q9 to Q11 below, particularly the recurring requests for more papers and tutorials about commercial applications and broader computing topics such as software engineering, graphical user interfaces, etc.
8. Respondents voiced 73 negative comments about APL93. The criticisms were aimed largely at aspects of the organization (timetabling, signs and site) (38.4%), the technical programme (21.9%) and the perceived pre-occupation of the organizers with making money (15.1%). [Q9]
9. With respect to improving the APL93 conference, almost one-quarter (23.4%) of the 64 comments suggested changes to the programme content and scheduling, with another fifth (20.3%) directed at improvements to certain areas of information and communication. A further 10.9% offered ideas regarding reducing costs or simplifying pricing. [Q10]
10. Of 34 proposed future conference locations, the largest number (12) were situated in continental North America, but four of these sites have hosted the conference in 1991 and 1993. Seven sites on the Pacific Rim were put forward, and five in Eastern Europe/Russia. [Q11]
11. Of 26 proposed themes for future APL conferences, over one-third (9) suggested commercial or packaged APL applications, followed by seven for the APL or related (J, K) language, and six for a computing theme such as software engineering, graphical user interfaces, and object-oriented programming. Suggestions for papers, workshops and tutorials fell into the same categories in roughly the same proportions. [Q11]

APLomb: the view through Quad-shaped spectacles

Ian A. Clark

*IAC/Human Interfaces, 9 Hill End, Frosterley-in-Weardale,
Bishop Auckland, County Durham DL13 2SX, England.*

Introduction

APLomb ("APL on the Macintosh for Beginners") is a specialised educational product for the Apple Macintosh, intended to compete with HyperCard for the easy production of science courseware (Clark [2, 3, 4, 5], Clark and Stevens [10]). It doesn't deliver APL so much as leverage it. Its capacity to handle the plusher Macintosh features such as colour pictures, digitised sound and animation, enables it to be classified as a multimedia development system, albeit a low-end, low-cost one.

In fact the content-free part of APLomb, I-APL/Mac 1.6 (Camacho et al [1]), is freeware — only the models and the sets of components on which they are built are sold to generate revenue. These content-laden "courseware" models are ready-to-use in class without additional work, certainly without needing to program in APL or even to know the language. Several APLomb courseware titles are now available in both the UK and the USA (Clark, [6, 7, 8, 9]) and more are on the way.

The Needs Addressed by APLomb

A recent enquiry (Laurillard et al [11]) into the rejection by academics of much existing courseware concluded that the "not-invented-here" syndrome was not a real issue, but that lack of adaptability was. Teachers want it both ways, of course. They want to be able to modify purchased courseware to suit themselves, but to avoid having to do so. This is borne out by the author's personal experience in higher education — there just isn't the time to fool about with something which doesn't work first time. APLomb is the basis for courseware that is both robust, ready-to-run, and yet adaptable. It makes no pretence to perform well outside its intended domain. It has bones to help it stand up, yet you don't need to break its bones to bend it to your purpose. It's a matter of putting joints in the right places, nowhere else. That's always easier said than done.

Since few professional APLers use the Macintosh (one exception being APL68000 users, MicroAPL [12]), it was planned that APLomb should not try to appeal to the APL community at large. It should concentrate on addressing the needs of science and mathematics teachers, especially those for whom APL in itself has no special attraction, even if this meant ignoring major APL concerns such as workspace size, fast performance and second generation functionality; at least, for the present. The door remains open to address these issues at a later date, by enhancing or replacing the I-APL engine which lies at the core. But the I-APL engine turns out in practice to be "good enough" for classroom models. Also, being freeware, it keeps the cost of the product range down and serves to permit APL to be brought to the attention of teachers who might otherwise not give it a moment's thought.

The Perspective to be Adopted

APLomb is a GUI-based APL. You've heard of Visual BASIC ... this, loosely speaking, is Visual APL. In itself this makes it interesting to the APL community at large, especially as the way it achieves its goals runs clean contrary to the general trend: crudely mapping the (C) programmer's GUI interface onto APL functional code.

Our main preoccupation has been with how the product looks from the end-user's point of view, but there's nothing especially APL-noteworthy about the result. It just looks like any other 90's quality Macintosh product. You'd never believe the effort it took to achieve that, with a 60's language like APL, the last word in user-interaction at a time when it was thought smart to connect a golfball-typewriter to a computer.

Of far more APL-oriented concern is a worm's-eye view of APLomb from the perspective of the humble APL engine buried in it. A view through quad-shaped spectacles perhaps, or if you know something about I-APL, a $\square MC$ -shaped keyhole. But neither $\square$ nor $\square MC$ play any significant role in how APLomb communicates with the screen. Indeed the I-APL engine has nothing to do with projecting the workspace data onto the screen and knows nothing about it. APLomb looks after this aspect directly. Strictly speaking, the I-APL engine's view of the screen is a blank wall. This has substantial benefits for the educational modelling use of APL. The language can now be restricted to its proper provenance, the succinct exposition of complex algorithms.

Direct Manipulation as User-friendly Interaction

Shneiderman [13] gives the name *direct manipulation* to the sort of human-computer interface which enables the end-user to see and alter data by interacting in a "natural" way with its graphic image on the screen. As he defines it, specifying a gun-elevation by dragging a slider, or dragging a picture of the gun barrel into the desired position, is an instance of direct manipulation, whereas typing a command: *SETELEVATION 80* or typing 80 into a dialog box is not.

It follows that overtyping the text of an APL function using a full-screen editor is direct manipulation too, provided you accept that the thing you're manipulating is the text of the function, not the function itself, which is not going to change until you quit the editor.

WYSIWYG ("what-you-see-is-what-you-get") is a similar idea, but not quite the same. A given data item can be represented on the screen by a multitude of different graphic devices (e.g. slider, or side-view of gun), all amenable to direct manipulation, whereas WYSIWIG has the connotation of just one "real" portrayal (*viz* what you actually get out of a word-processor as hardcopy).

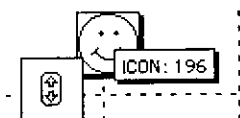
One disadvantage of direct manipulation is that although it makes things easy for the end-user, especially a pupil, it makes things hard for the developer, especially the teacher-developer. Over 90% of the time and effort to build courseware with even the better tools like HyperCard can be spent on achieving robust screen-effects, time which the hard-pressed teacher can ill-afford. Another disadvantage is more subtle. When you mouse-drag a slider on the screen, it looks so substantial (especially if it has a realistic 3-D colour effect) that you can imagine you're altering the actual data. You're not. You're setting up parameters for a function which is going to be called at some future time (e.g. upon pressing "OK") to alter the data. Even teacher-developers can be confused by this distinction, because it pops up in loads of different forms.

Kompos, Windoids and Watched Variables

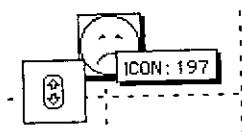
APLomb's goal is to make it easy for the teacher-developer to build direct manipulation models. To do so a collection of ready-made direct-manipulation devices are provided, called *kompos*. ("Kombo" is a neologism derived from "component" — all the best thing-names like *object*, *component* or *module* are employed already to mean something else.)

A typical kompo tracks the value of a single variable in the workspace (called its *watched variable*) and controls the contents of a *windoid*, a little window intended

for direct manipulation. User interaction with the kompo may alter the variable in-place, but more generally triggers an APL expression to update the workspace. The windoid can be dragged around the screen by the user at any stage. It may display a graph, a clock face, an animated picture or an array of symbols, or it may present a column of labelled buttons, checkboxes or radio-buttons:



Three simple kompos watching the same integer variable: ICON.



The same three kompos instantly after clicking the upper arrowhead.

A kompo displaying a picture will project it onto the backdrop if the windoid itself is made to vanish. In this way a composite varying picture may be seamlessly built up on the backdrop. Experiments are underway with a *sprite world* backdrop which supports snatches of fast arcade-style animation. If you're old enough to remember "Pong" and how influential it was in demolishing the mainframes' pre-eminence, you'll appreciate the urgency for educational computers to do battle with the Segas and Nintendos of this world for the attention of tender minds.

Models, Sets and Construction Kits

Kompos are grouped into *sets* which are dynamically linked as required. Kompos have unique IDs. Different sets can share the same kompos. If you

install more than one set you see a menu of the intersection of their kompos. This allows model-building kits to be published containing a single set of just those kompos needed to build the models in the accompanying workbook, but several sets can be installed together to support novel models. All sets will work together. The result is no less, if no more, flexible than a LEGO or Fischer-Technik snap-together construction kit. It depends for its success on an ever-growing assortment of appealing kompos. APLomb is the system of studs on the bricks (kompos), the APL workspace is the baseboard and I-APL is the engine which drives the assembled model.

To people who make a study of educational software, *model* means the representation, (potentially) inside the computer, of the thing you want to teach, such as an electrical circuit or an ecosystem. There's much interest nowadays in making the model accessible to the student, and safe to change in rewarding ways. This can confer a deeper understanding of the model's workings than just watching it run. To APLomb, *model* means the APL workspace. However the two meanings come together, because with APLomb the APL code contains scarcely more than the specification of the model. All mappings from the workspace variables to the screen are handled by kompos.

Kompo Architecture

Is a kompo an *object* in the object-oriented sense? Yes and no. *Yes* insofar as it is an encapsulated, independently-engineered, re-usable component. *No* insofar as it is not controlled by sending it *messages* (which in C++ would happen to be calls to its internally-grouped functions called *methods*), but by altering the values of particular variables in the workspace. A kind of shared-variable communication channel, but without setting-up protocols.

Such an architecture suggests that anything you need to show on the screen can be handled by a kompo. If you are incautious you might start off permitting a kompo to watch any number of variables, but APLomb soon discovered the need to impose a tight structure upon this. A development tool for novice programmers like teachers needs to behave intelligently if parts of the model are missing, whether kompos themselves, or their watched variables. If you start with an empty workspace and select a kompo from the installed set(s), something intelligible must happen. If you want to make a given kompo watch a different variable, it must be obvious what field to change in whatever dialog box. If a watched variable is deleted, or `)LOAD` or `)COPY` changes the collection of variables in some way, you don't want all the kompos to collapse in a sticky heap, but to do the sensible thing. Especially as `)LOAD` or `)COPY` can be

executed secretly, like CHAIN in some dialects of BASIC, without the end-user necessarily realising what's going on.

This is the APLomb solution to these problems. The kompos on the screen at any given time are controlled by a vector of their IDs, invariably called *KMSLATE*, short for "kompo-slate" (Kompos were originally called *Kennel Maids*, the *dog* was the I-APL engine, its *kennel* was the workspace and the *slate* attached told you who was currently attending it. The watched variables are left as an exercise for the reader). Kompos are selected or de-selected from a pull-down menu, which has the effect of adding or removing their IDs from *KMSLATE*. Loading a new workspace makes the new set of kompos spontaneously appear, but kompos common to both *KMSLATE*s stay on-screen without necessarily batting an eyelid. You can make a nice object-oriented data base this way, allocating one workspace per object.

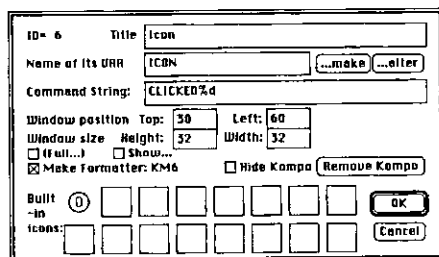
So far so good. You select a kompo and it appears. But suppose it expects to see a variable called *Z* and there's no such variable? Then it creates it and gives it an appropriate sample value. The kompo displays the proper text or picture for this value and you can look in the session log at the expression actually used to create *Z*. Needless to say you can modify and re-input this expression and watch *Z* change. What if you *)ERASE Z*? No, you don't want the kompo to create it again spontaneously, experience shows, so it sits there displaying nothing but "*Z*". One of the operations you can perform on a kompo is to "hit" it, which you'd typically do in frustration, to persuade it to do what you think it ought to. Holding down the option key changes the cursor to a little hammer. Clicking on a kompo makes the appropriate noise, the kompo disappears and reappears just as if you'd freshly selected it. Hitting the kompo which is stuck sitting there displaying "*Z*" makes it regenerate *Z*.

You do the same with several other kompos, then you write APL functions (or *)COPY* them) to map data from one collection of variables to another. That's how to build a model. No APL code is needed to manage the GUI. It all relates to the model, and indeed that model will run under a different (non-APLomb) APL interpreter (in fact you may have got the functions from a foreign workspace in the first place). Of course you'll have to handle the I/O yourself, and there will be variables with no relevance to the new environment, such as *KMSLATE*.

The Formatter Variable

But what about specifying the position of the kompo (i.e. its windoid)? Holding down the command key turns the cursor into a little hand, which lets you drag a kompo to a different position.

You can also change the kompo position and size by holding down the command and option keys (the cursor changes to a little eye), then clicking the kompo. This calls up a dialog box for the given kompo:



The dialog box to change a kompo's attributes.

But when we reload the model at a future date, what's to stop the kompo reappearing in its original position? We need to preserve in the workspace the information of the kompo's whereabouts, plus any other characteristics we might change via the kompo's dialog box.

The changed attributes are recorded in the workspace in the form of a character string variable called a *formatter*. The formatter is optional. If it's present, the kompo will use it. If it's edited or reassigned, the kompo instantly changes to keep track. For the kompo having ID=99, the formatter is invariably named *KM99*.

The formatter is typically generated automatically by APLomb, but only if required. One way to force it to be generated is to *nail down* the given kompo. Holding down the shift key as well as the option key turns the hammer into a hammer-and-nail. Clicking on a given kompo, say 99, makes the appropriate sound, but it also assigns a value to the formatter *KM99*, recording in the workspace the new windoid position, among other things. The formatter is not a dump of some C-structure, but a legible character string, having an *opcode syntax* resembling the graph-plotting format in Clark [2], but with different opcodes of course. Consulting the user guide, you can edit it reliably to alter the kompo, although there is a dialog box to do much the same thing. However the APL programmer can write functions to concatenate or assign substrings to alter a formatter (and hence its kompo) in any conceivable way. A formatter can contain any selection of opcodes. The absent ones will either be unchanged or defaulted. A formatter generated by APLomb happens to specify everything that

distinguishes one kompo from another, so that if KM99 has been automatically generated:

KM66+KM99

makes kompo 66 into a carbon copy of kompo 99, whatever it was before, in the context of the current workspace.

In practice the formatter architecture means that every kompo comes ready-to-run and doing something sensible, but that any or all of its characteristics can be changed. These changes can be recorded as variables in the workspace to which they apply.

The Second Formatter

Some kompos make use of a second formatter. All kompos can in principle accept a second formatter, e.g. *KMF15* in the case of kompo 15, but to date the only kompos making use of the second formatter are the graph-plotting kompos (ID=15, 16, 17 and 18). These expect the second formatter (if it is present) to have precisely the syntax described in Clark [2]. In its absence, they employ a built-in formatter which yields a line-graph with unlabelled axes.

Why two formatters? Originally this gave us a bad feeling, until we realised how beneficial it was. Simply extending the syntax of the first formatter to include this set of specialised graph-plotting opcodes was a programmer's quick-fix with repercussions. The result was an architecture both cumbersome to use and inelegant to watch on the screen. In practice, given the graph plotter kompo, ID=15, the situations in which you want to change *KM15* and *KMF15* are quite distinct. So it pays to have independent variables governing them. You frequently want to change or restore the format of the graph displayed within the windoid (*KMF15*) leaving unchanged the windoid position (*KM15*), or vice-versa. It's nice to do this without the window itself shuddering, which it inevitably does when the first formatter is changed.

Typically you'd copy the second formatter from a special workspace like this, since skill is needed to remember the opcodes and choose parameters for them from scratch:

```
)COPY FORMLIB MYPETFORMAT  
KMF15+MYPETFORMAT
```

FORMLIB consisting of a store of formats. On the other hand the cut-and-try turnaround is fast. Constructing the second formatter via screen controls or a

dialog box is a good candidate for an APLomb model, entailing just the sort of tinkering APLers do well. Since a usable format-generator needs to be specialised and task-dependent, no pressure was felt to support format construction as part of APLomb itself. The result would have been far too general. Nevertheless, format-generators are needed. Writing one would make a good student project. If anyone comes up with one, please contact the author.

The second formatter will be used with certain new kompos too, particularly text boxes and specialised displays like clock-faces. The first formatter has opcodes which let you place integers in a set of 16 general-purpose flags. These flags can be used by the kompo in any way it sees fit, including specifying its textfont, fontsize, style etc. This is untidy and anonymous, particularly as the second formatter syntax already accommodates such things.

Watching Several Related Variables

One vexed question is whether to permit a kompo to watch more than one variable, to handle things like labelled tables, lines of best fit through scatterplots, and so on. A "professional" (i.e. a novice-hostile) system would have no qualms about admitting legions of variables, but contemplating this for an educational product really does give me a bad feeling, since so much hangs on assuming a unique watched variable. A second-generation APL would solve the problem with a nested array, but APLomb uses vanilla-flavoured ISO-standard I-APL for the present. Consequently APLomb holds the line: two matched variables, two matched kompos. For one thing, they don't both flicker when you change one of them. For another, the taste of APL conveyed by APLomb's rendering of PLANETS is blander.

Conclusion

Originally designed as a development system specialised to classroom models, APLomb is perhaps of wider interest to APL GUI developers, due to the way in which screen interfacing problems are addressed. The avoidance of functional APL code to interrogate and update screen artefacts, much less create them, not only restricts APL to its proper provenance, succinct model definition (which may be bad news to the APL professional) but emphasises APL's special learning benefits in that area. It also makes material developed in and for APLomb far less platform-dependent than it would appear to be with most current APL GUI interfaces. It is all a matter of "rendering to C the things that are C's".

References

- [1] CAMACHO, A., CHAPMAN, P., CLARK, I.A., ZIEMANN, D. (1993), *I-APL/Mac 1.6 Instruction Manual*. IAC/Human Interfaces, 9 Hill End, Frosterley-in-Weardale, County Durham, DL13 2SX. ISBN 1-898728-06-2.
- [2] CLARK, I.A. (1992), *Graph-plotting in I-APL/Mac 1.2*. VECTOR, Vol 8, no 4, 113-118.
- [3] CLARK, I.A. (1993), *APLomb: a fashionable face talking a timeless language*. Presented at: APL in Education, British APL Association, London, 11th June.
- [4] CLARK, I.A. (1993), *APLomb: User Guide*. IAC/Human Interfaces.
- [5] CLARK, I.A. (1993), *Getting started building models with APLomb*. IAC/Human Interfaces.
- [6] CLARK, I.A. (1993), *EPIDEMIC: Illustrates the spread of infection in a population*. IAC/Human Interfaces.
- [7] CLARK, I.A. (1993), *SPARKS: Test simple electrical circuits to destruction*. IAC/Human Interfaces.
- [8] CLARK, I.A. (1993), *FIBONA: Breed rabbits according to Fibonacci's Series*. IAC/Human Interfaces.
- [9] CLARK, I.A. (1993), *COINS: Elementary Probability and the Law of Averages by tossing coins*. IAC/Human Interfaces.
- [10] CLARK, I.A., STEVENS, C. (1993), *APLomb: a snap-together construction kit for math, science and technology software*. Presented at: Kaleidoscope: Patterns of Change, 1993 Annual CUE Fall Conf., Santa Clara, Calif., USA, Oct. (CD No. 3, CUE Inc., 1210 Marina Village Pkwy, Suite 100, Alameda, CA 94501.)
- [11] LAURILLARD, D., SWIFT, B., DARBY, J., (1993), *Academics' use of courseware materials: a survey*. Alt-J, Association for Learning Technology Journal, Vol.1, no. 1.
- [12] MICROAPL, (1990), *APL68000 Level II*, MicroAPL Ltd, South Bank Technopark, 90 London Road, London SE1 6LN.
- [13] SHNEIDERMAN, B., (1987), *Designing the User Interface*. Addison-Wesley. ISBN 0-201-16505-8.

BAA-GUI: APLs and The Windows GUI

Dyalog APL Sessions

notes by Jonathan Barman

The Dyalog APL stream for the GUI Programming day was split into two groups. Peter Donnelly of Dyadic Systems Ltd. took one group while Adrian Smith, wearing his APL-385 hat, took the other. At lunch-time the groups swapped over. I attended Peter's tutorial in the morning and Adrian's in the afternoon, so I have described the tutorials in that order.

Peter Donnelly

Peter took us through the basic GUI facilities in Dyalog APL. It would be too much to reproduce it all here, but the following fragment of his introduction will give you the flavour of the way in which he started very simply and then gradually introduced more complex examples:

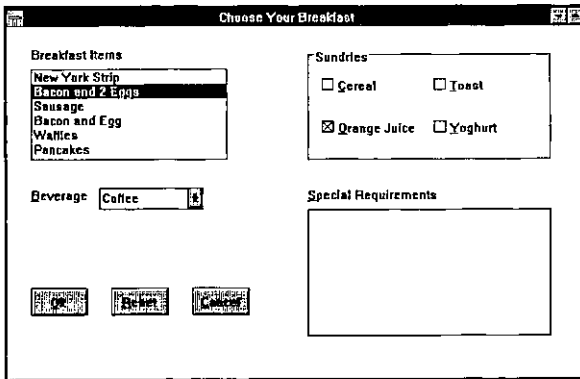
```
'pete'□wc'form'
'pete'□wc'form' 'Hello World'
'pete'□wc'form' 'Hello World'(10 30) (10 10)
```

The first line creates a blank form with everything left as a default. The next line redefines the window with a title. The third line redefines it again with a title, position and size. Peter continued this process with more and more examples until he had covered all of the □ functions □WC □WG □WS □WN and □DQ. The coverage was quite thorough and included how to create buttons, radio buttons, list boxes, groups etc. etc. We were shown how to run 'callback' functions which can be sparked off when a user carries out a particular event such as clicking on a button or selecting a control.

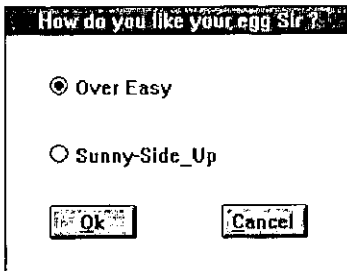
I attempted to type in the examples as Peter ran through them. I must admit that I had some trouble keeping up as Peter is quite a fast typist. Several people in the class also had trouble, so they wisely sat and watched carefully rather than try and reproduce everything at the terminal. Fortunately Peter's examples were almost the same as those in *WINTRO*, which is the introductory Windows workspace supplied with Dyalog APL, so one could always work through them at a more leisurely pace on one's own. In retrospect, I think that it would have been wiser to have got everyone to run the *WINTRO* workspace where all one has to do is to press the F1 key to get the example text. If you have Dyalog APL,

then it is definitely worth while running through the examples in this workspace before attempting to write any code.

After about an hour we were given an exercise to create a fairly complex dialog box with a number of pop-up menus of alternatives:



If you choose an item containing eggs, then the following dialog box should be presented:



The exercise required similar dialog boxes to pop up to specify all the incredible set of alternatives that are possible when ordering an American breakfast. Milk or cream in your coffee, how you like your toast, how well cooked your steak, etc.

The exercise was to be completed within an hour, but I don't think that anyone could really say that they managed to finish every detail. I have not really used Dyalog APL GUI features in anger, and cannot say that I am an expert, but I was quite surprised how far I managed to get in the time. Naturally, I took many

short cuts that I would not normally want to take if I was developing a real system. Peter told us that he took 3 hours to reach the final solution, which sounds about right if one was creating a real system. He distributed his solutions to all the exercises on a disk at the end of the day, so when I got home I was able to analyse the way in which he had solved the problem. I liked the way he used a generalised function to display the various alternatives that could be specified, which I attempted but was unable to complete in the time.

We then went on to explore graphics, with Peter giving us a quick demonstration of the facilities available before we attempted the next exercise, *Scribble*. A blank form had to be displayed on screen, and you had to make it draw a line when you click and drag the left mouse button. On releasing the left button and clicking again the previous drawing had to be cleared. As this only involved an empty form with just two small functions to write, most people were able to complete the task quite quickly. The functions Peter distributed as the solution were:

```

      ∇ MSG←start MSG
[1]   A Callback for EVENT 1 (MOUSE_DOWN) in S
[2]   →(1*MSG[5])/0
[3]   'S.L'□WC'POLY'(1 2pMSG[3 4])
      ∇

      ∇ MSG←draw MSG;P
[1]   A Callback for EVENT 3 (MOUSE_MOVE) in S
[2]   →(1*MSG[5])/0
[3]   P←'S.L'□WG'POINTS'
[4]   P←P,[1]MSG[3 4]
[5]   'S.L'□WS'POINTS'P
      ∇

```

Line [2] of each function is the key to the solution, which fortunately Peter emphasised before we started. This tests to see if the left mouse button is down. If it is, we go ahead and do the processing, but otherwise the mass of mouse-move messages are ignored. Seeing that these functions are called for every mouse-move message, which can occur hundreds of times per second, the process works remarkably quickly.

Finally we explored the *WDESIGN* workspace and developed some simple screen layouts using the facilities available.

I felt that I had learnt quite a lot from my morning's endeavours. When I got home I enjoyed exploring Peter's solutions on the disk that was handed out, and was impressed with the amount of material available.

Adrian Smith

Adrian started off by giving us an overview of three things:

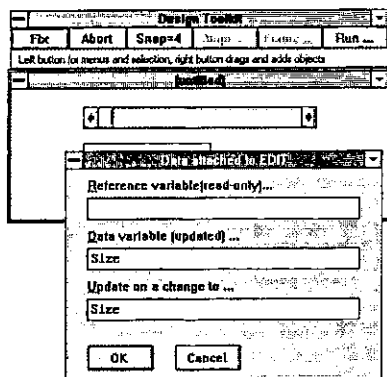
- what we would get out of the session
- what Adrian gets out of it
- the design philosophy behind his GUI workspace

What we got was an excellent GUI design workspace with open code so that we can all do the usual APL things of tweaking the bits we don't like to turn it into a tool that really sings. What Adrian got out of it was feedback on his design, which will hopefully continue. This includes reports of any bugs in his workspace, which is a beta-test version, and comments or suggestions for improvements in the facilities available. Adrian also wants everyone to have fun developing GUI systems, which should increase the use of APL generally.

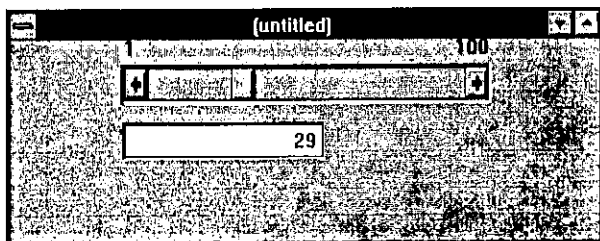
The design philosophy behind the GUI workspace is sort of object oriented. Adrian did not want to use the word *object* because of all the connotations with *Object Oriented Programming*. The way in which he has got messages passing to windows when an APL variable changes does make it feel a bit object-ish, though it obviously is not OOP.

I included in my review of Dyalog APL 6.3 (see Vector 10.1, July 1993, page 66) the initial version of the workspace, and was impressed then. This version is a great improvement. Adrian gave us the briefest of introductions on how the thing worked, and everyone seemed to be able to get going really fast creating GUI objects and getting quite complex things to work. I would have preferred a little more introduction to the features available, but that was probably because I had seen a prototype and immediately tried to do some tricky things where I had had some difficulty.

The design tool looks the same as I reported in Vector 10.1. In the following example I have defined a *Scroll Bar* and a *Numeric Edit Box*, both linked to the variable *Size*, and both to report any change in the variable to the other control:



The pop up *Data box* is new. In it you can define the APL variables that are to be referenced (typically a format specification), shown, and watched for changes. When any monitored variable changes, the object is automatically updated. As you can see from the example, I specified the variable *Size* for the Edit box, which I had also specified for the scroll bar. Having set the variable *Size* in the workspace I ran the whole thing by clicking on the *Run* button in the top right hand corner. This gave me the following:



Moving the slider updates the number in the box, and changing the number in the box updates the slider. All done in a couple of minutes' work.

The array in which the specifications are held is now much simpler. The above example is a 3 by 10 matrix, containing the form, slider and edit box specifications. A simple function (*show*) displays the matrix:

<i>Ty</i>	<i>Caption</i>	<i>Pos</i>	<i>Size</i>	<i>Sty</i>	<i>Ref</i>	<i>Data</i>	<i>Listen</i>	<i>E.A.T.</i>	<i>Id</i>
<i>FM</i>	(untitled)	228 264	144 416						
<i>PA</i>		20 80	24 257			<i>Size</i>	<i>Size</i>		
<i>NH</i>		60 80	24 140			<i>Size</i>	<i>Size</i>		

Everyone I spoke to felt that Adrian's workspace made everything so much simpler. I shall certainly use it for any Dyalog APL development work. I particularly liked the way that Adrian has left the code open so that I can make minor changes. So far, the only thing that I have found that I would want to change is the function for displaying a pop-up list box, where I would want to calculate the size depending on the number of items to be selected.

Conclusion

I felt that the Workshop was a great success. Everyone had a PC to themselves so there was no problem in getting down and solving one's own problems. There were not too many people in each class so the tutors could give individual attention where necessary. The course material clearly set out the way in which GUI systems could and should be built. And, as an extra bonus, we all carried home disks filled with useful code. I would certainly attend similar tutorials which may be held in the future.

What the Presenter got out of it!

by Adrian Smith

This was the best 'usability lab' I could have wished for. I was generally delighted with the ergonomics of the designer, but picked up an obvious need for 'caption', 'position', 'style' to be grouped into a single 'Appearance' dialogue ... done! Less obvious was the need for 'local variables' at the form level. This makes it very much easier to leave things strictly alone until 'OK' is hit (mostly done). Some objects need to be more forgiving (fewer *LENGTH ERRORS* when your boolean doesn't match your options!), and also more catholic in their tastes (nested vectors or simple rank-2 arrays in list boxes).

Anyone who didn't get a copy, and could usefully play with this stuff, please send a disk to APL♦385 with a return address and appropriate postage. Just don't build a real system yet — the design is stabilising, but I don't want a user-base just yet.

BAA-GUI: APLs and The Windows GUI APL*PLUS II Sessions

There were three speakers in the APL*PLUS II stream of the BAAGUI Workshop. Peter Day with interjections from Dave Phillips both of Cocking and Drury, Duncan Pearson of Nestlé-Rowntree, and Dave Piper of Eagle Star.

Peter Day and Duncan Pearson

notes by Anthony Camacho

Peter Day of Cocking and Drury introduced us to the interpreter and the differences between version 4.0 and 5.0. APL*PLUS III will be in a different league and will not be fully upwards compatible. It will only work on 32-bit machines and will not try to run the `□WIN` or `□WGET` or `□WPUT` functions we all learned in APL*PLUS/PC. To make conversion to PLUS III easy one should be prepared to replace these and all the DOS-dependent features such as `□CMD`, `□INKEY`, `□ARBIN`, `□G*(anything)`, all the peeks and pokes, the printing methods and so on.

In APL*PLUS II version 5.1, which we used for the day, there is a function *Win* which sends its right arguments to an APL*PLUS Agent which runs minimised under Windows. This agent is what actually communicates with Windows.

Duncan Pearson explained that if you adopt the procedures recommended by Manugistics, each object (form, key, radio button, edit box, label, combobox and so on) requires a one or two line function to be created. A workspace with say 20 forms each with twelve objects on it would require a workspace of over a thousand functions and as these are created automatically and have long names (made up of the names of their elements) maintenance gets to be very tedious.

Instead he and Adrian Smith have devised a tabular way of recording and implementing the object. The details are entered into a ten column matrix (a nested array) per form where the first line defines the form and the subsequent lines defines the objects in it (which are sometimes, confusingly, called controls).

An advantage of this approach is that there is a single nested matrix for each form in the workspace. The workspace is not cluttered with hundreds of functions. It is then possible to write utility functions to display and amend the objects of a definition matrix. This makes the workspace much easier to understand and to maintain, as a print of each definition matrix effectively summarises the windows interface of the workspace.

We spent a large part of the afternoon trying to construct a definition matrix for a form to collect or amend name and address records. It had to have appropriate fields for name, initials, the lines of the address including country and zip or postcode, some other details such as a classification, telephone number or email address and the necessary keys and buttons to look people up, add, amend or delete them.

It is a tribute to Duncan's exposition that some people actually got something working before the exercise was brought to a halt to allow Dave Piper to show us a different method of overcoming the difficulties in using APL*PLUS II under Windows.

David Piper of Eagle Star *Notes by Duncan Pearson*

David Piper of Eagle Star showed us a completely different approach to the interface with windows. Instead of using all the support functions in the APLGUI workspace to define and create his windows, he uses a standard dialog box editor to define the boxes and a few quite simple windows routines to read them off a resource file and define them. He showed us a small application built using these tools, a PC/Mainframe file transfer utility that unsurprisingly did not have its full functionality, but which showed the user interface well. Then he gave a presentation of his design objectives and implementation details, which was a set of slides each of which was a dialog box run by his utilities. The result was very fast and looked easy for the application developer to use. However it obviously could not have been done without the system programmer (David) knowing a fair amount about Windows dialog box classes.

The process of defining windows interactively at run time, which is how the APL*PLUS II APLGUI does it, is horrendously slow. Windows has to allocate resource individually for the form and for each of its controls (buttons, edit fields, scroll bars etc.) at the time of creation. However there is a class of window available whose definition contains the definition of all its controls and the creation of which is very fast. This is the dialog box. It is a little restricted in that it can contain only 255 controls, but obviously this design constraint is far outweighed for David by the speed of creation and the relative modularity of the code needed to manage it. It is also not possible for the APL program to change aspects of the definition (e.g. the height, width and number of edit fields) at run time.

The standard procedure with compiled languages is to define the dialog box with a resource editor and then store the compiled definition, firstly into a

resource file and then finally into the application's executable file. Instead of using a windows call to create each control in the box the application uses one call giving the name of the dialog box within the executable file; at which point windows creates the whole dialog box and all its controls in one fell swoop. This process is effectively what he has enabled his application developers to do from APL. He has produced a small set of APL utilities that read a dialog box definition from a resource file, call the windows function to create the dialog box from the definition and associate APL expressions with events on the dialog box.

The window definition is done in a standard dialog box editor from the Software Developer's kit or from Borland Resource Workshop. The result is saved to a resource file (with extension .res) and when needed the definition is read from the file into the workspace using `ⓘNREAD` and then used in a windows call to create the dialog box. The call is fast because all the hard work is done by the dialog box editor. Furthermore, as the definitions are held on file and only retrieved when they are needed for creation, it is possible to have the resource editor running at the same time that the application is running and to make small changes to the appearance of the boxes without halting the APL execution.

For a production system in which no change to the dialog boxes is envisaged the definitions can be compiled into the APL Agent executable file (the windows task that communicates with the APL interpreter) by the SDK resource compiler. This reduces the number of application files and means that the definition need no longer be read from file and passed to the 'create dialog box' API routine. Instead a slightly different routine is used that is passed the name of the box, which it then reads direct from within the Agent. This saves even more time as the definition does not have to be read from file.

The behaviour on a user interacting with a control is defined using the `APLWIN` function `W_CreateFilter`, which associates a line of executable APL with a windows event, on a control. Whenever the event happens to the control the line gets executed. This is very flexible in contrast to the `APLGUI` where one needs to have one function per event per control per form, a restriction which can cause a vast number of one line functions in the workspace, but which will however be lifted in `APL*PLUS III`. It also gives access to all the messages that Windows passes instead of having them filtered by a well-meaning middle layer.

To conclude, this is a good fast solution to the problem of the Windows interface from `APL*PLUS II`. It will be interesting to see if `APL*PLUS III` addresses the problem of speed and modularity to the extent that David thinks that it is viable. There is no doubt though that he feels happier working close to the metal and relying on his own code. The results were certainly impressive.

A First Look at APL*PLUS III James Wheeler at Cocking & Drury 10th December 1993

notes by Adrian Smith

Introduction

This was a one-day stop on James' hectic European tour. He used the time very effectively; in around three hours we were given a clear understanding of how Manugistics see the Windows market moving, how they feel their APL should fit, and what it will look like. What he had to show us was 'APL*PLUS III as it stood at 5.00am last Saturday' — and a remarkable amount of it actually worked! What follows is more or less a verbatim transcript of James' presentation — *my comments are in italics.*

Background

Part of Bill Gates' plan to rule the world involves turning Windows into a modern operating environment. This means the WIN32 API, which is important for APL, because all good APLs need 32 bits! We already have WIN32s (a subset to allow Windows 3.1 to run 32-bit applications), and Windows NT is just the beginning:

"The future is the WIN32 API" (Microsoft VP)

What I am going to show you is the first APL development system to run on this platform. APL*PLUS III for Windows is the first of a series of products — beta release due mid-January 94, with expected final release date in the first half of the year. What you will see is the APL as it stood 5.00am Saturday — I understand it's made considerable progress since then.

Design Objectives

Assuming someone already knows APL (I'm not going to address that issue just yet) and Windows — can they become functionally literate with this product without using the manuals?

A fast, tidy development system. Users of APLGUI said "too slow", "too much junk in the workspace" and I agree with them. This afternoon I'm going to show you a GUI running with no functions **at all** in the workspace!

An "open" APL that supports and encourages integration. Software is becoming a set of components (from various vendors) that the application programmer (or even the user) can connect together:

- DDE links
- ODBC connectivity to all major databases
- extensible GUI toolkit with 3rd-party objects
- all those DLLs (not just the ones that come with Windows) are there to be called.

The follow-on product will be APL itself packaged as a DLL so that it can easily be called from C++, Visual Basic, and so on. Clever Excel programmers will make APL appear as just another Excel 'Add-in'.

As part of this policy:

- no dongles
- free unfettered runtime

Although it is being treated as a 'new' product, there will be a migration policy for existing APL*PLUS II users.

The Language

At the bottom of the heap is the standard APL*PLUS II interpreter, but stripped of:

- the inbuilt keyboard driver
- character screen IO (`⎕WPUT`, `⎕WIN` and so on)
- serial port handling (`⎕ARBIN` etc)
- sound and graphics

... i.e. anything 'close to the metal' which must be left to Windows to manage. The immediate benefit is a dramatic reduction in the weight of documentation — around 2" of manuals replace the current APL*PLUS II package which is nearer 7" in thickness.

The only major addition to the language comes as simple control structures:

```

:if stuff>100                :select Allocn
  do something here          :case 'Gilling'      ◊ do this
:else ◊ 'other stuff'         :case 'York'        ◊ do that
:endif  a or just :end        :caselist 'APL' 'GUI' ◊ etc
                                :else                ◊ default
:for ct :in 112               :endselect
  xx+ /data[ct;]
:endfor

:while i>0                    :repeat
  process i                   do something
:endwhile                     :until 0=ρdata

```

*I suppose we should all stop beefing about this subject now! The way APL*PLUS III implements these is as non-invasive as anything I have yet seen, so please will everyone else copy this! We can choose to ignore it all, and carry on branching, or use it where it helps clean up messy code. Either way, it has to be a good decision, and all credit to Manugistics for taking the plunge and pioneering this.*

At the moment the GUI support is all 'external' to the core language definition, and is driven through ρ functions. However, this too will be fully integrated, as part of a general move to make the interpreter more aware of 'objects'.

How the GUI Support Works

The 'object approach' is fundamental — each object is a self-contained control structure containing functions and data. This is essential to hide the complexity and minimise interactions. The essence of an object is that other software cannot see inside it! Objects are accessible only through a well-designed and documented interface.

Some terms which need to be defined:

- **FORM** Very much like a mainframe 'screen'.
- **CONTROL** In the trivial case, like a mainframe field, but potentially much richer. Think of the controls on the dashboard of a car.
- **METHOD** This is something you can ask an object to do. Kind of like an APL function, in fact an object is kind of like a tiny APL workspace!
- **PROPERTY** Very like a shared variable. If you assign a value to the 'caption' property, the visible picture on the object changes.
- **EVENT** Something which can be signalled on a object. Event-handling is the heart of GUI development. An event is a notification from the system that may invoke an ...

- **EVENT HANDLER** A bit like an Error-trap, but much easier to specify and use.

APL*PLUS III is itself an event-driven interpreter — it is continually waiting for 'events' from Windows. These may cause the interpreter to do something, or may cause parts of your application to respond.

Building a GUI Program

1. Structure all application code to reflect basic user actions
2. Define user-interface objects to invoke the actions
3. Use a visual editor to draw it
4. For each 'control' decide which things the user might do, and write handlers to call (1).

The program executes very simply ...

1. Initialise all necessary data, forms etc.
2. Show the form(s)
3. Sit back and wait. The user may do something.

The tools in this release of APL*PLUS III are:

`□WI` — short for Windows Interface. This handles all the interactions with the GUI objects ...

```
'object' □WI 'new' 'class' ('prop' value) { ... } ⍝ make one
qq+ 'object' □WI 'property'           ⍝ reference a property
'object' □WI 'property' value         ⍝ set a property
```

These last examples will be true reference and assignment — come back next year! In addition, there are some extra `□`functions to help with the processing of events. These are:

<code>□wself</code>	during a callback, this is who invoked you
<code>□wevent</code>	what the event name was (e.g. 'click')
<code>□wargs</code>	and additional details (where the mouse was)
<code>□wres</code>	the result of the callback (you might reject it)
<code>□wgive</code>	yield to Windows during very heavy APL processing
<code>□wcall</code>	interface to Windows DLLs
<code>□NA</code>	interface to anything else.

As a simple example ...

```
name+'myfm' □WI 'New' 'Form'      A create a new form
name+'myfm.b1' □WI 'New' 'Button' ('where' x y w h) A add child
```

... and so on. Anything can be a child of anything ... even buttons on buttons if that is what you want! The size and position are defined in terms of the 'base font' average character size. If necessary, you can set the base font on a 'system' object which notionally owns all the top-level forms, and hence control this globally.

Active (open) forms live in dynamic Windows memory. When a form is closed, it can be stored in the workspace as a 'magic' variable (effectively a vector of machine code) which can be assigned (but not edited), stored on file etc.

A form has some other helpful properties:

- **Children.** The list of all child objects — so you can document them hierarchically.
- **State.** The list of all properties which have a non-default value.
- **Class.** What sort of animal this is.
- **Properties.** A full list of everything it can know about itself.
- **Events.** A catalogue of the events it can respond to. (built-in events only — APL*PLUS III does not have user-defined events).
- **Methods.** A list of all the things it knows how to do.

A system object '#' owns all forms, which may in turn own other forms or controls. This is not yet 'MDI' but it will be in a future release. However you can easily program Unix-style 'widgets' — i.e. intelligent forms which re-arrange themselves to show the essential bits when re-sized.

Event Handlers

Every class has a list of things it will listen for: 'Click' 'Keypress' and so on. You simply set the 'onEvent' property of the object to an APL expression, and it runs the expression whenever it receives that event. Note that this is **any APL expression** — very different from the APLGUI approach!! No longer do you need a workspace full of carefully-named 1-line functions.

Graphics and DDE

All visible objects have a 'draw' method, which itself has a simple graphics language:

```
'photo.image' □WI 'Draw' 'Bitmap' (732 32p15)
```

Forms can act as DDE servers (they have a 'ddeTopic' property which might be set to 'excel|mysheet.xls') and controls can be DDE clients ('ddeItem' could be 'R2:C3'). There is an 'onChange' event which is used to notify controls that the data which they are showing has changed.

Extending the Base System with .APX libraries

"Totally shameless theft is a very important design principle. There are lots of good ideas out there — if you like it, rip it off!"

Visual Basic has a .VBX, essentially a DLL by another name. So why shouldn't APL have a .APX? This would be a repository for callable APL functions as well as GUI definitions which would be seamlessly added to the existing GUI classes. Manugistics will release the specification of the .APX, and will also repackage existing 3rd-party toolkits (e.g. the bits-per-second graphics server) in this form. Maybe there will even be a 'universal adaptor' so that APL systems can hook up directly to .VBX libraries?!

Where Next?

The next major release will include 'capsules', which will add true objects to APL. Objects currently accessed through `⌊NT` will become capsules accessed through the normal APL primitives:

```
form.button.where←23 34
```

... but capsules will provide much more than this ... essentially they are an arbitrary namespace where applications and modules can reside. The capsule itself won't be a `⌊NL` object, but something called a 'capsule reference' will be.

What Does it all Look Like?

A very standard MDI frame, in which reside the APL session, multiple edit sessions, and the debug and trace tools (unfortunately not yet hooked up). The APL*PLUS II debugger in particular is "one of the most overlooked advances in APL technology for a long time" and fits well into a Windows look and feel.

Summary

Not everything is fully hooked up yet, but what is there all works, and this new 32-bit WIN32s code runs very fast. Speed comparisons with APLGUI are unfair, but you can expect at least a 15-fold improvement in the GUI, and a very competitive interpreter to back it.

Commentary by Adrian Smith

There is no doubt that Dyalog really scared the wits out of Manugistics at Toronto! The result has been a U-turn of quite startling speed — out goes INCA (whatever that was) and in comes APL*PLUS III at the gallop. Basically, everything we saw looks good, and the design has improved immeasurably over the APLGUI that Duncan struggled with at BAA-GUI. It is comforting from an application-builder's point of view that Manugistics have quite shamelessly copied all the best ideas from the Dyalog APL GUI, and may even have improved on some of them. This should mean that everything we now have in Dyalog we could easily port to APL*PLUS III as soon as the finished product hits the streets. This level of cross-platform compatibility must be good for APL.

So, where does this leave the current APL*PLUS II developer? Here are some points to consider:

- Manugistics are almost exactly a year behind Dyalog. I was working on the first beta release of the full Dyalog GUI early in January. Most of it functioned correctly, and the bugs have got steadily more obscure as time has passed. However good the Manugistics programmers, APL*PLUS III is unlikely to reach stability until early 1995.
- The APL*PLUS II (for DOS) session and debugger are currently ahead of anything Dyadic have to offer. If these are done really well in Windows, they will give APL*PLUS III a genuine edge. I really liked the fact that the session stays active while your forms run — this is when you most need a debugger!
- the issue of porting code from APL*PLUS II is almost exactly as big for APL*PLUS III as for Dyalog. Anything which touches the screen or listens to the keyboard (or plays tunes or draws graphs) will need a bottom-up rewrite. At the end of this process, the GUI bits should be portable anyway!

I think the overall conclusion has to be that GUI support will no longer be a strong discriminant. Now is undoubtedly the time to evaluate thoroughly both Dyalog and APL*PLUS III, but you should be more interested in raw APL speed, developments in the base language (control structures and 'capsules'), connectivity and the developer support than in the implementation of the Windows GUI.

On final whinge — please will both Dyadic and Manugistics stop being APL bigots and use 'New', 'Open', 'Save' and 'Exit' on their file menus instead of 'Clear' and so on. How many APL programmers are there who don't know Excel? OK, how many Excel programmers are there who don't know APL?? Enough said.

When is Easter?

by Ray Cannon

"There are many indications that the sole important application of arithmetic in Europe during the Middle Ages was the calculation of Easter date" (Knuth)

Easter, the major feast day of the Christian calendar, celebrates the anniversary of Christ's resurrection. This is documented as having occurred on the Sunday following the Last (Passover) Supper. In the Hebrew calendar which is strictly lunar based, Passover is celebrated on the full moon of the 7th month, Nisan, which begins with the first appearance of the crescent spring (equinox) moon. So Passover should be on the first full moon after the spring equinox: the 14th day of Nisan.

Thus many early Christians (but not all) celebrated Easter on the 14th of Nisan, but by the 4th century, this had led to several problems.

Problems

The Christian (Roman) world used the Julian calendar (developed at Julius Caesar's request by Sosigenes in 46BC from an earlier Egyptian calendar). This is a solar calendar with no lunar base.

The Hebrew calendar did not observe the equinox with any precision. This made it difficult for far off communities to know when the month of Nisan had started. By the 4th century, the rift between the Christians and the Jews had grown to such an extent that the Christian Church could not allow Easter to fall on its Hebraic equivalent Passover.

The Church now insisted that Easter must be celebrated on a Sunday. Christ's resurrection is documented in Luke 24 as having occurred on the Sunday morning. This was the reason that Christians adopted Sunday as the day of rest in preference to Saturday, the Jewish Sabbath. As a result, Easter was celebrated on different dates in different Christian communities. In particular, three methods were in use, the Alexandrian, the Roman, and the Quartodecimani (based on the 14th of Nisan).

Decision Time

In 32AD the Council of Nicaea (Iznik in Asia Minor) had, as one of its goals, to set a single date for the celebration of Easter by Christians in both Eastern and Western Roman Empires. The "COMPUTISTS" who were specialists in charge of Christian calendric computations, came up with the following rule: "Easter Sunday is to be set on the first Sunday following the full moon that occurs on or after the Vernal Equinox."

The actual definition refers to the "14th day after the new moon" rather than the full moon. This is useful, as it removes "rounding problems" of trying to pinpoint the exact time and date of the full moon. At this time, the day was deemed to start at sunset (not midnight). A new moon appears first low in the western sky just after sunset, and is full 14 nights later. Under the Julian calendar, the Vernal Equinox was deemed to occur on the 25th of March (and was the start of the new year, according to some).

The computists drew up long-winded tables based on average full moon intervals. These tables called Epacts listed the 'calendric' age or phase of the moon on New Years Day. From these tables the date of Easter could be readily calculated. (For examples see the 1662 Book of Common Prayer.)

The first systematic algorithm for the Easter date appeared in the "*canon paschalis*" due to Victorius of Aquitania in 457 A.D. He made use of the 532-year cycle over which date, moon phase and day of the week re-occur in the Julian calendar.

More Problems

In 730AD The Venerable Bede announced that the 365.25 day long Julian year was 11 minutes, 14 seconds too long, making a cumulative error of about 1 day every in 128 years. By the 16th century, the Vernal Equinox was 11 days out of step with the Julian calendar.

In 1582 Pope Gregory XIII appointed a commission to reform the calendar, and bring Easter back to its proper place in the seasons. This resulted in two steps being taken: first, 10 days were dropped from the calendar (Sunday, 4th October 1582 was followed by Monday, 15th October); and secondly the rules relating to leap years were reformed (a century would be a leap year only if divisible by 400). Thus the Gregorian calendar was created. Great Britain and its colonies did not adopt this calendar until 1752, and Russia not until 1917.

With the Gregorian calendar, the average length of the year (over long periods of time) is 365.2425 days, which is only 0.0003 longer than the tropical (astronomical) year, resulting in an error of 1 day in 3,300 years. (Minor reforms to the Gregorian calendar, relating to the years 4000, 8000 and 12000, and century leap years divisible by 900, made this century, have reduced this error to about 1 day in 44,000 years. Further accuracy appears to be futile, since the tropical year is itself changing.)

Under the Gregorian calendar, the vernal equinox occurs on (or around) the 21st of March. With this definition, Easter can occur any day between March 22 and April 25 inclusive.

The Julian Period

In 1582, the Julian Period (not to be confused with the Julian Calendar) was devised by Joseph Scaliger, and named after his father Julius.

Scaliger had Julian Day number one (JD #1) begin at noon January 1st 4713 B.C. All other dates are referred to by their subsequent day number, so Noon of Dec 31 1989 was JD# 2447892. Use of the Julian Period makes the calculation of the number of days between two dates a very simple matter. Astronomers use the Julian Period to this day with a decimal part representing the time of day. (Many computer systems mis-call the day number within the year the Julian Date.)

The choice of 4713 B.C. for the start point was that Scaliger believed it to be the most recent time that 3 major cycles began on the same day. These cycles are the 28-year Solar cycle, the 19-year Metonic cycle, and the 15-year Indiction cycle (used by the Romans to regulate taxes). Sometimes the modified Julian Date (MJD) is quoted. This is equal to JD-2400000.5, so MJD zero begins on November 17 1858.

```

V r←julan2date JD;I;F;A;B;C;D;E;G;d;m;y;X;S
[1]  *Convert Julian day number to Gregorian date in CCIYMMDD format
[2]  *Note on BC dates. 1BC is year 0 and 2BC is year -1
[3]  * so 31 Dec 1 BC and 31 Dec 2 BC are entered as 1231 and -11231
[4]  X← 0 1 T,JD+0.5
[5]  A←int(X[1;])-1867216.25)+36524.25
[6]  B←X[1;]+(X[1;]>2299160)*1+A-int A+4
[7]  C←B+1524
[8]  D←int(C-122.1)+365.25
[9]  E←int D*365.25
[10] G←int(C-E)+30.6001
[11] d←C-E+int G*30.6001
[12] m←100×G-1+12×d>13.5
[13] y←10000×D-4715+m>250
[14] S←(xy)+0=xy

```

```

[15] r←(pJD)oS*(|y)+m+d+X[2;]
      v

      v r←julian2dov JD
[1] aReturn DayOfWeek number for given Julian Period day number
[2] r←|0.5+7*1r(12+JD)+7
      v

      v r←date2julian date;ymd;y;m;d;a;b;c;D
[1] aConvert Gregorian date in CCYYMMDD format to Julian period day number
[2] aNote on BC dates. 1BC is in year 0 and 2BC is in year -1
[3] aSo date of 31 Dec 1 BC and 31 Dec 2 BC are 1231 and -11231
[4] ymd← 0 100 100 r.|date
[5] y←ymd[1;]**date
[6] m←ymd[2;]
[7] d←ymd[3;]
[8] y←y-m<3
[9] m←m+12*m<3
[10] a←int y+100
[11] b←(date>15821015)*2+(-a)+int a+4
[12] c←int(365.25*y)-0.75*-1=x*y
[13] D←int 30.6001*m+1
[14] r←1720994.5+b+c+D+d
[15] r←(pdate)oS
      v

```

The Solar Cycle

This has nothing to do with the movement in the heavens of the Sun, but to do with the days of the week, the first of which is Sunday (*dies solis* in Latin).

Under the Julian Calendar, the days of the week repeated themselves on the same date over a 28 year cycle. 28 being the product of 4 (the year leap year cycle) and 7 (the days in a week).

The Metonic Cycle

19 solar years equal 235 lunar months. (To be precise 235.003.) This is the Metonic cycle. In this cycle, the moon will have the same phase on the same date after 19 years. i.e. 1st Jan 18AD was a new moon, so was 1st Jan 37, 19 years later. The Golden number of a year is its index value (origin 1) in the 19-year Metonic cycle. The Golden number for year 1990 is 15.

The *epact* is the "age of the moon at the start of the year". For example, the epact for 1990 was 3, hence the full moon occurred on the night of 11th of January 1990.

Functions

Most computer programs used to calculate the date of Easter are based on the algorithm due to Aloysius Lilius and Christopher Clavius in the 16th century. It is this algorithm that D. E. Knuth cites in several publications (D. E. Knuth *The Art Of Computer Programming*, vol 1, Addison-Wesley, 1968). Listings 1, 2 and 3 use this algorithm, (as does the *EASTER* function included in the Toronto Toolkit, part of the APL93 software exchange).

(Note. The Toolkit version 2.1 Easter function has a minor bug resulting in the date of 31st March being returned as 31st April. It also fails in 1583, returning the date of Easter in the Julian calendar, not the Gregorian.)

Here is my APL implementation:

```

V R←EASTER1 Y;G;C;X;Z;D;E;N;M
[1]  A      LISTING-1
[2]  ACompute the date(s) of Easter for the given Year(s)
[3]  AAuthor Ray Cannon 10/10/1993
[4]  ABased on the 16th century algorithm developed by the Neapolitan
[5]  Astronomer Aloysius Lilius and the German Jesuit mathematician
[6]  AChristopher Clavius.
[7]  AReference D.E.Knuth 'The art of Computer Programming' vol 1.
[8]
[9]  A Y←Year(s) in the Gregorian Calendar (I.E. after 1582). Any rank
[10] A R←Dates in CCYYMMDD format. (pR)←(pY)
[11]
[12] A Easter is to be held on the 1st Sunday following the first full
[13] A moon which occurs on or after 21st March, as calculated from
[14] A defined tables.
[15]
[16]  G←1+19|Y A      Golden Number
[17]  A G is the so called 'Golden Number' for the year Y in the 19 year
[18]  A Metonic Cycle. Every 19 years (according to the Julian Calendar)
[19]  A the moon has the same phase on the same date.
[20]
[21]  C←1+|Y+100 A      Century number
[22]  A Note that the year 1900 is taken to be in the 20th Century
[23]
[24]  X←-12+|3×C+4 A      Gregorian Correction
[25]  A X is the correction for the number of leap year days dropped by
[26]  A Gregorian calendar to keep in phase with the sun
[27]
[28]  Z←-5+|(5+8×C)+25 A      Clavian Correction
[29]  A Z is the correction for the Metonic cycle which is out by
[30]  A 8 days over 2500 years
[31]
[32]  D←(|5×Y+4)-X+10 A      Date of Sunday
[33]  A March the ((-D)mod 7) in year Y is a Sunday
[34]
[35]  E←30+|(11×G)+20+Z-X A      Approximate Epact
[36]  A The Epact is the 'age' of the moon at the start of the year Y
[37]
[38]  E←E+(E=24)∨(E=25)∧G>11 A      Church Epact
[39]  A E is the value of the Epact found in the tables used by the Church.

```

```

[40] a It can be out by up to 3 days in some years.
[41]
[42] N+44-E a Find a full moon in year Y
[43] a Note that 44 days (44.295885 actually) represents 1.5 lunar months
[44]
[45] N+N+30*N<21 a Full moon after March 21
[46] a Add 1 month (30 days) if full moon is before the Vernal Equinox
[47]
[48] N+N+7-7|D+N a Next Sunday
[49] a Advance from full moon to following Sunday
[50]
[51] M+3+N>31 a Month number
[52] a March the 3rd month has 31 days
[53]
[54] N+N-31*M=4 a Day number
[55] a Adjust day if in April
[56]
[57] R+N+100*M+100*Y a Date of Easter
[58] a Date in CCYYMMDD format
v

```

For a J implementation of this algorithm, see Gene McDonnell's letter to me, reproduced on page 76. For a little light relief, here is a debased form of Adrian Smith's code that first appeared in Vector's fore-runner "Crab Apple". Adrian described his original (reproduced on page 75) as written in rhyming couplets. This one is more akin to cockney rhyming slang.

```

R+EASTER3 Y
[1] a LISTING-3
[2] a Compute the date(s) of Easter for the given Year(s)
[3] a Author Ray Cannon 10/10/1993
[4] a Based on a 20th century APL program by Adrian Smith
[5] a Reference Adrian Smith 'Crab Apple'
[6]
[7] a Y+Year(s) in the Gregorian Calendar (I.E. after 1582) Any rank
[8] a R+Dates in CCYYMMDD format (pR)+-(pY)
[9]
[10] a Note. Adrian Smiths origional was written in 'rhythming cuplets'
[11] a This version is in cockney rhyming slang.
[12]
[13] R+{17,p,Y}p0
[14] R[17;]+,Y
[15]
[16] R[ 2 3 ;]+ 0 100 r0 + 1 0 1 1 *.+R[1 17 16 16 ;]
[17] R[ 4 1 ;]+ 0 19 r0 + 1 1 5 1 +.R[1 3 2 16 ;]
[18] R[ 4 5 ;]+ 0 4 r0 + 1 25 3 1 +.R[6 2 6 16 ;]
[19] R[ 8 9 ;]+ 0 4 rR[3;]+ 1 60 5 1 +.R[6 6 5 16 ;]
[20] R[ 6 15 ;]+ 0 25 r0 + 1 11 8 1 +.R[6 2 16 16 ;]
[21] R[15 7 ;]+ 0 30 r0 + 1 19 -1 1 +.R[4 1 6 16 ;]
[22] R[11 15 ;]+ 0 319 r0 + 1 11 1 1 +.R[1 7 16 16 ;]
[23] R[15 10 ;]+ 0 7 r0 + 1 -1 -1 2 +.R[11 9 7 8 ;]
[24] R[12 14 ;]+ 0 30 r110 + 1 -1 1 1 +.R[7 11 10 16 ;]
[25] R[15 13 ;]+ 0 33 r5 + 1 -1 1 1 +.R[14 12 16 16 ;]
[26] R[15 17 ;]+ 0 0 r0 + 1 1 100 10000 +.R[13 16 12 17 ;]
[27]
[28] R+(pY)pR[17;]

```

Listing 4 uses a different method devised in 1876, which first appeared in Butcher's Ecclesiastical Calendar. Its internal working is however undocumented. Can anyone tell me how it works?

```

V R←EASTER4 Y;GN;Z;CC;T;FM;DN;MD

[1] A LISTING-4
[2] A Compute the date(s) of Easter for the given Year(s)
[3] A Author Ray Cannon 10/10/1993
[4] A Based on a 19th century algorithm defined in
[5] A Butcher's Ecclesiastical Calendar (1876)
[6] A Reference P.Duffett-Smith 'Practical Astronomy with your calculator'
[7]
[8] A Y←Year(s) in the Gregorian Calendar (I.E. after 1582) Any rank
[9] A R←Dates in CCYYMMDD format (pR)←(pY)
[10]
[11] GN←19|Y A Golden number-1
[12] Z←0 100 T,Y A Split CCYY into CC and YY
[13] CC←Z[1:] A Save CC
[14] Z←0 4 TZ A Leap years and Leap Centuries
[15] T←(1+CC-10.04×8+CC)÷3 A ?Clavian and Gregorian correction
[16] FM←30|15+CC-T+Z[1:]-19×GN A ?Full moon
[17] DN←7|32-FM+Z[2:]-2×Z[1:2;]+Z[2:1:] A ?Day number of week
[18] T←7×[(GN+11×FM+2×DN)÷451] A ?
[19] MD←0 31 T114+FM+DN-T A MM and DD of easter
[20] R←(pY)p1+MD[Z:]+100×MD[1:]+100×Y A Date of Easter in CCYYMMDD format
V

```

I have tested these APL functions against each other, and they produce identical results for a 4000-year period starting from 1583. They have also been verified against the dates printed in the World Almanac 1990 for the date of Easter from 1901 to 2100.

Listing 5 is my implementation of the original rules. It uses utilities to convert between calendar dates, day numbers and day of the week numbers. It produces some WRONG results.

```

V R←LISTINGS year;dates;Equinox;ANewMoon;DaysInYear;DaysInMonth;
      DaysBetween;MonthsBetween;EquinoxInYear;NewMoon;
      FullMoon;SpringMoon;DayOfWeek

[1] AReturn 'astronomically correct' date of Easter for the given Year(s)
[2] AHEALTH WARNING-This function does not return the TRUE date of EASTER
[3]
[4] AAuthor Ray Cannon 10/10/1993
[5] ABased on the definition:
[6] A 'Easter falls on the first Sunday after the first
[7] A full moon on or after the Vernal Equinox'
[8]
[9] A year←Year(s) in Gregorian calendar
[10] A r←Date(s) of Easter in CCYYMMDD format
[11]
[12]
[13] AUtilities used:
[14] A julian2date←Returns the date in CCYYMMDD.TIME format from julian day

```

```

[15] A      number. Time of day specified as a decimal.
[16] A date2julian↔Returns the Julian day number for a date given
[17] A      in CCYYMMDD.TIME format.
[18] A julian2dow ↔Returns day of week number for given julian day number
[19] A      0↔Sunday, 6↔Saturday
[20] A j2d      ↔Rounds Julian day number to 'start' of day.
[21] A      Julian period days start at midday
[22] A      j2d assumes days start at 6pm, that is sunset.
[23] A int      ↔returns the integer part of its argument, with the
[24] A      correct sign. eg -4↔int -4.5
[25]
[26] A Define some constants
[27] dates=date2julian 19900320 19900225 +(19 54 +60× 21 8)+1440
[28] Equinox+dates[1] A      Equinox on 20 March 1990 at 21:19 GMT
[29] ANewMoon+dates[2] A      New moon on 25 Feb 1990 at 8:54 GMT
[30] DaysInYear+365.2422 A      Number of days in the tropical year
[31] DaysInMonth+29.53059 A      Number of days in the lunar month
[32]
[33] A Calculate the time between 1990 and the given year(s)
[34] DaysBetween+DaysInYear×year-1990 A      Days between
[35] MonthsBetween+DaysBetween÷DaysInMonth A      Months between
[36]
[37] A Find (the start of) the day of the spring Equinox in the supplied year
[38] EquinoxInYear+j2d Equinox+DaysBetween
[39]
[40] A Find a New Moon around the equinox just found
[41] NewMoon+ANewMoon+DaysInMonth×(MonthsBetween
[42]
[43] A Find the Full Moon following the New Moon just found
[44] FullMoon+NewMoon+14
[45]
[46] A Find next full moon if this one is before the equinox
[47] SpringMoon+(FullMoon+DaysInMonth×EquinoxInYear>j2d FullMoon-1.5
[48]
[49] A Find the day of the week the Spring Full Moon falls on
[50] DayOfWeek+julian2dow j2d SpringMoon
[51]
[52] A Return the Sunday after the Spring Full Moon
[53] r+(julian2date 0.5+j2d SpringMoon+7-DayOfWeek

```

v

It works by using the known exact length of the tropical year and lunar month, along with the exact date and time of the spring new moon and equinox in 1990, to calculate (via Julian day numbers) the exact date and time of the equinox and spring new moon in the desired year. From these it calculates the appropriate full moon and following Sunday. This method is astronomically accurate, but is not quite 100% ecclesiastically correct. As Johannes Kepler is reputed to have said "Easter is a feast, not a planet". Problems like converting (rounding) the instance of a full moon (which occurs at midnight local time) to a (GMT) date or day of the week, lead to some differences. Question: Should we be using GMT or Rome or Jerusalem local time?

Other differences can be attributed to inaccuracies in the Epact table (which can often be 2 to 3 days out) used to define under English law when Easter occurs.

One final question which one day may face the church: How do you fix the date of Easter on a different planet?

For further commentary see:

Puzzles and Paradoxes by T. H. O'Beirne. Oxford University Press 1965.

Counting the Eons by Isaac Asimov. Grafton Books 1984.

Practical astronomy with your calculator by Peter Duffett-Smith.
Cambridge University Press. 3rd edition 1988

Stonehenge Decoded by G.S.Hawkins. Doubleday and Co. Inc, 1965. (Within chapter 9 Eclipses, he describes how to fix the date of Easter Sunday using Stonehenge as the calculator.)

The Crab-APL original ...

P.S. Here is a bit of graffiti which might possible have some practical value. It is an attempt to cast a very old algorithm indeed (1660 approx) into executable rhyming couplets. The result is valid up to around 4000 AD or until the churches finally get round to fixing Easter!

The original of this is in 'Puzzles and Paradoxes' by T M O'Beirne (OUP). It executes in about 50ms for 100 years of data, which compares rather well with an old PL/1 version which took about 5 sec to compile and 1.3 sec to run!!

```

R+EASTER YR;X
[1]  *COMPUTE THE DATE OF EASTER FOR GIVEN YEAR(S)
[2]  * <R> CONTAINS THE MONTH AND DATE OF EASTER SUNDAY
[3]  * ** SAVED ON 7/07/82 AT 15.03
[4]  X←(15.ρ,YR)ρ0
[5]  X[ 2 3 ;] + 0 100 T ,YR
[6]  X[15 1 ;] + 0 19 T 5 1 +.× X[2 3 ;]
[7]  X[ 4 5 ;] + 0 4 T 3×25×X[2;]
[8]  X[ 6 15 ;] + 0 25 T 8×11×X[2;]
[9]  X[15 7 ;] + 0 30 T 19 1 -1 +.× X[1 4 6;]
[10] X[11 15 ;] + 0 319 T 1 11 +.× X[1 7;]
[11] X[ 8 9 ;] + 0 4 T X[3;]+60×5-X[5;]
[12] X[15 10 ;] + 0 7 T 2 -1 -1 1 +.× X[8 9 7 11;]
[13] X[12 14 ;] + 0 30 T 110+ 1 -1 1 +.× X[7 11 10;]
[14] X[15 13 ;] - 0 33 T 5+ 1 -1 +.× X[14 12;]
[15] R←X[13 12;]

```

V

Letter from Eugene McDonnell

1993 September 4

Ray Cannon
21 Woodbridge Road
Blackwater
CAMBERLY
Surrey GU17 0BS
United Kingdom

Dear Mr. Cannon,

At APL93 you asked me if I knew an algorithm for the date of Easter. Here is one I've carried with me for many years. If you transcribe it into APL you have to make the following ASCII-to-APL changes:

ASCII APL
NB. lamp
=. left arrow
<. floor
* times
_ overbar
+. or
*. and

If you want to use it in J, it needs no alterations. As you can see, J and APL are closely related.

The comments are ample, but could be elaborated even more. There is a great deal of astronomical and ecclesiastical history in these few lines. In the old *Encyclopedia Britannica* article "Calendar" you can find some more interesting details. According to Donald Knuth, "There are many indications that the sole important application of arithmetic in the Middle Ages was the calculation of Easter date, and so such algorithms are historically significant." Here it is:

NB. $z =$ Easter y (function header)

NB. y is the number of a Gregorian year.

NB. z is the month and day on which Easter Sunday falls in that year.

NB. For example, Easter 1989 is 3 26. Easter falls on the first

NB. Sunday following the first full moon which occurs on or after

NB. March 21. See Augustus DeMorgan, "A Budget of Paradoxes", and

NB. Donald Knuth, CACM 5, 4, April 1962, "The Calculation of Easter...".

NB. and his "Fundamental Algorithms", Section 1.3.2, problem 14.

NB. a is the Golden Number, the number of the year in the 19-year

NB. Metonic cycle, used to determine the position of the Moon. (The

NB. phases of the Moon have a cycle of approximately 19 years).

$$a = .1 + 19 | y$$

NB. c is the number of the century in which y falls. This algorithm

NB. accepts the year $xx00$ as occurring in the $xx+1$ th century.

$$c = .1 + <. 0.01 * y$$

NB. g is the Gregorian correction, the number of preceding years

NB. like 1700, 1800, 1900 when leap year was not held.

$$g = ._{12} + < . 0.75 * c$$

NB. v is the Clavian correction, a correction of the Metonic cycle

NB. of about 8 days every 2500 years, used to synchronize Easter

NB. with the Moon's orbit.

$$v = ._{5} + < . 0.04 * 5 + 8 * c$$

NB. x is the extra days specifying when Sunday occurs in March.

NB. (March 7) - x will be a Sunday).

$$x = . (_{10} + < . 1.25 * y) - g$$

NB. p is the approximate age of the calendar moon at the beginning

NB. of the year.

$$p = . 30 | 20 + v - g - 11 * a$$

NB. e is the epact, a better approximation than p .

$$e = . p + (p = 24) + . (p = 25) * . a > 11$$

NB. t is the day of the full moon, calculated using s .

$$s = . 44 - e$$

$$t = . s + 30 * s < 21$$

NB. u is the day of the following Sunday, but may be greater than 31.

$$u = . t + 7 - 7 | x + t$$

NB. d is the day of Easter Sunday.

$$d = . u - 31 * u > 31$$

NB. m is the number of the month, 3 (March) if u was less than 32,

NB. and 4 (April) otherwise.

$$m = . 3 + u > 31$$

NB. z is the month number and day of Easter in Gregorian year y .

$$z = . m , d$$

Yours truly,

Eugene McDonald

A Note on the Easter Algorithm in J

by Keith Smillie (smillie@cs.ualberta.ca)

The date of Easter Sunday for any year after 1582 can be determined by an algorithm due to the astronomer Aloysius Lilius and the Jesuit mathematician Christopher Clavius in the late 16th century. In the following version of the algorithm, which is taken from Knuth (1968), each step is given in conventional notation and is followed by the corresponding J verb or verbs:

Golden number of the year Y in the 19-year Metonic cycle:

$G \leftarrow (Y \bmod 19) + 1$
 G=. >: @ (19&|)

Century

$C \leftarrow \text{Floor}(Y/100) + 1$
 C=. >: @ <. @ (%&100)

Corrections. X is the number of centennial years in which leap year was dropped, and Z is a special correction to synchronize Easter with the moon's orbit:

$X \leftarrow \text{Floor}(3C/4) - 12$ $Z \leftarrow \text{Floor}((8C + 5)/25) - 5$
 X=. _12&+ @ <. @ (4: (%~) 3: * C)
 Z=. _5&+ @ (<. @ (25&(%~) @ (5: + 8: * C)))

Find Sunday:

$D \leftarrow \text{Floor}(5Y/4) - X - 10$
 D=. _10&+ @ (X (-~) (<. @ (4: (%~) 5&*))

Epact which specifies when a full moon occurs:

$E \leftarrow (11G + 20 + Z - X) \bmod 30$
 If $E = 25$ and $G > 11$, or if $E = 24$, then increase E by 1.
 E1=. (30&|) @ ((11&* @ G) + (20&+ @ (Z - X)))
 E=. E1 + 24&= @ (E1 @]) +. 25&= @ (E1 @]) *. 11&<@ (G@])

Find full moon:

$N \leftarrow 44 - E$ If $N < 21$, then increase N by 30.
 N=. (30&+) ^: (<&21) @ (44&- @ E)

Advance to Sunday:

```
N ← N + 7 - ((D + N) mod 7)
Sunday = . 7 : + N - (7 &| @ (D + N))
```

Get month:

```
If N > 31, then the date is April (N - 31); otherwise, it is March N.
Easter = . (> &31) , > : @ (31 &|) @ < : ) @ Sunday
```

The value of `Easter y`, where `y` is any year after 1582, is a two-item list giving the month (0 for March and 1 for April) and the day of the month of Easter for that year. For example, `Easter 1993` has the value 1 11 since Easter occurs on April 11, and `Easter 1997` is 0 30 since Easter occurs on March 30.

The use of the power conjunction `^`: for applying a verb conditionally in the verb `N` for finding the first full moon is due to McDonnell (1993). If we use his verb `if=`, `^`: the present verb may be expressed as

```
N = . (30 &+) if (< &21) @ (44 &- @ E) .
```

The verb:

```
EasterTable = . (20 5) &$ @ (< "1 @ (] , . "2 Easter "0) @ ((1.100) &+ @ ]))
```

gives a table with 20 rows and 5 columns of the dates of Easter for a one-hundred-year period starting with the specified year. For example, `EasterTable 1901` gives the dates of Easter for each year of the 20th century; the last two rows of this table given by `_2 5 { . EasterTable 1901` are

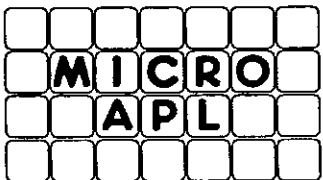
```
+-----+-----+-----+-----+-----+
|1991 0 31|1992 1 19|1993 1 11|1994 1 3|1995 1 16|
+-----+-----+-----+-----+-----+
|1996 1 7 |1997 0 30|1998 1 12|1999 1 4|2000 1 23|
+-----+-----+-----+-----+-----+
```

(A convenient reference for verifying the dates of Easter is *The Oxford Companion to English Literature* which gives in an Appendix the dates of Easter for each year from 1066 to 2000.)

References

Knuth, D. E., 1968. *The Art of Computer Programming*, Volume 1. Addison-Wesley Publishing Company, Reading, Mass.

McDonnell, E. E., 1993. *At Work and Play in the Fields of J.* Iverson Software Inc., Toronto.

—		APL Preferences		▽	
APL.68000					
Platforms		Versions			
☒ RISC System/6000		☐ Level I			
☒ Apple Macintosh		☒ Level II			
☒ Commodore Amiga					
☒ Atari ST					
APL for GUIs					

MicroAPL Ltd., South Bank Technopark, 90 London Road,
LONDON SE1 6LN, UK

Tel: 071 922 8866 Fax: 071 928 1006

Applelink: microapl Internet: microapl@applelink.apple.com

ASLGREG: Proposed Additional Features Predictions with Confidence Limits

by Ellis Morgan

I present a possible approach to adding predictions to the ASLGREG Regression and Generalised Linear Models Workspace. I include ideas to increase robustness and a method that allows *STEPWISE* to return a linear predictor equation that can be passed to *FIT* and other ASLGREG functions.

Many of the good ideas behind this are due to Alan Sykes and I am grateful for his help and his tolerance as I change his code and workspace structure. I propose that my extensions should be added to ASLGREG and welcome comments and suggestions for improvement.

ASLGREG

My descriptions of the following examples are sparse where they cover old ground, see: ASLGREG by Alan Sykes (Vector 7.4); the ASLGREG manual (available from BAA Administration); ASLGREG reviewed by Jon Sandles (Vector 9.2).

```

HOUSEDATA
pPRICE
29
PRICE
60500 47500 46000 110000 148000 65000 79950 54000 120000
85000 65000 55000 70000 82500 78500 49950 46950
46950 77000 102000 37500 68000 65000 145000 98000
75000 96500 53950 75000

```

The workspace is *ASLGREG*, *HOUSEDATA* sets up Alan's data on 29 houses. Other variables are *TYPE* (1 terraced, 2 semi-detached, 3 detached, 4 bungalow), *AGE*, *BEDS* (number of bedrooms) and *AREA*.

START

Regression Volume 1 Version 2

A.M.Sykes & E.P.Morgan, February 1993

Copyright(C) British APL Association 1991-1993

YVAR'PRICE+1000'

Y variable is PRICE+1000

Units set to 29

FACTOR'TYPE,BEDS'

1 LP'GM+AGE+BEDS+TYPE+AREA+TYPE.AREA+BEDS/TYPE'

TABLES 0

LPA+STEPWISE 3 3

LPA

GM+AREA+(TYPE=1)*AREA+(BEDS=4)*TYPE=3

STEPWISE has selected this model to estimate price and in the new version you can now write functions that call STEPWISE and go on to use the result in FIT and PREDICT. It can be dangerous to proceed from STEPWISE to PREDICT without thought but, provided diagnostics are displayed too, it is good to be able to do so. In the new workspace LP is ambivalent and the left argument, when present and equal to one, makes each item in the linear predictor an APL expression. With any other left argument the result is the same as in the original version.

TABLES 1

Output in Table form

FIT LPA

Source	ss	df	ms
-----	--	--	--
Due to model	1.851E4	3	6205
Residual	3.826E3	25	153.1
-----	-----	-----	-----
Total Corrected	2.244E4	28	801.5

Percentage Variation Accounted for = 82.95

F-statistic = 40.54

p-value= 0.00

DISPLAY 'ESTP'

Var	Est	Std Err	t-stat	p-val
---	---	---	---	---
GM	31.06	6.095	5.096	2.909E ⁻⁵
AREA	0.05541	0.007573	7.317	1.149E ⁻⁷
(TYPE=1)*AREA	-0.02236	0.005579	-4.007	4.861E ⁻⁴
(BEDS=4)*TYPE=3	38.67	8.292	4.664	8.906E ⁻⁵

This model estimates the average house price at £31060 plus £55.4 per square foot, but subtract £22.4 per square foot if it is a terraced house and add on £38670 if it is a four-bedroomed detached house.

```

+/(BEDS=4)*TYPE=3
3
+ /BEDS=4
6
+ /TYPE=3
5

```

There are only 5 detached houses (three of which are 4-bedroomed) and 6 four-bedroomed houses, obviously you would want to gather more evidence before using this predictor.

+8++DIAGNOSTICS 'XFO'					
Unit	AREA	(TYPE=1) *AREA	(BEDS=4) *TYPE=3	Fitted Value	Observed
----	-----	-----	-----	-----	-----
1	1326	1326	0	74.89	60.5
2	782	0	0	74.39	47.5
3	312	0	0	48.35	46
4	1460	0	0	112	110
5	1056	0	1	128.2	148

I have shown the first eight lines of the diagnostics, you will see that I am using Dyalog APL. Some new right arguments have been added to *DIAGNOSTICS*, *X* calls for all the explanatory variables.

PREDICTING from the original data

+8++PREDICT 'SYMUL'						
Row	AREA	BEDS	TYPE	Observed	Predicted	Upper Lower
----	-----	-----	-----	-----	-----	-----
1	1326	4	1	60.5	74.89	82.3 67.48
2	782	3	2	47.5	74.39	78.24 70.55
3	312	1	2	46	48.35	53.83 42.87
4	1460	3	3	110	112	120.3 103.6
5	1056	4	3	148	128.2	137.7 118.8

PREDICT is similar to *DIAGNOSTICS* in that the right argument specifies the columns to print. In this case I am calling for: *S* the explanatory variables before interactions, nesting and other equations; *Y* for the observed *Y* value; *M* for the predicted Mean value; *U* and *L* for the Upper and Lower confidence limits of the mean. The optional left argument of *PREDICT* is described below and specifies the confidence level (default is 80%) and the type of limit (default is independent for each prediction).

PREDICTING for a few extra observations

The simplest way is to add a few extra observations to your data, giving the response variable a value of zero and then keep them out of the fitting process by giving them a weight of zero. All real cases will have a weight of one.

```
PRICE+0 0,PRICE
TYPE+3 4,TYPE
BEDS+4 3,BEDS
AREA+1200 800,AREA
```

Two new "cases" at the beginning.

```
YVAR PRICE+1000'
Y variable is PRICE+1000
Units set to 31
```

```
W+0 0,29p1
PRIORWEIGHT 'W'
Prior Weight Variable is W
```

```
SINK+FIT LPA
DISPLAY 'ESTP'
```

Var	Est	Std Err	t-stat	p-val
GM	31.06	6.095	5.096	2.909E ⁻⁵
AREA	0.05541	0.007573	7.317	1.149E ⁻⁷
(TYPE=1)*AREA	-0.02236	0.005579	-4.007	4.861E ⁻⁴
(BEDS=4)*TYPE=3	38.67	8.292	4.664	8.906E ⁻⁵

The same answer as before.

Row	AREA	BEDS	TYPE	Observed	Priorwt	Predicted	Upper	Lower
1	1200	4	3	0	0	136.2	145.7	126.8
2	800	3	4	0	0	75.39	79.27	71.51
3	1326	4	1	60.5	1	74.89	82.3	67.48
4	782	3	2	47.5	1	74.39	78.24	70.55
5	312	1	2	46	1	48.35	53.83	42.87
6	1460	3	3	110	1	112	120.3	103.6
7	1056	4	3	148	1	128.2	137.7	118.8

Now printing out the first ten lines with an extra column *W* to show the prior weights we see from row one that the average 1200 square foot, four-bedroomed, detached house should sell for between 127 and 145 thousand pounds.

PREDICTING for a table of new observations

Any number of sets of explanatory data can be used for predicting.

```
A+800 800 1000 1000 1000 1200 1200
B+2 3 3 4 5 4 5
T+1 2 2 3 3 3 4
SUBSTITUTE 'BEDS,B:AREA,A:TYPE,T'
```

Here I have associated the variables *A*, *B* and *T* with the original explanatory variables using the function *SUBSTITUTE*.

```
REFRESH 7
```

Prediction New offset Variable Reset to the Default

REFRESH stores the new data in a "predicting matrix", a global variable called *newmat* which *PREDICT* will use in much the same way as *DIAGNOSTICS* uses the variable *desmat* set by *FIT*. The *REFRESH* right argument is the number of new cases, if any of your new data is the wrong length a message results. Of course in a function you would call it with something like *REFRESH* ρ *A*. Here you see evidence that *ASLGREG* does generalised as well as ordinary linear regression, the message tells you that the default value is used for the offset associated with this set of explanatory variables (zero in this case as I have set no offset).

```
PREDICT 'SMUL'
```

Row	A	B	T	Predicted	Upper	Lower
1	800	2	1	57.5	62.38	52.63
2	800	3	2	75.39	79.27	71.51
3	1000	3	2	86.47	91.23	81.72
4	1000	4	3	125.1	134.6	115.7
5	1000	5	3	86.47	91.23	81.72
6	1200	4	3	136.2	145.7	126.8
7	1200	5	4	97.56	103.7	91.38

Now *PREDICT* detects that *newmat* is present and uses it to calculate predictions and limits. Note that *newmat* is expunged by *YVAR* or by *SUBSTITUTE* ' '. This suggests the following sequence of functions:

To predict with the original data

```
SUBSTITUTE ' ', PREDICT
```

To recalculate the equation and predict

```
FIT, PREDICT
```

To change one of the explanatory variables and predict again

```
A[2]←900 (say), REFRESH, PREDICT
```

To use a different set of explanatory variables

```
SUBSTITUTE, REFRESH, PREDICT
```

Confidence Limit Options

PREDICT 'SMTK'						
Row	A	B	T	Predicted	Upper New	Lower New
1	800	2	1	57.5	74.51	40.5
2	800	3	2	75.39	92.13	58.65
3	1000	3	2	86.47	103.4	69.51
4	1000	4	3	125.1	144	106.3
5	1000	5	3	86.47	103.4	69.51
6	1200	4	3	136.2	155.1	117.4
7	1200	5	4	97.56	115	80.14

You can calculate confidence limits for new observations (T, K).

90 PREDICT 'SMUL'						
Row	A	B	T	Predicted	Upper	Lower
1	800	2	1	57.5	63.84	51.17
2	800	3	2	75.39	80.43	70.36
3	1000	3	2	86.47	92.64	80.3
4	1000	4	3	125.1	137.4	112.8
5	1000	5	3	86.47	92.64	80.3
6	1200	4	3	136.2	148.5	124
7	1200	5	4	97.56	105.6	89.54

You can change the confidence level (90% in this case).

80 2 PREDICT 'SMUL'						
Row	A	B	T	Predicted	Upper	Lower
1	800	2	1	57.5	66.12	48.88
2	800	3	2	75.39	82.24	68.54
3	1000	3	2	86.47	94.87	78.07
4	1000	4	3	125.1	141.9	108.4
5	1000	5	3	86.47	94.87	78.07
6	1200	4	3	136.2	152.9	119.6
7	1200	5	4	97.56	108.5	86.65

You can calculate simultaneous confidence limits, in this case Bonferroni limits (2 in left argument).

80 3 PREDICT 'SMUL'						
Row	A	B	T	Predicted	Upper	Lower
---	-	-	-	-----	-----	-----
1	800	2	1	57.5	67.08	47.93
2	800	3	2	75.39	83	67.78
3	1000	3	2	86.47	95.8	77.15
4	1000	4	3	125.1	143.7	106.6
5	1000	5	3	86.47	95.8	77.15
6	1200	4	3	136.2	154.8	117.7
7	1200	5	4	97.56	109.7	85.45

Or use Scheffe simultaneous limits (3 in left argument). As you would expect with so few cases the Bonferroni limits are narrower.

80 4 PREDICT 'SMUL'						
Row	A	B	T	Predicted	Upper	Lower
---	-	-	-	-----	-----	-----
1	800	2	1	57.5	66.12	48.88
2	800	3	2	75.39	82.24	68.54
3	1000	3	2	86.47	94.87	78.07
4	1000	4	3	125.1	141.9	108.4
5	1000	5	3	86.47	94.87	78.07
6	1200	4	3	136.2	152.9	119.6
7	1200	5	4	97.56	108.5	86.65

Or get the program to choose the narrowest for you (4 in left argument).

80 3 PREDICT 'SMTK'						
Row	A	B	T	Predicted	Upper New	Lower New
---	-	-	-	-----	-----	-----
1	800	2	1	57.5	100	15
2	800	3	2	75.39	117.3	33.53
3	1000	3	2	86.47	128.9	44.05
4	1000	4	3	125.1	172.3	78.03
5	1000	5	3	86.47	128.9	44.05
6	1200	4	3	136.2	183.3	89.16
7	1200	5	4	97.56	141.1	54.01

Scheffe simultaneous limits for new observations.

A+28pA ♦ B+28pB ♦ T+28pT

REFRESH 28

Prediction New offset Variable Reset to the Default

Increase the number of cases from 7 to 28.

+12++80 2 PREDICT'SMUL'						
Row	A	B	T	Predicted	Upper	Lower
---	-	-	-	-----	-----	-----
1	800	2	1	57.5	68.39	46.62
2	800	3	2	75.39	84.05	66.73
3	1000	3	2	86.47	97.08	75.87
4	1000	4	3	125.1	146.3	104
5	1000	5	3	86.47	97.08	75.87
6	1200	4	3	136.2	157.3	115.2
7	1200	5	4	97.56	111.3	83.78
8	800	2	1	57.5	68.39	46.62
9	800	3	2	75.39	84.05	66.73

The first 12 rows of Bonferroni simultaneous limits for the mean which get wider as the number of cases increase.

+12++80 3 PREDICT'SMUL'						
Row	A	B	T	Predicted	Upper	Lower
---	-	-	-	-----	-----	-----
1	800	2	1	57.5	67.08	47.93
2	800	3	2	75.39	83	67.78
3	1000	3	2	86.47	95.8	77.15
4	1000	4	3	125.1	143.7	106.6
5	1000	5	3	86.47	95.8	77.15
6	1200	4	3	136.2	154.8	117.7
7	1200	5	4	97.56	109.7	85.45
8	800	2	1	57.5	67.08	47.93
9	800	3	2	75.39	83	67.78

Scheffe simultaneous confidence limits for the mean are independent of the number of cases.

+12++80 4 PREDICT'SMUL'						
Row	A	B	T	Predicted	Upper	Lower
---	-	-	-	-----	-----	-----
1	800	2	1	57.5	67.08	47.93
2	800	3	2	75.39	83	67.78
3	1000	3	2	86.47	95.8	77.15
4	1000	4	3	125.1	143.7	106.6
5	1000	5	3	86.47	95.8	77.15
6	1200	4	3	136.2	154.8	117.7
7	1200	5	4	97.56	109.7	85.45
8	800	2	1	57.5	67.08	47.93
9	800	3	2	75.39	83	67.78

For this model with 28 cases the Scheffe limits are narrowest for simultaneous means. Remember that these limits are approximations see, for example, page 152 of "Introduction to Linear Regression Analysis", DC Montgomery and EA Peck, Second Edition, John Wiley ISBN 0-471-53387-4.

↑12↑+80 3 PREDICT'SMTK'						
Row	A	B	T	Predicted	Upper New	Lower New
1	800	2	1	57.5	138.3	-23.31
2	800	3	2	75.39	155	-4.186
3	1000	3	2	86.47	167.1	5.833
4	1000	4	3	125.1	214.7	35.59
5	1000	5	3	86.47	167.1	5.833
6	1200	4	3	136.2	225.7	46.76
7	1200	5	4	97.56	180.3	14.77
8	800	2	1	57.5	138.3	-23.31
9	800	3	2	75.39	155	-4.186

For new observations (unlike means) the Scheffe simultaneous limits do depend on the number of observations. On the subject of simultaneous limits for many cases Montgomery and Peck warn "the individual intervals may be so wide that they are practically useless". What this tells me is that there is so much variation in house prices not explained by this model that it is not practical to draw a boundary below which not even one out of 28 houses could be expected to fall.

A General Model

KILLED←0 5 15 28 37 49 50

DOSE←17

START

Regression Volume 1 Version 2

A.M.Sykes & E.P.Morgan, February 1993

Copyright(C) British APL Association 1991-1993

YVAR'KILLED'

Y variable is KILLED

Units set to 7

ERROR 3

Error Chosen is BINOMIAL

Use NVAR to declare the N-variable

Link selected is LOGIT

NVAR'50+DOSE-DOSE'

Fairly standard up to the last command, Alan Sykes's data on 50 insects exposed to different dose levels of insecticide. Alan would have said $N \leftarrow 7p50$ and $NVAR'N'$. If I specify an APL expression that returns the right number of 50s based on the length of *DOSE* then, if I later substitute *DOSE* with another variable for predicting, I get an appropriate new *NVAR* automatically. It is not necessary but by taking pains now I can be lazy later on.

```

FIT'GM+DOSE'
Cycle      Deviance      df      Scale Parameter
-----
4          8.343         5         1

```

```

DISPLAY 'ESTP'
Var      Est      Std Err      t-stat      p-val
---
GM       -5.089    0.5356    -9.503      0.0002182
DOSE     1.333     0.133     10.03      0.0001687

```

```

DIAGNOSTICS'XNFO'
Unit      DOSE      Out of      Fitted Value      Observed
---
1         1         50         1.142             0
2         2         50         4.073             5
3         3         50        12.59            15
4         4         50        28.04            28
5         5         50        41.44            37
6         6         50        47.42            49
7         7         50        49.29            50

```

```

SUBSTITUTE'DOSE,NEWDOSE'
NEWDOSE+=0.5+18
REFRESH 8
Prediction New offset Variable Reset to the Default
newnname
50+NEWDOSE-NEWDOSE

```

I have set up *NEWDOSE* to get a table of predicted results for the intervening extra half a dose strengths and *REFRESH* has inspected *nname* and made the substitution to create *newnname* that will provide an appropriate n-variable for use with *NEWDOSE*.

```

PREDICT'XNFUL'
Row      DOSE      Out of      Predicted      Upper      Lower
---
1        0.5       50         0.593         1.186     0.2947
2        1.5       50         2.178         3.57      1.313
3        2.5       50         7.366         9.931     5.374
4        3.5       50        19.8          22.91     16.85
5        4.5       50        35.66         38.34     32.65
6        5.5       50        45.21         46.72     43.1
7        6.5       50        48.64         49.23     47.62
8        7.5       50        49.63         49.83     49.22

```

```

PREDICTED
0.593 2.178 7.366 19.8 35.66 45.21 48.64 49.63

```

The function *PREDICTED* returns a vector of predictions.

```

NEWDOSE+3 4 5
NEWNVAR'100 200 300'

```

REFRESH 3

Prediction New offset Variable Reset to the Default

PREDICT'XNPUL'					
Row	DOSE	Out of	Predicted	Upper	Lower
---	---	---	---	---	---
1	3	100	25.18	31.15	20.02
2	4	200	112.2	124.1	99.81
3	5	300	248.7	261.5	232.7

You can take control of the n-variable used for predicting by specifying it explicitly.

Using Offsets

```

Y+2 1 3 2 5 10
X+16
SINK+START
SINK+YVAR'Y'
ERROR'BIN'
Error Chosen is BINOMIAL
Use NVAR to declare the N-variable

Link selected is LOGIT
NVAR'10+X-X'

OFFSET'.2*X'
Offset Variable is .2*X'

FIT'GM+X'
Cycle,Deviance,df,Scale Parameter
-----,-----,-----
4,9.979,4,1
SUBSTITUTE'X,NEWX'

NEWX+6+14
REFRESH 4
Prediction New offset Variable is .2*NEWX

newoffname
.2*NEWX
newnname
10+NEWX-NEWX

```

PREDICT'XNZMUL'						
Row	X	Out of	Offset	Predicted	Upper	Lower
---	---	---	---	---	---	---
1	7	10	1.4	8.797	9.575	7.035
2	8	10	1.6	9.397	9.849	7.88
3	9	10	1.8	9.707	9.948	8.523
4	10	10	2	9.86	9.982	8.991

```

NEWOFFSET'2 3 4 5'
Prediction New offset Variable is 2 3 4 5

```

NEWNVAR' 12 14 16 18'

REFRESH 4

Prediction New offset Variable is 2 3 4 5

PREDICT'XNZMUL'						
Row	X	Out of	Offset	Predicted	Upper	Lower
1	7	12	2	11.16	11.71	9.746
2	8	14	3	13.78	13.95	13.13
3	9	16	4	15.95	15.99	15.7
4	10	18	5	17.99	18	17.9

If you specify your original *offset* carefully then *REFRESH* will take care of it for you but you can take control if you need to.

Robustness

A number of places where you used to get APL errors are now trapped. I agree with Jon Sandles that when this happens something is terribly wrong with what you are trying to do, I try to tell you why.

Function Protect against (and stop with explanation if necessary):
COOKSDIST, *LEVERAGE* = 1
PRESS,
STANDRESID

HATMATRIX *scp*=0

FIT The case(s) causing the problem are indicated when the solution goes outside the domain of the link function.

BOXCOXLOGLIKE *Y* not positive. Also adjust for *alias*, *gmflg* and *priorwt*>0

Several Divide by zero

Corrections to version 1

START: "labels" not "lables" and include "forder" when setting up "sysvar" near the top.

YVAR: "labels" not "lables" and / before "alias" when expunging variables near the top.

OFFSET "offset" defaults to zero not one, look for *offset+unit**sp*1 and change to *offset+unit**sp*0

ΔMU5 make *RES* a local variable

Implementation

I have tried to make sure that user functions written for version 1 will work with version 2. This will be completely true if you use only those functions defined in the ASLGREG manual with a version of APL that allows ambivalent functions. The functions *LP*, *FIT* and *TESTHYP* are now ambivalent, providing a left argument of zero in your current functions is all you need to do. If you are using any subroutines (that is functions whose name begins with Δ) then you may find they have changed, in particular the link and error functions have changed and your own new link functions will need modifying.

Here are *DIAGNOSTICS* and *PREDICT* right arguments:

	<i>DIAGNOSTICS</i>	<i>PREDICT</i>
A		Variance
C	Cooks Distance	Cumulative
D	Dev Residual	Cum Variance
E		Upper Cum
F	FittedValue	Lower Cum
G		Upper Cum New
H		Lower Cum New
K		Lower New Obs
L	Leverage	Lower limit
M		Predicted
N	Outof	Out of
O	Observed	Observed
P		Predicted
R	Residual	
S	Std Residual	All substitutes
T	t-Residual	Upper New Obs
U		Upper limit
V	Valid Fit	Valid
W	Priorwt	Priorwt
X	All explanatory	All explanatory
Y		Observed
Z	Offset	Offset
1	X	X
?	This list	This list

The *V* for valid option confirms that these explanatory values lead to a valid prediction or fitted value when the link function is applied. The numbers 1, 2, ... refer to the first, second, ... explanatory variable and when you use argument ? their names are listed.

The N-Queens Problem in 1 line

by R.G. Selfridge
selfrid@nervm.nerdc.ufl.edu

The July 1993 edition of Vector has an article discussing putting Queens on a chessboard so that no two can attack each other (Backtracking, Queens and Permutations, pp34-38). Given the old myth (?) that any program can be written as a single APL statement, I naturally ask if this is possible for the Queens problem. Here is the 'yes' answer to that question:

```

      ∇ z←x q b
[1]   →⊂((0=ρx)/'0',(ρp+z+' '))q(b,b)ρ0×d←
      ((n-k)⊙d)∨d∨ϕd+n∘.=n+1-1+2×k+b'),
      (z+0=ρx)/'1'⊂((k=ρx)∨~i)/'0''),
      ((i∧k=ρx)/' '(x,0)q b∨(ρb)+(ρb)-x[ρx],ρx)+d'),
      ((i∧k=ρx)/' ',ρp+x,p'),((1=ρx)∧~i)/' ',p'),
      ((i∧(⊂pw<(2×k+1)×1+(ρp)÷k)∧k=ρx)∨
      (1=ρx)∧~i+k≥x[ρx]+x[ρx]+(x[ρx]+,b[;ρx])10)/
      ''+0ρ⊂+' ' | _Q' '[(2+p[1;])>2],[1]
      p+1+((2×1+ρp)ρ0 1)\1+p-(((1+k)×(1+ρp)÷k)ρ(kρ1),0)\
      1+p-(1k)∘.=p' '

```

∇

' ' q 4

```

|_ | Q | _ | _ |   |_ | _ | Q | _ |
|_ | _ | _ | Q |   |Q | _ | _ | _ |
|Q | _ | _ | _ |   |_ | _ | _ | Q |
|_ | _ | Q | _ |   |_ | Q | _ | _ |

```

Let me add that no claim is made that this is 'good' APL, only that it was a challenge to see if it could be done. Timing against a 'proper' program actually shows that this function is only twice as slow. The function will take any board size, greater than 3, and will print out as many boards across the screen as permitted by $\square PW$ (sanity suggests that board sizes greater than 9 or 10 not be tried, since there will be quite a wait for the output to complete).

(36 minutes for ' ' q 11 on a DX2/50 with Dyalog APL/W ... and I think Richard Smith deserves the next Outstanding Achievement Award for typing this in from the listing and getting it to run first shot! *Production Ed.*)

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner: Minding your .INIs	Adrian Smith	96
At Play with J	Eugene McDonnell	100
Loops in APL2	H Haussmann	106
A Control Operator	Denis Samson	113
The Ultimate TURING Proof	G.A. Langlet	124
A VSAM Simulation for APL*PlusII/386	Allan Gay	133

Hackers' Corner: Minding your .INIs

by Adrian Smith

Introduction

Some Windows applications are better behaved than others. Winword is one of the worst — it writes about 100 lines of junk all over your WIN.INI, which is a wonderful way of slowing the Windows startup. A well-behaved application (such as Dyalog APL/W) just slots in a minimal entry:

```
[DyalogAPL]
dyalog=c:\wdyalog
inifile=c:\wdyalog\apl.ini
sound=quack
```

to tell it where to go and look for a private .INI file, which can be hidden away in its own directory. (As for how that spurious "sound=" entry got there ... well, you will have to read on to find out.)

Many applications compromise, and maintain a private .INI, but stick it in the Windows directory where they can be sure to find it. MS-mail is typical, as is CLOCK, which leaves behind a CLOCK.INI looking something like this:

```
[Clock]
Maximized=0
Options=1,1,0,0,0,0
Position=119,708,155,744
```

Which leaves you with a couple of choices to make, and a little gentle research to do to find out how to maintain these files safely from your APL application.

Choices

Here are my views on the three methods outlined above:

1. Sticking everything into a [section] in WIN.INI. This is for lazy programmers only. It is definitely the soft option, and Windows gives you a simple utility which does just that. It has the unfortunate side-effect of adding to an already overloaded file, and also makes your application a pain to un-install. I suggest you don't do it.
2. Making an APPL.INI file in the Windows directory. This is more reasonable, as this is where most simple folk start looking for .INI files. Again, there are simple utilities available to help you. The only disadvantage is that your

application is now scattered over a couple of directories, and someone may re-install Windows and clean you out accidentally.

3. Making an APPL.INI file in your own home directory, and inserting a pointer to it in WIN.INI. You could use the existing [DyalogAPL] section, or you could start an [APLApps] section on the assumption that you won't be the only APL program on the machine. I think I like this option best, and it is what the better-behaved Windows programs are now tending to do.

Assuming that you want to do something sensible to record your startup parameters (no, don't put them in a component file!) where they can be fixed up with any old text editor — read on.

Option-1: Infesting WIN.INI

Before you go updating things, make sure you can read them! This is the first function you need to know about:

```

▽ r←GPS arg;gps;section;entry
[1]  A Access to profile string from WIN.INI
[2]  A e.g. GPS 'dyalogAPL' 'inifile'
[3]  'gps'⌈NA'I kernel.P16|GetProfileString <0T <0T <0T >0T I'
[4]  A Usage <gps 'DyalogAPL' 'inifile' 'default value' 40 39>
[5]  section entry+arg
[6]  r←2>gps section entry'(undefined)' 256 255
▽

```

This takes a vector of two elements, the first of which is the section (but don't put the [] around it); the second is the parameter you want to know about. It returns a simple vector, with the parameter value, or "(undefined)" if it failed to find anything. For example:

```

GPS 'Colors' 'ButtonFace'
192 192 192

```

Anything you can get, you can set:

```

▽ {r}←WPS arg;wps;section;entry;value
[1]  A Write info to profile string in WIN.INI
[2]  A ***** USE WITH CARE!! *****
[3]  A e.g. WPS 'DyalogAPL' 'sound' 'quack'
[4]  A
[5]  'wps'⌈NA'I kernel.P16|WriteProfileString <0T <0T <0T
[6]  A Usage <wps 'section' 'entry' 'new value'>
[7]  section entry value+arg
[8]  r←wps section entry value
▽

```

... which is how that "quack" got in there! If you use a new section name, Windows creates the section for you, if you use a new parameter name, it is added automatically to the section. It really is amazingly helpful, which probably explains why the average WIN.INI is so full of junk.

Option-2: Infesting the Windows Directory

For simple applications, this is probably a good compromise. This time, to create your .INI file, you use a related Windows call:

```

V {r}+fi WPPS arg;wpps;section;entry;value
[1]  A Write info to private profile string such as appl.ini
[2]  A e.g. 'clock' GPPS 'clock' 'position' '200,200,100,100'
[3]  A Assumes '.ini' unless file extension given.
[4]  A
[5]  'wpps' \NA'I kernel.P16|WritePrivateProfileString <OT <OT <OT <OT'
[6]  A Usage <gpps 'section' 'entry' 'new value' 'infile'>
[7]  fi+fi, (~'.<cfi>)/'.ini' o section entry value+arg
[8]  r+wpps section entry value fi
V

```

This continues the helpful approach by creating the entire file for you if it didn't already exist. For example:

```

'fluff' WPPS 'Options' 'colour' '255 255 255'
'fluff' WPPS 'Options' 'size' 'small'
'fluff' WPPS 'Toolbar' 'setting' 'off'

```

results in the creation of a 'fluff.ini' file which looks like:

```

[Options]
colour=255 255 255
size=small

[Toolbar]
setting=off

```

... which can later be recovered just as straightforwardly with:

```

V r+fi GPPS arg;gpps;section;entry
[1]  A Access to private profile string such as clock.ini
[2]  A e.g. 'clock' GPPS 'clock' 'options'
[3]  A Assumes '.ini' unless file extension given.
[4]  A
[5]  'gpps' \NA'I kernel.P16|GetPrivateProfileString <OT <OT <OT >OT I <OT'
[6]  A Usage <gpps 'section' 'entry' 'default value' 40 39 'infile'>
[7]  fi+fi, (~'.<cfi>)/'.ini' o section entry+arg
[8]  r+2>gpps section entry'(undefined)' 50 49 fi
V

```

You might even find these handy for fixing the settings on some of the standard applications, for example to start a scientific calculator from an APL button.

Option-3: Hiding the .INI in your own Directory

This is really just a combination of the first two. I think the best compromise is to add an entry to an [APLApps] section:

```
WPS 'APLApps' 'Fluff' 'c:\apps\fluff\fluff.ini'
```

... and to maintain FLUFF.INI as above with WPPS. This does mean one extra lookup when you start, but you can be sure the [APLApps] section is there, and you have generated the least possible clutter in the Windows directory.

Summary

There really is no excuse for not following the way of Microsoft, and putting all your startup options in an editable .INI file. You don't even need to write the editor!

Postscript

```

V {r)+{flg}PLAY snd;sps
[1]  A Play a .WAV file (asynchronous by default)
[2]  A flg+{/0(synch) 1(asynch) 2(no deflt) 8(repeat) 16(nostop)
[3]  snd+snd, (~'. 'esnd) /'.WAV'
[4]  'sps' NA'I mmsystem.P16|sndPlaySound <0T U'
[5]  ±(0=NC'flg')/'flg+1'
[6]  r+sps snd flg
V

PLAY 'c:\dash\dog'      A for the friendly Dashboard dog
or  PLAY 'c:\windows\sndbits\music\didjeri'  A {Sic)

```

... for that true Australian flavour! Does anyone know of a good duck?

At Play with J

by Eugene E McDonnell

You may be among the readers of Vector whose stomachs start churning or whose eyes glaze over when they read the words "tacit definition" in articles about J. This column is meant for you. It is going to take the approach that you know how to write an APL function, and might not be averse to learning how to write a J function in a similar fashion.

The example I shall use is fairly short — just ten lines — and may possibly even be of use to some of you. It was shown to me by Joey Tuttle many years ago. It's called "factors" and it factors a positive integer n . The result of `factors n` is an ordered list of primes such that n is the product over the list, that is, $n = \prod \text{factors } n$. It was able to find the factors of the 18-digit number $(2^{59}) - 1$ (576460752303423487), which are 179951 and 3203431780337, in 3 minutes and 37.1 seconds on a Macintosh Quadra 700.

Various methods of finding the prime factors of a number are given in Knuth, *Seminumerical Algorithms*, Section 4.5.4 "Factoring into primes", pp. 338-360. The method used here is the simplest one, Algorithm A, which divides the number by increasingly larger primes until all factors have been found, but the method has been vectorized so that more than one factor may be found at once.

The argument must be a positive integer not greater than `max`, where `max` is a number derived from the floating-point characteristics of your computer. This discussion assumes that your computer uses IEEE floating point arithmetic, which is the case for PCs, Macintoshes, and most Unix machines. On PCs and Unix machines the maximum is the 16-digit $(2^{53}) - 1$ (9007199254740991). For Macintoshes it is the 19-digit $(2^{63}) - 1$ (9223372036854775807), or 1024 times as large. These are the largest integers which can be represented to full accuracy on those machines without special programming.

First I'll show the function definition, then give explanations of its lines, paying special attention to those things that differ from APL.

Function Definition

Let's assume for the moment that you created the function `text`, called "text", below. It could be in the form of a character matrix, a character string delimited by carriage returns, or a vector of boxed character vectors.

```

0.   f =. 1. 0
1.   t =. 2 3 5 7 11 13 17 19 23 29 31 37
2.   o =. +/\ 432 $ 4 2 4 2 4 6 2 6
3. L) m =. (-. * t |1.0 y.) # t
4.   f =. f , m
5.   y. =. y. % */ m
6.   $. =. > (* # m) { $. ; L
7.   t =. o + {: t
8.   $. =. > (y. >: *: {. t) { $. ; L
9.   f =. /:~ f , y. -. 1

```

The line numbers are merely for reference. They are not part of the function.

The first thing to notice is that there is no header line, so it would seem that we can't tell whether there is an explicit result or not, nor what its name is if there is one, nor what the function name is, nor what its valence is, nor what the argument is named, nor which variables are local.

Here is how each of these questions is resolved:

Explicit result. Every J function has an explicit result, and it is the value of the last expression executed.

Function name. In J a function is named the same way that a variable is named, by assignment, denoted by ($=$). However, we can't just write `factors =. text` because all that this would do would be to create another variable named "factors" with the same value as "text". We have to use the "definition" operator, symbolized by the colon ($:$), whose result is a function, not a variable. Our function is monadic, so to give it a name we write `factors =. text : ''` and this creates the function named "factors".

Valence. For an ambivalent function, the monadic case is given by the left argument to ($:$), and the dyadic case is given by the right argument. If the function is monadic, as in our case, the right argument is empty; if it is dyadic, the left argument is empty. There is no such thing as a niladic function in J.

Argument name. By convention, the argument to a monadic function is ($y.$). In a dyadic function the left argument is named ($x.$) and the right argument is named ($y.$).

Local variables. J makes a distinction between local assignment ($=$) and global assignment ($=:$). This removes the need for a list of local names. The rule is that on the first assignment of a name using ($=$) the name is made local.

Here beginneth the detailed description of the function's lines:

Line 0 sets the initial factor list to empty. J's iota is denoted by (i.). By the way, if the argument is 1, this will be the result as well, since 1 is not a prime and has no prime factors. However, it will still be true that $n = */ \text{factors } n$, since the product over an empty vector is 1.

Line 1 creates as the initial list of trial divisors the first 12 primes. This enables the function to use just one iteration for all arguments less than 1370 (37^2 is 1369).

Line 2 forms the list of offsets to be used in creating a new list of trial divisors, after there are no more values left in the current list that divide the current value of the number being factored. In J, the scan operator is defined differently from the APL scan operator. In APL the scan operator implies the reduction operator. In J the function argument to scan is monadic, not dyadic, so that one has to use "sum" (+/) not "add" (+) if we want the sum scan.

There are 432 items in this vector: 54 repeats of the eight items 4 2 4 2 4 6 2 6. The purpose of this vector is to remove multiples of 2 3 5 from consideration as trial divisors, since these can't be primes. For example,

```
37 + +/\ 4 2 4 2 4 6 2 6 ...
41 43 47 49 53 59 61 67 ...
```

Some of the items in these new trial divisors will not be primes. In the list above 49 is composite. Eliminating multiples of 2 3 5 reduces the number of trial divisors by over 73%.

Line 3 looks as if it begins with a label, but the label name is followed by a right parenthesis instead of a colon. This is because the colon is used for function definition, as described above, and so J uses the right parenthesis ")" to separate the label name from the instruction. A label in J is a vector. The value of label "L" is 3 4 5 6 7 8 9; that is, it is a vector of instruction numbers, beginning with the number of the instruction in which it appears, and followed by each of the subsequent instruction numbers. We'll see the use of this when we discuss instructions 6 and 8.

This line begins the iteration, and forms the vector *m* of newly-found factors of the argument. The factors are found by using the vector of trial divisors of the argument as the left argument to the residue function. However, the residue function in J is fuzzed, just as it is in some APLs, in order to permit proper-looking results when used with near-integers, and also to permit the use of decimal rational numbers as arguments to residue. We are dealing with exact

integer arguments, so in order to extend the domain of the residue function we require that it not be fuzzed. To accomplish this in APL systems, the comparison tolerance system variable, $\square ct$, is set to 0. In J this is done by explicitly modifying the residue function with the "fit" or "customize" operator (1.) using 0 as right argument. Thus, instead of writing $t | y$. We write $t | !. 0 y$. and this allows us to work with a residue that has zero fuzz.

Curiously, the "Dictionary of J", which says that the "fit" operator "modifies certain verbs in ways prescribed in their definitions", doesn't describe this use of it with respect to residue. Tsk tsk. The advantage of having the modification of the verb occur in direct connection with its use is obvious: one doesn't have to remember whether or not $\square ct$ has been modified, nor run into the hazard of not restoring it when it should be. The use is direct and immediate, and applies only to the function in question. Furthermore, anyone reading the function knows immediately that it is unfuzzed.

After finding the residues, their signum (*) is taken, yielding a vector of 0s and 1s, and these are negated (-.) complementing them to 1s and 0s. This boolean vector is used to select (#) the corresponding items from t , giving in m the new factors to be appended to the result. In APL there has always been a confusion about slash: is it an operator or a function? If we write $+ / 1 \ 2 \ 3$ it acts like an operator, but when we write $1 \ 0 \ 1 / 1 \ 2 \ 3$ it acts like a function. In J the "#" function is used for the functional case, as in $1 \ 0 \ 1 \# 1 \ 2 \ 3$.

Line 4 appends the new factors to the result vector.

Line 5 factors the argument, by dividing (%) it by the product (*) of the new factors, thus diminishing it.

Line 6 is a branch instruction. There is no branch arrow in J, however. What takes its place is a variable called "suite" and denoted by (\$.). This variable determines the sequence in which instructions of a defined function are executed. At the beginning of execution of a defined function, suite is set to a vector of the instruction numbers in the defined function. In our factors program, which has ten instructions, it would have the initial value $0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9$. At the beginning of execution of each instruction, the leading item of (\$.) is removed. Thus, if a five instruction program were written in which each instruction displayed the current value of (\$.), the initial value of (\$.) would be $0 \ 1 \ 2 \ 3 \ 4$, and the display of each instruction would be:

```

Instruction 0  1  2  3  4
            1  2  3  4
            2  3  4
            3  4
            4  (empty)

```

The expression at the end of line 6 (\$. ; L) forms a two-item vector of boxes, using the link (;) function. The head of the vector is the boxed current value of suite, and its tail is the boxed vector L. Selection by index (x { y means select item x of y) is used to choose one of them, and the one chosen is opened (>). The signum (*) of the number of (#) factors in m is determined. This will be 0 if there are no new factors, and 1 otherwise. If there are no new factors, suite is reassigned to itself — effectively a fall-through. If there are new factors, L is assigned to suite, causing a branch to line L. Thus, if there were any factors in the current vector of trial divisors, we may not have finished with it, and return to L to try again with it. If there were none, we are through with the current vector, and fall through to the next instruction.

Line 7 forms a new vector of trial divisors, by adding the offset vector to the last (:) item of the current vector.

Line 8 is another branch instruction. This one causes control to be returned to instruction L if the reduced argument is greater than or equal to (>:) the square (*:) of the first item (:) of the new vector of trial divisors, since in this case there may be more factors. If it is less, this means that there can't be any new factors in the reduced argument, and thus it is either 1 or a prime.

Line 9 removes 1 from the argument (the result of x -. y is x with items equal to y removed; 17 -. 1 is 17; 1 -. 1 is empty). This leaves it unaltered if it is a prime, or makes it empty otherwise. It then appends the argument to the list of factors (essentially doing nothing if it had been 1), sorts (/:-) the list into ascending order, and terminates. In J the semantics of dyadic upgrade and downgrade have been changed. They no longer have the significance of using the left argument as a collating sequence to produce a grade with reference to it. Instead, they are used to sort the left argument into an order specified by the right argument. The definition of dyadic upgrade (x /:- y) is (/:-y) { x; that is, the permutation that puts y in ascending order is used to permute the items of x.

The most frequent use of these sort verbs is with the left and right arguments identical, in which case the result is the sorted argument, either ascending or descending. The reflexive operator (~) applies to a monadic verb to produce a dyadic verb with left argument the same as the right argument. For example, +~1.2 is 2.4, and /:- 2 7 1 8 2 8 is 1 2 2 7 8 8.

Below are some examples of the use of the factors function with some ten-digit numbers to give you some idea of how it behaves. Notice that the time to factor a number is longest when the argument is a prime, and fairly long also when there are two factors roughly equal to the square root of the argument. The comment symbol in J is (NB.).

```
time =. 6!:2 NB. yields seconds to execute its string argument
fmt=."!:20 NB. formats (":) numbers to 20 places (1.20)
```

```
time 'k=.factors 6307059899'
1.11667
fmt k
7 19 47421503
```

```
time 'k=.factors 6307059901'
0.85
fmt k
379 16641319
```

```
time 'k=.factors 6307059903'
0.983333
fmt k
3 127 3461 4783
```

```
time 'k=.factors 6307059907'
0.65
fmt k
1201 5251507
```

```
time 'k=.factors 6307059909'
3.51667
fmt k
3 24749 84947
```

```
time 'k=.factors 6307059911'
10.0167
fmt k
6307059911
```

Loops in APL2

H Haussmann, Universität Hohenheim
(haussman@ds0ruh11.bitnet)

I have been using APL for decades and I have been handling vectors and matrices for centuries! Of course the last statement cannot be true as I am only 54 years old, but it feels as long as that. In any case I don't know anybody at my university who has done so much work with matrices. Why do I say this? I want to say that I do not need APL to make me use vectors and matrices! — I had read an argument, saying that the reason there is no loop mechanism in APL is to force people to think in arrays.

As I said already, I have been using APL for a long time. And all that time I have felt that APL is incredibly wonderful and that it is my great love! But I also felt that APL had some serious deficiencies. Later some of these deficiencies have been cured, again in a wonderful way, by APL2! — APL2 is wonderful! It is ten times more powerful than APL. But, of course, it is no longer quite so easy and it needs some experience to make use of this power — especially to know when you need a *disclose* more or an *enclose* less! In the meantime I have learnt: whenever your function doesn't work and you search and search and you cannot find any error: check the depth of your variables and you will immediately find the reason for the trouble! — but one main deficiency still remains: the lack of a loop!

I have not used FORTRAN now for at least 10 years, but, whenever it comes to a loop, I sigh and remember my first love: O dear! How easy is it with FORTRAN to write `DO 10 I = 1, 100` and then `10 CONTINUE`! There is no need to waste a second of thinking and you can be absolutely sure, that the loop will be performed exactly 100 times! Why does my great love APL2 deny me such a caress?

To think in arrays and to use array handling functions is not trivial, and is not the natural approach of anyone or a schoolboy or schoolgirl nowadays, for dealing with numbers and characters stored in a computer. The normal approach to nearly every simple problem that can be tackled by beginners is the loop! And if it is easier to write a loop in BASIC or some other language, then it is no wonder that APL cannot become a generally accepted language. And this is what bothers me most: the most powerful language is not accepted by normal students. It is a pity that a 100% efficient tool is only nowadays used by .1% of computer users.

A few years ago my son was 13, now he is 16. How can I recommend him and his friends a programming language that doesn't have a convenient loop? In spite of my enthusiasm for APL2, I just can't honestly do it. Let's say they have a text matrix and they are looking for a certain string in each line. There must be an easy way to go through the matrix row by row and to process each line separately! This is how a normal person looks at such a problem. Of course APL and especially APL2 provide a lot of elegant means to do it otherwise but a child of ten years is not yet interested in such crazy things. — a programming language is a tool and not a straitjacket. It has to enable things and not to prevent anything! The advantage of APL2 is that you don't need to code a lot of loops all the time because you can use vectors, nested vectors and matrices, the each operator and so on. The disadvantage is that you can't code a loop in an easy way when you need it or just want it.

Loops are necessary, in my work too, although I know to use arrays and avoid loops. But there are situations where a lot of loops have to be defined and where there is no other way! And furthermore: does it make sense to think for half an hour (or for three days!) and to manipulate an array by ten primitive functions if the alternative is just to apply a simple loop which goes from 1 to n? This may be a nice exercise for somebody trying to learn APL and to get familiar with arrays and array manipulation. It may also be a nice sport, like chess. (And it is even more like chess — chess problems — if you want to modify your function two months later!) But if you are a busy man and you want to get your work done and save your brain for problems that really need it, then you have to write a loop and you need a convenient way to do that.

Loops can be avoided by using the each operator and a subfunction. But the disadvantage of this solution is that you cannot see both functions, the calling function and the subfunction, on the editor screen simultaneously. This is rather inconvenient if several nested loops are dealt with in this way. Say, I have to maximize a likelihood function for an eight-dimensional parameter space and I want to do this the elementary way by varying all eight parameters in turn. So I have eight nested loops and it gets very impractical to develop the program with eight nested subfunctions! (I have tried it.)

I did FORTRAN when there were no THEN, ELSE and ENDIF statements, but recently I translated some FORTRAN programs with such structures into APL2. It was not trivial for me to understand them. My feeling is, that branching to different labels is not significantly inferior to using IF, THEN, ELSE and ENDIF. It may be even easier for beginners to use labels and branching. These control structures may be nice but I think they are much less essential than a convenient

loop mechanism! Certainly they would clash with the concise character of APL code a bit.

The situation for these control mechanisms is different from loops: to do something in a loop is the elementary approach. And to branch to a different statement or block of statements is also the elementary approach of the beginner. And branches and labels have to be learnt in any case (perhaps there are some fundamentalists who would disagree). To formulate things with THEN, ELSE and ENDIF is more elegant, but it calls for some learning at the beginning.

I tried to create my own loop by the use of two functions, *LOOP* and *STILL*. The resulting technique was:

```

LOOP n 'variable name' vector
label: . . .
. . .
→ (STILL n) / label.

```

I didn't like it because it needed an integer (*n*) and a label for each loop, and both the integer and the label had to be mentioned twice — I got involved in other things and many months passed.

Now I came across the above notes which you have just read and I had the idea to try if it would be possible to do it without using a label. It turned out that it was possible! By the use of $\rightarrow \square LC$. The two functions are now called *DO* and *DOLOOP* and the loop looks like this:

```

DO n 'variable name' vector
. . .
→ DOLOOP n.

```

Isn't this nice? As convenient as in FORTRAN and more flexible, as the loop variable can assume any values (defined in the vector). And it works! The advantage compared with FORTRAN, that no label is needed at all, is unfortunately balanced by the fact that a *DOLOOP* statement is always needed here whereas a *CONTINUE* statement is not always needed in FORTRAN.

Now I have my loop! I have just to copy two functions in my workspace whenever I need loops and the long-desired loop mechanism is there! You understand that being 54 years old there was not much hope that a wait for a built-in loop in APL would be rewarded — especially when you have already waited for 15 years. But now I don't need to wait any more, I have it! — Another

proof that in APL2 you can do everything yourself! Isn't it wonderful? Examples of this *DO* loop and a formal description are given in the attached program *DEMODO*.

Of course there have been many attempts to develop user-defined control structures in the past (e.g. Giloi, 1976; Piper, 1992). But to my knowledge they were not widely accepted, although many APLers argue in favour of (built-in) control structures. There may be several reasons for this. Certainly such techniques are more difficult to construct in plain APL than in APL2. So the earlier products were probably not very elegant. Also the solution given above would not be possible in plain APL because the argument of the *DO* function is a nested vector (in the normal case). On the other hand, if perfection is aimed at and all the IFs, THENs etc. are simulated by functions, then there are a lot of control functions. The disadvantage is that they all have to be copied into each new workspace and that the list of functions is always mixed with functions which are not of interest. Two such functions, *DO* and *DOLOOP*, are tolerable, at least for me.

Of course the use of these functions makes programs a bit slower. Six lines of APL2 code have to be processed each time round the loop. But I write a lot of functions where speed is of no interest. And I have learnt that future computers will be so fast that the problem will be to give them jobs!

I will probably not use *DO* and *DOLOOP* in basic functions for reasons of speed and portability. With a built-in loop mechanism of course this restriction would not be necessary. And also the preparations needed with this solution — importing *DO* and *DOLOOP*, defining the vector *LOOPS* and localizing the name *LOOPS* — would be unnecessary with a language-immanent solution.

There are two versions of the *DO* statement as indicated in the program *DEMODO*. A third version, simulating a *DO WHILE* construction, would also be possible. But I decided to leave it without that in order to keep APL simple and (relatively) fast. A *DO WHILE* problem has to be solved by leaving the loop irregularly by a branching statement located within the loop.

Some additional thoughts which may be of no practical value: In APL versions with only one statement per line it should also be possible to write loops without the loop index *n* as each *DOLOOP* statement would be uniquely defined by its line number. Even with multiple statements per line it would be possible if all loops could be relied on to end regularly, i.e. without branching from within to outside a loop. But both of these conditions are too restrictive to be considered in practice.

APL2 is a powerful but no longer a simple tool. In my opinion it has become a language for specialists and for full-time programmers. Future developments, if necessary at all, should therefore not increase its complexity and its level but rather make it a tiny bit more elementary by introducing a simple loop and a simple way to read ASCII records. How can a language exist which cannot read an ASCII file? (I will come back to this question later.) It must necessarily remain an exotic business. My fear is that one day APL will be the most powerful language in the world but it will exist in books only and in archives.

References

Giloi W. (1976): *Programmieren in APL*. Walter de Gruyter, Berlin

Piper D. (1992): *DOIF Considered Harmful*. Vector 9, 1, 133-142.

Appendix: the Functions

```

0 DO V;Z
1 a See function DODEMO.
2  $\rightarrow(0\neq(+V)\geq LOOPS)/0 \diamond \rightarrow(2\neq V)/A$  a loop already installed  $\diamond$  simple loop
3  $LOOPS[+V]\leftarrow 1(\rho Z)(2[V])(Z+3\neq V)(+1\neq LC)$ 
4  $\rightarrow(2\neq V),'+',\rightarrow Z \diamond \rightarrow 0$  a assign first el. of vector to loop var.
5  $A:LOOPS[+V]\leftarrow 1 V[2]0(+1\neq LC)$ 
6 a The vector assigned in line 3 contains:
7 a 1- first value of loop counter (always 1)
8 a 2- maximum value of loop counter
9 a 3- name of loop variable
10 a 4- vector of elements to be assigned in turn to loop variable
11 a 5- line number containing the DO statement in calling function

0 R $\rightarrow$ DOLOOP L $^-$ ;Z
1 a See functions DODEMO and DO. L $^-$  = loop index.
2  $\rightarrow((L^- - 2\geq LOOPS)\geq Z+1+L^- - 1\geq LOOPS)/A$  a counter exceeds not yet the maximum
3  $(L^- - 1\geq LOOPS)\rightarrow 0 \diamond R\rightarrow 10 \diamond \rightarrow 0$  a loop L $^-$  released
4  $A:R\rightarrow 1+L^- \geq LOOPS$  a line no. of DO statement
5  $(L^- - 1\geq LOOPS)\rightarrow Z$  a new value of counter stored
6  $\rightarrow(0\neq L^- - 3\geq LOOPS)/0$  a this is a loop without a loop var.
7  $\rightarrow(\epsilon L^- - 3\geq LOOPS),'+',\rightarrow L^- - 4 Z\geq LOOPS$  a assign next el. of vector to loop v.

```

0 DODEMO;I;IM;JM;K;L;LOOPS;M
 1 *A Demonstrates and describes the use of the loop functions DO and*
 2 *A DOLOOP.*
 3 *A When the functions DO and DOLOOP and the vector LOOPS (n_p0) are*
 4 *A present, loops may be coded as demonstrated in the following examples*
 5 *A A formal description is given below after the examples.*

Examples of "DO loops"

```

10 A
11 LOOPS+10p0           A Defining LOOPS: n ≥ number of loops which
12 A                   are active simultaneously (here 2).
13 A
14 DO 1 'I' (14)         A
15 I+10 100 1000        A loop with loop variable and vector
16 +DOLOOP 1            A
17 A
18 DO 1 3                A
19 'ABCD'               A loop without loop variable and vector
20 +DOLOOP 1            A
21 A
22 M+0p0                A
23 DO 1 'IM' (3 5 -2 -4) A
24 DO 2 'JM' (φ15)      A
25 M+M, <IM JM          A two nested loops
26 +DOLOOP 2            A
27 +DOLOOP 1            A
28 4 5pM                A
29 A
30 M+0p0                A
31 DO 3 'K' (3 5 -2 -4) A
32 DO 4 'L' (φ15)       A
33 +(Λ/K L=-4 1)/C2    A nested loops with conditional execution
34 M+M, <K L            A
35 C2:+DOLOOP 4         A
36 +DOLOOP 3            A
37 4 5p20+M            A
38 A

```

The same loops in condensed form:

```

39 A
40 A -----
41 DO 1 'I' (14) ◇ I+10 100 1000 ◇ +DOLOOP 1
42 A
43 DO 1 3 ◇ 'ABCD' ◇ +DOLOOP 1
44 A
45 M+0p0
46 DO 1 'IM' (3 5 -2 -4)
47 DO 2 'JM' (φ15) ◇ M+M, <IM JM ◇ +DOLOOP 2 ◇ +DOLOOP 1
48 4 5pM
49 A
50 M+0p0
51 DO 3 'K' (3 5 -2 -4)
52 DO 4 'L' (φ15) ◇ +(Λ/K L=-4 1)/C20 ◇ M+M, <K L
53 C20:+DOLOOP 4 ◇ +DOLOOP 3
54 4 5p20+M
55 A

```

Formal description of "DO loops"

When the functions DO and DOLLOOP and the vector LOOPS (np0) are present, loops may be written as follows:

```

DO "DO vector"                (DO statement)
. . .                        (statements)
→DOLLOOP "loop index"        (DOLLOOP statement).

```

The "DO vector" can be of two forms corresponding to two types of loops:

- a) the normal DO loop:
 DO vector = "loop index", name of loop variable, "loop vector"
 = elements to be assigned to the loop variable in turn (a scala
 is also possible if the loop has to be performed only once)
- b) the simple loop:
 DO vector = "loop index", number of rounds to be performed.

A loop is controlled by a 'co-operation' of the program itself and the functions DO and DOLLOOP. This requires that the loop parameters are stored in a variable. They are stored in an element of the vector LOOPS. The loop index indicates which element of LOOPS is occupied by a certain loop. Thus the loop index has to be a positive integer which is $\leq \rho\text{LOOPS}$.

If all rounds of a loop have been carried out regularly (if the loop was not left by a branch from within the loop but via the DOLLOOP statement), the corresponding element of LOOPS is automatically (by DOLLOOP) reset to zero. Then another loop may use the same element of LOOPS, i.e. the loop may be written with the same loop index.

n (ρLOOPS) has to be $\geq$ the number of loops which are active simultaneously. Nested loops are always active simultaneously. In this program there are actually only two loops running simultaneously, so $\text{LOOPS} + 2\rho 0$ would be sufficient. (Of course the loop indices 3 and 4 would have to be changed to 1 and 2 in this case.) Notice that a loop which is left by a branch from inside to outside the loop is, not released and the pertaining loop index must not be used again for other loops. There is no such case in the example above. There is a branching arrow inside of loop 4 but it is branching to the end of the loop and not outside of it.

The following restrictions have to be attended to:

1. A DO statement must be the first statement of a line,
2. The name L^- must not be used as name of a loop variable.

In most cases it will be usefull to 'localize' LOOPS and the loop variables (in this program I, IM, JM, K and L).

A Control Operator

by Denis Samson

Over the last 25 years, dozens of proposals have been made about adding control structures (of the if-then-switch-for-while-until variety) onto APL. At the last APL conference in Toronto, there were no less than three such papers!

Two things come to mind: first, there IS a need for some acceptable form of control structures in APL; second, no widely acceptable control structure scheme exists yet, since no APL system offers any.

The problem with all proposals so far is that either they do not extend the language, or they do extend it!

- if they do not extend APL, they use defined *IF THEN ELSE* functions that use quoted arguments: they are essentially slow and unreadable if there is more than one level of imbrication;
- if they do extend APL, they add on top of it meta-constructs that are too foreign in design to have a chance to mix with APL. For example, control structures are traditionally expressed in a redundant-prefix-and-often-postfix form — e.g. *if <condition> then <statements> endif* — whereas APL is a pure infix language: these mix like oil and water!

This paper should help solve that dilemma by offering a general-purpose operator that fits perfectly with APL syntax and gives control structures for free. The operator will be used in conjunction with a revised use of the colon and a new notation for defined functions.

Operators

Operators, also called adverbs or conjunctions in Iverson's "Dictionary of APL" [Iv 86] have been around in APL since the beginning, first as second-class APL citizens — you would call "+/" a reduction, but "/" was just part of the notation — then later [Iv 79] as distinct grammatical entities. Over the years, the syntax for operators — <function> <monadic operator> gives <derived function> and <function> <dyadic operator> <function> gives <derived function> — has been refined, new operators — e.g. each, rank, and various forms of composition — have been added and the syntax widened a bit: <value> <operator> and <operator> <value> were also permitted.

In parallel, a healthy dose of retrofitting allowed us to turn specification into an operation: the left arrow in Dyalog's $V \leftarrow +1$ (for $V \leftarrow V+1$) and $V \leftarrow \ddot{+}1$ (for $V \leftarrow 1+V$) is a (slightly anomalous) monadic-or-niladic operator that is the modern version of Burrough's APL/700 pre-operator notation.

Control Arguments

A distinction absent from APL syntax is that of "control arguments" to a function. APL does not provide any formal distinction between a "data argument" of a function and a "control argument" used to modify some behaviour of the function.

The only formal distinction was between scalar and mixed functions, where the latter's left arguments were often concerned with structure. The term "mixed" even disappeared in later dialects with the advent of function ranks. Control arguments have been recognized as such in [Iv 79] and [IPS 84] as "custom, or variant parameter", with a "variant operator" to take care of such arguments. This distinction did not surface in any implementation, however.

On the contrary, numerous ad-hoc techniques continue to be used in order to specify a variant of a function, or, in other words, to provide a function with a control argument. For example, they may use:

- a variant of the symbol: ∇V
- an implicit argument:
 $\square CT \leftarrow 0 \diamond A=B$
 $\square IO \leftarrow 1 \diamond iN$
 $\square PW \leftarrow 5 \diamond \nabla X$
- an explicit "modifier":
 $'S<., .>BF12.2' \square FMT AMT$
- brackets:
 $A, [\square IO-0.5] B$
- an operator:
 $, \ddot{\circ} 0 M$
- the left argument:
 $1 \circ X$
- a global variable:
 $FILL \leftarrow '*' \diamond 5 TAKE 'ABC' \text{ (gives 'ABC**')}$
- arrays:
 $(c5 '*') TAKE 'ABC'$

This proposal is that control arguments are supplied to a function with a control operator. The symbol we will use here is " $\rightarrow$ ", for reasons that will become obvious later on. Syntax is $\langle \text{control argument} \rangle \rightarrow \langle \text{function} \rangle$. A variant of the operator can be thought of, $\langle \text{function} \rangle \leftarrow \langle \text{control argument} \rangle$, but maybe we're getting a bit too recursive for now...

This use does not interfere with that of the "go to", which could be considered as an — outdated, slightly anomalous, but unambiguous — monadic use of " $\rightarrow$ ". The expressions in the example above could be rewritten so that they use their control argument in a consistent way, for example: $A \rightarrow B$ or $1 \rightarrow N$ or $1 \rightarrow 0X$ or $5 \rightarrow * \rightarrow \uparrow 'ABC'$ or $5 \rightarrow * \rightarrow TAKE 'ABC'$.

Now, while APL implementers can happily provide variants to primitives that will free our APL code of all those viciously hidden arguments — and certainly this is a worthwhile achievement — the real value of this operator lies in the answer to the question "how can a defined function, such as *TAKE* in the last examples, benefit from the variant specification?" Or, in modern APL speak, "how do I derive a function from ' $* \rightarrow TAKE$ '?" The mechanism provided here is definitely object-oriented: rather than tell the operator how to apply the control argument to the function argument, we pass, or "send", the control argument to the argument function, which then decides by itself what variant it is going to turn into. This can be put in contrast with APL2's use of defined operators for applying function argument(s) to (data) argument(s).

The notational mechanism used here is quite simple, and simply consists of prefixing statements with constants representing variants. APL implementations allowing named constants could certainly use them in that context. In the admittedly simplistic case of a monadic *CIRCLE* function that can emulate *CoX* with $C \rightarrow CIRCLE X$, our *CIRCLE* function would look like:

```

V R←CIRCLE X
[1] 0: R←(1-X*2)*0.5
[2] 1: R←1oX
[3] ~1: ⚠ERROR (~X=⌊1[X[1] / 'DOMAIN'
[4]    R←~1oX
[5] 2: R←2oX
    ...
V

```

And $\sim 1 \rightarrow CIRCLE$ would derive — at least conceptually — the function:

```

V R←CIRCLE X
[1] ⚠ERROR (~X=⌊1[X[1] / 'DOMAIN'
[2] R←~1oX
V

```

As one can guess from the example, the rules are quite simple: " $C \rightarrow F$ derives, for each item of C , a function built by selecting only the statements prefixed with that item". The appendix presents a more complete tentative syntax and semantics for prefixes, so we shall discuss only a few points here:

As can be seen in the example above, a prefix applies to all statements to its right and below until the next prefix. There is no reason a prefix would need to be at the beginning of a line: $1:R+1 \circ X \diamond 2:R+2 \circ X$ certainly looks quite reasonable!

One can also note that prefixes are remarkably similar to cases (provided by almost all other modern languages), except we do not need a special <break> statement to move around "unselected" cases: the derived function does not even contain the unselected statements, and the prefixes themselves are gone!

Prefixes shown so far are all scalar. But there is no reason that an array language such as APL should be restricted to scalar variants. On the other hand, rank-2 and higher arrays can be formed only with functions — as in $2 \ 3 \ \rho \ 1 \ 5$ — so we will restrict ourselves here to scalar and vector prefixes.

The prefix rank, or case rank, of a function is the maximum rank of all prefixes in the function. It gives the rank of items taken from the control argument and matched to prefixes of the function in order to derive the function or functions. For example, if FN has cases such as $0 \ 0: , 0 \ 1: , 1 \ 0:$ and $1 \ 1:$ we could get a derived function with $(0 \neq \square IO, \square CT) \rightarrow FN$ and a vector of 4 derived functions with $(\diamond \ 2 \ 2 \ \top \ 1 \ 4) \rightarrow FN$

We won't go any further here on the subject of function arrays, since our concern here is control structures — and we're almost there! Nonetheless, we would like to note arrays of functions so created do fit very naturally with APL syntax and can express very powerful ideas: for example, $X \ 1 \rightarrow FOO'' \ Y$ applies $1 \rightarrow FOO$ to each pair of elements from X and Y .

By the way, all derived functions apply to the data argument or arguments, so the control operator is a form of outer product. And $X \ (1 \ 2 \ 1 \rightarrow FOO) \ Y$ might be a nice notation for specifying three parallel executions of $1 \rightarrow FOO$, $2 \rightarrow FOO$ and $1 \rightarrow FOO$ on X and Y !

Control Structures

The astute reader probably has already noticed that a statement such as $(A > 0) \rightarrow F$ with F defined as:

```

      ∇ F
[1] 1: 'TRUE'  ∅ 0: 'FALSE'  ∅ 2: 'MAYBE'
      ∇

```

expresses a simple *if then else* in a very APL-looking style!

Further, we can even use:

1→F	0→F	1 0 2 1→F
TRUE	FALSE	TRUE
		FALSE
		MAYBE
		TRUE

The rightmost example being a "for case of". Of course, function *F* could be monadic or dyadic:

```

      ∇ R←X F Y
[1] 1:R←X ∘ 2:R←Y ∘ 0:R←'NEITHER!'
      ∇

      'LEFT' (1 0 2 1→F) 'RIGHT'
LEFT
NEITHER!
RIGHT
LEFT

```

What we've got here in fact is a very general use of our control operator. Going just slightly further, we could also define two more uses of the operator:

- *<function 1>* → *<function 2>* is a *<while>*,
- *<function>* → (0 | 1 | 2) creates a derived function with the specified valence.

As in a *<case>*, any result of *<function 1>* in a *<while>* is used to derive variant(s) of *<function 2>*. The difference with the *<case>* is that after the variants have been used, *<function 1>* is called again to derive another (set of) variants of *<function 2>* that are then used, and so on.

The cycle stops when no more variants of *<function 2>* can be derived, that is when *<function 1>* does not return any variant that can be matched to a prefix in *<function 2>*.

This is actually pretty simple:

∇ F	∇ R←TEST
[1] 1:N ∘ N←N-1	[1] R←N>0
∇	∇
N←3 ∘ TEST→F	
1 2 3	
1 2	
1	

By the way, niladic functions used as arguments of a defined operator are treated differently in APL2: rather than being passed as functions to the operator and used by it, they are simply evaluated outside the operator and their result passed to it.

If this behaviour would be changed — I would say corrected — there would be no conflict with their treatment here as arguments of " $\rightarrow$ ". Besides, if-then constructs could be expressed with a defined *IF* operator and a niladic function.

Bracket-Notation for Functions

So far, so good. Except that if we have to define a function each time we need to express even a simple *if then else*, things suddenly get less exciting, and readability improvements are not very evident. For example, replacing:

```
→(A>0)→ELSE ◊ 'POSITIVE' ◊ →END
ELSE: 'ZERO OR NEGATIVE'
END:
```

by:

```
0 0ρ⊞FX 'F' '1:' 'POSITIVE' ' ' '0:' 'ZERO OR NEGATIVE' ' '
(A>0)→F
```

does not seem to buy us a big improvement in readability! Besides, function arrays [BH 91] [Bk 84] [Bn 84] [Br84] could also be used in a similar hard-to-read way: for example, if *FA* is a two-Function Array with an <else> function as first element and a <then> function as the second, then *FA* [⊞IO+A>0] would be quite as bad as our *(A>0)→F* above.

So what we really need to introduce here is at the notational level: we need to be able to express our constructs without having to use pre-defined functions. What we introduce is a variation on direct-definition of functions: one or more APL statements, on one or more lines, enclosed in braces {} [Sa 84][Bu 87] is a function.

The form of function definition offered here: { $+/w$ } is equivalent to $+/$ in tacit-definition [HIM 91]. It is intended for use in expressing more complex algorithms where the arguments and result cannot simply be glued to the left and right of a single expression. The $+/$ notation could of course be used instead of { $+/w$ } for simple cases. Since there is no header, the valence of the function is not explicit. The arguments, if any, are accessed as α and ω , and the function may refer to itself as Δ . The result is that of the last expression executed in the braces. The

specific value matching a prefix (see the appendix) is accessible as $\#$. (note that, since $\#$ is a national character, it's probably not very well suited for such a use, so another *variant* might be better).

The fact that a $\{\}$ -function has no explicit valence allows it to be free of the anomalous treatment of niladic functions. If we need to assign a valence to a $\{\}$ -function, perhaps another version of our " $\rightarrow$ " operator could be used: $\{+/w\} \leftarrow 1$ (or $\{+/+1\}$) would be the monadic variant. But this is outside the scope of this paper.

Also outside our scope here is function assignment. If a $\{\}$ -function can be assigned to a name or used as a normal APL data entity, it becomes a first-class APL citizen, and can be used for example in arrays of functions. The only thing we need to sort out here is under what circumstances are the function or its variants applied (to their arguments, if any) and when are they referenced (as functions, not as results of their execution).

In other words, after $F \leftarrow \{ \dots \}$ is F a function, or is it the result of the execution of $\{ \dots \}$? In current APL, the "niladic-function syndrome" prohibits named-function assignment because its niladic functions are syntactic anomalies: while $F \leftarrow F1$ (with $F1$ a monadic function) could be allowed (which name would be used in the header of F ?), would, on the other hand $F \leftarrow F0$ (with $F0$ niladic) turn F into a copy of $F0$ or assign it the result of its execution?

Of course, the interpretation of $F \leftarrow F0$ might depend on the syntactic class of F at the time of the assignment: $\langle \text{function} \rangle \leftarrow \langle \text{function} \rangle$ would assign a function, and $\langle \text{not a function} \rangle \leftarrow \langle \text{function} \rangle$ would execute and assign a result. Is the APL community ready to get that near to declarations?

Back to $\{\}$ -functions, this anomaly does not exist, since $\{\}$ -functions have no explicit valence (should they be called "anavalent functions"?). The rule assignment vs. execution is simpler: $\{\}$ -functions are executed ONLY if they are given argument(s) -e.g. $\{+/w\} \ V$ — OR they are sitting alone in a statement — e.g. $\{ 'Hi!' \}$ — OR they are derived -e.g. $(A>0) \rightarrow \{1: 'yes'\}$.

Anything else is an assignment. Assignment of a variant can be expressed easily: $VF \leftarrow \{ \text{case} \rightarrow \{ \dots \} \}$.

Now, since there's no reason $\{\}$ -functions shouldn't use prefixes, we can use these to write much-easier to read statements, such as

```
(*A) ← {1:'positive' ∘ 0:'zero' ∘ ~1:'negative'} A (case)
```

or

$$Fib+1 \ 1 \diamond \{10 \triangleright pFib\} \rightarrow \{1:Fib+Fib, +/\sim 2+Fib\} \quad \# \ (while)$$

A $\{\}$ -function can of course spread over multiple lines, just like a current defined function. Immediate-execution mode should of course provide a reminder — a “” under the opening “{” — that a $\{\}$ -function is still open.

Name Scopes

$\{\}$ -functions need local names, and they are free from the local-names list of the header, since there is no header at all. So we can — and have to — design appropriate name scope rules and notation. Here are a few ideas:

- Declaration of a local name could be a side-effect of its initialisation: any statement in the function before the first prefix could be used for setting a name space shared by any derived function(s) and the left argument of the operator. For example,

$$root+ \{Y>tol\} \rightarrow \{tol+0.001 \diamond X+Y+1 \diamond F+\{\dots\} \diamond 1:Y+F \ X-X+\{\dots\} \ X \diamond X\}$$

would have local variables tol , X and Y , plus a local function F . The rule for localisation here is that any name assigned before it is used is local. Further, if we have compilers in mind, we might enforce a rule essentially saying that locals cannot change rank or type after they are initialised.

The function derived from the example above would be equivalent to the following, expressed as a current defined function:

```

v result+derived;tol;X;Y;F
[1]  tol+0.001  $\diamond$  X+Y+1  $\diamond$  0 0p CR 'R+F X ...'
[2]  while:  $\rightarrow$ (Y>tol)+end
[3]    Y+F X-X+{...} X
[4]    +while
[5]  end:result+X
v
```

- Static localisation of names — that is, names visible only inside the braces, NOT from functions defined outside even when called from inside — could be the rule, or at least available. This could be provided with prefixes, e.g. $=: M+0 \ 0p '$ might declare and initialise a static local character matrix. Anyway, a good deal of experimentation with different schemes is probably needed before anything gets cast in concrete!

Conclusion

In this author's opinion, the time has come for APL to get a long overdue renewal and provide control structures before it's too late: newcomers to APL have become a rare species in our community — the average age of participants at APL93 is very striking: we were a gathering of veterans from the Fortran wars! — so the language is threatened with extinction within the current generation.

APL is doomed if nothing is done soon to attract people who have been exposed to structured programming and are repelled by a spaghetti language like APL! Now that we can integrate control structures as first-class APL citizens, there is a window of opportunity the APL community can't afford to miss! We are desperately looking for APL3!

Appendix — Tentative Syntax and Semantics of Control Structures

1. Prefixes

Prefixes are used inside a named or unnamed defined function in order to select which statements are going to be part of a particular derived function. Since prefixes are — at least conceptually — absent from the derived functions, they need not be APL expressions and can have their own syntax. Examples of non-APL syntaxes are found in system commands, `▽-editor`, `□TRAP` or `□FMT`.

Alternatively, prefixes could be any APL statement, but this is probably not necessary if a simple notation can cover 90 percent of cases and the rest is taken care of by providing access to the actual case value with `#` or `□case`.

Each prefix is matched to variants provided by the left argument of the control operator “+” and, in case of a match, the statement(s) at the right and below the colon are included in the derived function.

The maximum rank of all prefixes is called the case rank of the function. This gives the rank of items taken from the left argument and matched with prefixes. So a prefix is either a scalar numeric or character descriptor or a vector of two or more descriptors. A descriptor is either a number, a range of numbers, a character, or one of the following special symbols:

- ? matches any number or character
- * matches a length-0 or longer sequence of numbers or characters
- ~ matches any number or character not previously matched in this variant.
- * and ~ reset the “not matched yet” attribute.
- 0: and ' ': match numeric and character empty vectors, respectively.

For example:

1 *: 2 *:

matches any variant # with # [1] ∈ 1 2

~ 0:

then matches the others who also have # [2] = 0

*:

matches all

? ? *:

matches all length-two or more vectors

'load': 'save': matches these two character vectors
1..3 4..5: matches 1 4, 1 5, 2 4, 2 5, 3 4 and 3 5.

2. Escapes

While we do not need to “branch around” unselected statements, we need to provide the following:

- $\square BREAK$ <condition> exits this variant of the (derived) function.
- $\square EXIT$ <condition> exits all variants. This is useful for a <while>. (<condition> $\square BREAK/EXIT$ <result value> may be worthwhile as well)

3. Explicit names

In non-tactic { }-functions, names have to be provided for arguments, function, case value (more exactly, variant selectors) and perhaps result. This paper uses the following symbols, shown with possible $\square$ -variants:

α ($\square X$) left (data) argument

ω ($\square Y$) right (data) argument

Δ ($\square F$) original function, allows recursion and use of other variants

($\square C$) case used to derive this variant

@ ($\square R$) explicit result: may be useful (?) in cases where it is not convenient to use the last statement to provide a result.

4. Branches and Labels

These two archaic "features" of APL do not interfere with our control operator and prefix notation, so they can be tolerated, at least on the grounds of compatibility with existing APL functions. A few obvious restrictions are needed for controlling their fossil use, such as "branches are not allowed into a {}-function".

Denis Samson
Accès Informatique
940, avenue de Manrèse
Québec G1S 2X1
tel/fax: (418) 688 9645

References

- BH 91 R. Bernecky, R.K. Hui, "*Gerunds and Representations*", APL 91 proceedings, APL Quote-Quad vol. 21 no. 4
- Bk 84 R. Bernecky, "*Function Arrays*", APL 84 proceedings, APL Quote-quad vol. 14 no. 4
- Bn 84 J.P. Benkard, "*Syntactic Experiments with Arrays of Functions and Operators*", APL 84 proceedings, APL Quote-quad vol. 14 no. 4
- Br 84 J.A. Brown, "*Function Assignment and Arrays of Functions*", APL 84 proceedings, APL Quote-quad vol. 14 no. 4
- Bu 91 J. Bunda, "*APL Function Definition Notation*", APL 91 Proceedings, APL Quote-Quad vol. 21 no. 4
- HIM 91 R.K. Hui, K.E.Iverson, E.E. McDonnell, "*Tacit Programming*", APL Quote-Quad vol. 21 no. 4
- IPS 84 K.E. Iverson, R. Pesh, J.H. Shueler, "*An Operator Calculus*", APL 84 proceedings, APL Quote-quad vol. 14 no. 4
- Iv 79 K.E. Iverson, "*The Role of Operators in APL*", APL 79 proceedings, APL Quote-Quad vol. 9 no. 4
- Iv 86 K.E. Iverson, "*A Dictionary of APL*", I.P. Sharp Associates, July 1986
- IW 81 K.E. Iverson, P. K. Wooster, "*A Function Definition Operator*", APL 81 proceedings, APL Quote-Quad vol. 12 no. 1
- Sa 84 Denis P. Samson, "*A Proposal for Control Structures in APL*", APL 84 proceedings, APL Quote-quad vol. 14 no. 4

The Ultimate TURING Proof

by G.A. Langlet

Alan Turing is, with J. von Neumann and K. Iverson, the most important spirit in Computer Science. British readers who are involved in this last field, even if they do not live in Cambridge, probably all know the famous **Turing Machine**, which, as it was proven, could theoretically process ALL POSSIBLE IMAGINABLE ALGORITHMS.

Turing's work on decision theory is less known; however, together with Goto's in Japan and Peirce's in the US, it represents a chef-d'oeuvre of simplicity and efficiency. And Turing could not have proposed his genial machine without thinking on decision first.

Here are two different APL models of a Turing Machine.

Exercise : Scan these models to find the difference.

Let us execute a small session and explain the details after.

```

      ∇ TURING;C;I;e
[1] e←'P[I]←P[I]' ∘ I←[]IO ∘ P←B ∘ vdo 'C←P[I] ∘ I←I+1 ∘ ⌊C/e'
      ∇
      ∇ TURINGD;C;I;e
[1] e←'P[I]←P[I]' ∘ I←[]IO ∘ P←B ∘ vdo 'C←B[I] ∘ I←I+1 ∘ ⌊C/e'
      ∇
      RNDB []←? 99 ∘ I←B ∘ TURING ∘ B←P ∘ TURINGD ∘ I≡P
3
1
      RNDB []←? 99 ∘ I←B ∘ TURING ∘ B←P ∘ TURINGD ∘ I≡P
85
1
      RNDB []←? 99 ∘ I←B ∘ TURING ∘ B←P ∘ TURINGD ∘ I≡P
72
1
      RNDB []←? 99 ∘ I←B ∘ TURING ∘ B←P ∘ TURINGD ∘ I≡P
8
1

```

The APL line which is executed is a conjecture line. One may replace 99 by infinity and never get any answer: the Turing Machine would never stop. The computer's universe is finite, so you are not allowed to replace 99 by infinity. Then, this Turing Machine will stop.

Another case which would stop the Machine is a Parity Error, in fact, any error, because all possible errors are combinations of parity errors. Even a break, coming from you, depressing the <Attn> key, or hitting the computer with a mallet or hammer (in case you're fed up with your system) or coming from the electric power plant, will be perceived by the Machine as some parity error. Will it decide to stop? Anyway, it will stop.

Here comes an essential extension to APL, the "vital do" or "vital execute" function which programs you and me:

vdo e is supposed to iterate the execution of *e*, any APL expression which might program correctly the Universe and Life Game, as much as it can, i.e. until extinction or death, a) by cancer, brain or vessel or heart-break: any inner parity error, b) by outer influence: virus, bomb, bullet, Attn-key, starvation. Cancer may be also be induced by outer influence (e.g. AIDS or bit-mutation by a particle (neutrino?), ingestion of some aromatic hydrocarbons). The difference between a) and b) is small: both cases may kill the job.

Exercise : Program the *vdo* function which is strongly implementation-dependent.

RNDB N produces as the global variable *B* with shape *N*, a Boolean sequence, binary vector with any possible content. So, *B* is any small part of the unknown Initial Program, or a small part of the unknown Initial Data.

The least-action-principle, formulated by Maupertuis, only states that "Nature is thrifty in all its actions". Applied by Hamilton to Newton's mechanics, it simplified the original equations, then allowed Maxwell to propose his general equations for electromagnetism. Reformulated by Ernst Mach as the "economy principle", it was applied by Einstein in all his equations. So, we may suppose it is correct, at least in its original definition, because the Turing machine cannot be approximated correctly by any equation. Any bit may contain parity 0 or 1. There is nothing between. Transition from 1 to 0 or from 0 to 1 is the elementary quantum jump. "No transition" is "no jump", and nothing else. The elementary principle of quantum mechanics is Pauli's principle, which constrains particles (fermions, such as electrons, protons) to have, in the fundamental state, opposed spins $\uparrow \downarrow$ (e.g. 0 1 or 1 0 in Boole's notation) when all other parameters (quantum numbers) are the same.

Then, we shall replace "thrifty" by "minimal" which will also mean "optimal". As a first consequence, *e* must contain the most elementary possible function which is monadic *NOT*, and nothing else. As a second consequence, for reasons of economy, the data ribbon and the program ribbon will be the same ribbon, i.e. a

self-organising structure of information at its quantum level, the bit-level. (While initial APL made a difference between functions and data, LISP does not; "execute" $\rightarrow$ allows you to execute data as a niladic fn. It was introduced into APL-SV at the same time as scans). The vital execute overlaps execute as an endless loop; it corresponds to some special case of the $\square EA$ extension in APL2. But the regular execute has some terrible power over the vital execute. If it fails, life, i.e. the Machine, stops at once. The use of $+$ in $I \leftarrow I+1$ ONLY means: now, process the next bit. The value of I has no importance except in present-time computers based on arithmetic... on any sequence of information, index I could be initialised at any value. The most important thing is death, i.e. caused by indexing (index error), when there is no more information written on the ribbon, "The End" of the movie. As long as there is something, i.e. a parity on the ribbon, life goes on. So, the Machine is not allowed, this time, in this implementation, to rub any bit out and to replace it by "NOTHING"; if it were allowed to do so, it could and would commit suicide...

So, this APL-reduced Turing Machine shall work for eternity, either with an infinite NOTHING-free ribbon, or with a circular (looping) one, which cannot be a Moebius ribbon, because the scenario changes, the ribbon having organised itself differently between two successive readings or writings of the same bit position. But, since combinations of bits in an original finite sequence are in finite number, the same initial sequence MUST appear again. A looping ribbon will produce periodic structures, whatever the content of e is.

Another rule has to be respected by the machine, time irreversibility. So, the APL-reduced implementation is not allowed to go back: index I cannot decrease; however, some structures, already observed in the past, will be reproduced because of periodicity effects (eclipses) with periodic ribbons (orbits). An infinite ribbon will produce self-affine structures, i.e. structures which look like the ones observed in the past, but not quite: exactly what is observed e.g. magnifying the Mandelbrot set (one can see, at any scale, structures which will reproduce, almost, the original one.)

The APL Turing minimal machine is faster, more accurate, more simple than any machine or formula. It does not use floating-point conversions or complex numbers, or integers. Just *NOT* is its heart and brain. The indexing business is a pointer-business. It could be replaced by a $1 \leftarrow 1 \downarrow$ combination or a 1ϕ shift. It is programmed with I for a couple of reasons: a) the APL prototype can may be easily rewritten in any other programming language; b) we want to keep records of the past, in order to make post-mortem autopsies of the results (which would be lost otherwise, if the ribbon was cut just left of the read/write head). The

Turing machine shall make the future. The present implementation preserves the past.

Why are there a couple of machines: *TURING* and *TURINGD*?

For a couple of reasons: *TURING* is the true future-oriented machine; *TURINGD*, a slightly different function, is another interesting machine which allows one to go back to the past although it works exactly the same way as *TURING*, except that *TURINGD*'s control variable is itself in the past (old *B*). *TURING*'s control variable for the future is *P*, the present.

So, *TURINGD* is an H.G.Wells machine, as the inverse function of *TURING*. The conjecture using *RNDB* and the identity $\equiv$, which corresponds to a strong theorem at the end of the paper, shows it, answering 1 for any combination of bits of any length. The couple is a marvellous Stephenson Machine for the exploration of a new Laputa island as one will discover it soon. It is also a Wonderland, a Lewis Carroll machine.

TURING and *TURINGD* are Stephenson machines too, able to run both ways with the most simple flip-flop expression using *NOT*.

Now, let us prove that *TURINGD* is a Babbage machine also, i.e. the prototype of the "error-free Difference Machine".

Take any bit-sequence as *B*. Form the successive Boolean differences between bits. Submit *B* to *TURINGD* and compare the results. Then, *TURINGD* is a Pascal Machine, a Leibniz Machine too, a Prony machine too.

Of course, *TURINGD* is a Gray machine, since it produces for any *B*, the Gray code of *B*, used in signal and image processing...

If *TURINGD* is a difference machine for any *B*, i.e. a Differential machine, then, *TURING* is the Integral Machine, able to integrate infinitely any information at any order of integration with never an error!

If we can prove that *TURINGD* is useless, i.e. that *TURING*, the Least-Action Machine (LAM), is also able to perform the job of *TURINGD*, although *TURING* is a future-oriented integrator, then would not *TURING* become be the Universal Machine, i.e. the Machine of the Universe, the Absolute International Association for Computer Machinery-SIG/APL Vector?

First, if 1 0 is the elementary sequence, also symbolised by *X* as 1 and *Y* as 0, then with *X Y* for *B*, *TURING* produces *X X*. With *X X*, *TURING* produces *X Y*. All right, then, *TURING* is a Sex Machine. But *TURINGD* produces the same results!

This will always be the case with a two-body problem such as man and woman. Period is 2 for a 2-body problem, 4 for a 3- or 4-body etc..., so that, indeed, *TURINGD* produces the results of *TURING* in reverse order. Except for an infinite universe for which the period is infinite, but all the sub-periods (cycles) also exist, *TURING* may produce the results of *TURINGD*, so that *TURINGD* is useless for any finite computer or algorithm.

But, now, does *TURING*, which is able to compute everything with almost nothing, represent itself an optimum in APL notation? No, of course, because APL is the most clever tool ever invented and implemented.

For *B* any binary vector or parity sequence, *TURING*, the Universal Machine, produces, as its product *P*, the same result as what one obtains (moreover on hyper-arrays) without branch, indexing, take or drop, circular shift, execute or even vital do, etc..., as:

```
≠\B      A NO COMMENT.
```

End of Proof.

Begin Future

Note. The original Turing Machine was able to run both ways. It is not necessary if one can prove that future reproduces past at least on finite systems, so that if the results of *TURINGD* which goes backwards are also produced by *TURING* going forwards only, then, *TURINGD* can be sent to the museum.

Annexes

In APL, $0 \star 0$ is 1, even after ISO8485. How NOTHING, raised to No power, gave birth to SOMETHING, is the most mysterious question of the Universe, in order to explain the Bit Bang!

Given *L* and *R*, a couple of vectors with the same shape, the least-square criterion is: $(L-R) \star 2$ for any item. What is the least-square criterion if, in APL2, items become words?

Now, for *L* and *R* expressed in bits, correctly, with a unique code and not with multifractal polysemic "ad hoc" codes, such as IEEE or EBCDIC or many other ones (on exotic computers), expression $(L-R) \star p$ with *p* any power except 0, THUS the least-any-non-null-power criterion, reduces to $L \neq R$ without brackets or exponent, for any meaning of the bits in *L* and *R*.

If α do ω executes α , an APL expression, ω times, then, for any content of L and R with D an integral power of 2, expression:

$$V \leftarrow L, R \diamond 'V \leftarrow \neq V' \text{ do } D \leftarrow \rho B \diamond D \div V$$

or:

$$V \leftarrow R, L \diamond 'V \leftarrow \neq V' \text{ do } D \leftarrow \rho B \diamond D \div V$$

produces the same result as $R \neq L$ or $L \neq R$ (achiral expressions), proving that scalar dyadic $\neq$ also reduces to chiral (asymmetric) monadic $\neq \backslash$, or, rather, that $\neq \backslash$ IS the elementary interaction, able to produce, as consequences of its Existence, all other useful APL constructs which never alter any Bit of information, producing all useful shapes and signals without a single incrementation of entropy. $\neq \backslash$ is perfect, producing past, present and future as a gigantic spacetime linear hologram. Can it also think?

Action Results from Decision

Do it if allowed, if the control variable is dominant, if the neurobit is excited (1).

Don't do it if prohibited (also inhibited), if the control variable is recessive, if the neurobit is at rest (0).

All decisions, then all actions, follow this scheme which is generally admitted. Now, only the status of the neuron, either 0 or 1, will be taken into account, hence the word "neurobit". The quantitative "measure" of neural potential disappears: it has no more meaning. Formerly, the control variable resulted from a collection of weights, after some arbitrary averaging. Alas, information, considered as bits, cannot (and MUST NOT) be averaged (what does "1.8 child per family" mean?); but it can be sampled.

In bits (rather in isomorphous modulo 2 algebra), weights are either 0 (not important) or 1 (important), i.e. information itself; this corresponds to a terrific economy principle... for which data and program were already unified; the results are new data which are also the new program, each bit being its own weight at all orders of integration, differentiation and power, each current bit being the control variable of its own next future, and also the parity check of its past. Could any processor be more clever than this one, knowing that the properties of such an all-mighty self-organiser, have not been inventoried yet?

A recursive decision chain such as:

If A then if B then if C then if D then do it,

is equivalent to "reverse thinking":

If D then if C then if B then if A then do it.

or to "any-order thinking" such as, e.g.:

If B then if D then if A then if C then do it.

All control variables A, B, C, D , in any order, must be TRUE (1).

Then, $\wedge / A, B, C, D$ is equivalent (altogether-control variable).

Then, $\neq / 1 \ 0 \ 0 \ 0$ (parity check) is also equivalent.

This latter expression, seen as $\neg 1 \neq \neg 1 \ 0 \ 0 \ 0$, means, with much more economy of initially-excited neurobits:

If A (i.e. if you are to make a decision so that A is 1), then swap B, then if there is no inhibition coming from B (i.e. if B was 0 so that $A \neq B$ has now swapped B to 1), go on, otherwise, do nothing (the decision chain is broken).

So, $\neq \backslash$ spares excited states, and has the same result as "AND propagated" although, theoretically, it was impossible to deduce AND from XOR in Boole's algebra! But vector processing (parity integration) with $\neq \backslash$ as the unique model of nerve influx, makes miracles: the important clue was to deduce $\wedge \backslash$ (which, of course, contains AND) from $\neq \backslash$, THINKING of vectors and no more of scalars.

$\neq \backslash N + 1$ with any N , is the "domino falling chain" with 1 for "domino falling or fallen" i.e. "going to reach or having reached its quantised stable state" and 0 for "domino standing, potentially able to fall, only if hit by its formerly-standing left-neighbour". If any domino in the chain does not fall on its still-standing right-neighbour, the percolative (physical or decisional) chain breaks in both cases with the same mechanism, also known in electro-magnetism as the {unexplained} Branly effect, or, in coherent light-emission, as the Kastler effect, i.e. laser optical pumping.

When the AND-gate has appeared, then, everything becomes possible, (even arithmetic!) because scalar α AND ω is, in fact, just a special case of the more general triadic MAJORITY function, when the 3rd argument is 0. The essential majority function is necessary to obtain the carry in addition for the 3rd register C. Such a carry becomes the third argument which is left-propagated in addition, but right-propagated in subtraction; the Boolean sum on 3 registers or the Boolean difference on 3 registers A, B, C is, of course, $A \neq B \neq C$ i.e. $\neq / A, B, C$ i.e.

$\neg 1 \neq A, B, C$ the parity check. It would make no difference using $= A, B, C$ instead, since two negations (Boolean, arithmetic or, more generally grammatical, always destroy themselves). $\neq /$ and $= /$ return the same result for any odd number of neurobits along the chain, and opposite results for any even number of neurobits.

A clever brain need not count its neurobits; all it has to know is the parity of the chain in order to decide. And the parity is 1 if the number of bits which have been integrated by $\neq \backslash$ from the beginning of the (now exponent-free) Markov chain, up to the current item (neurobit), i.e. the new current item, is 1, for all terms, for any chain.

A clever brain need not know the status of all the items in order to decide. The knowledge of the last swap is enough: if the chain were broken anywhere, in any order, i.e. if no parity swap has occurred in the last bit of the domino chain, it already knows that it has nothing to do. All the formerly-presupposed high-complexity combinatory arithmetics of the brain has vanished: the decision of "not to do it" as the result of any inhibition, may become as fast as light is...

So, time is, for the brain, created by swaps. No-swap is just sleep. When you sleep, you are not "aware of time". When the result of $\neq /$ is 1, it is simply "time to wake up", at any scale of time, either a fast decision (in quantum-time) on three bits, since the MAJORITY function is also the decision function, or up to a full night of unconscious time on billions of terabits. Unconscious time is dream time, when the whole chain is broken somewhere by any 0, while the whole memory is scanned, integral after integral, in the Sierpinskian process, now generalised to the MAJORITY decisions everywhere in the memory. After almost a whole cycle, when the initial pre-percolative chain $N \uparrow 1$ is restored again, with only one neurobit excited, the first one, the $\neq \backslash N \uparrow 1$ percolates it to the end of the chain, so that $\wedge /$ is produced by $\neq /$ as the alarm-clock; this single bit means "do it", i.e. "wake up" i.e. "be conscious again": another day begins.

The important point was indeed to realise that $\wedge /$ is a temporal process, and to introduce quantum time into logics. "If then if then if then..." is equivalent to $\neq /$ on resting neurobits except the first one, which contains the main decision (on ONE excited neurobit) to explore the chain, i.e. to scan it.

$\wedge \backslash$ was atemporal in conventional binary algebra, and required too much stress, i.e. all bits excited. $\neq \backslash$ THE clever parity-check algorithm, the laziest algorithm of all algorithms, excites all the sleeping neurobits by its wave, produces $\wedge /$ as a side effect within a short flash, while the second wave of $\neq \backslash$ reduces the stress between neighbouring 1's, swapping every even neurobit to 0 for rest, immediately, without aspirin, because of it short-range repulsive action:

" $1 \neq 1$ produces 0", i.e. $\neq / 1 \quad 1$ the "short-range repulse", will become the secret key of a correct dynamic interpretation of mental cryptography, BOTH encryption and decryption; it was the secret of conventional cryptography already. But it will also decrypt gravitation in general, with and without inhibition, i.e. external repulsive effect: the dominos simply fall and stay lying because the Earth's gravitational field (another $\neq \backslash$ effect) inhibits the second wave and becomes dominant once the first wave has processed the chain: this explains what the macroscopic notion of the mysterious "potential energy" was. Avalanches and Newton's apples indeed stay on the floor after the first wave; just remove the inhibitor field (e.g. experiencing stroboscopy on dominos or sugar lumps in space), then, because of time shift, i.e. phase delay, the full Sierpinski effect of the elementary interaction will appear on the spacetime image, because all successive waves of $\neq \backslash$ will execute without Earth's gravitational inhibition. This is what occurs within the atom itself, Earth's action (inhibition) being screened there too (inhibition is inhibited). Altogether, this will explain the famous chaos game by Barnsley, when tossing a hypothetical 3-sided coin (Head, Tail, Side) one million times and give an idea of what "chance" is.

The conventional Gaussian shape of combinatory statistics is just the envelope of the discrete sequence of the binomial coefficients $p!n$ which form each row of the Pascal triangle; for any p and any n which are different from 0, i.e. exist; then, in parity algebra, with $n \equiv 1$ and $p \equiv 1$, of course, the result is 1. For all other cases, the thrifty brain sleeps. Sierpinski is both the parity of Pascal and the result of "tossing" in the parity field. $\neq \backslash$ produces the universal "roll". And exclusion is also a "deal". A good deal. A good action. A good auction.

God does not play dice. Everything plays $\neq \backslash$. Pascal's bet is Sierpinski's, i.e. chance on parities. Our best chance of understanding lies with the best tool for thinking & checking: APL.

A VSAM Simulation for APL*PlusII/386

by Allan Gay, Cocking & Drury Ltd

Introduction

This article describes a VSAM simulation programmed in APL*PLUS II/386 and used by some recently-migrated mainframe APL2/TSO applications.

For convenience, the discussion is divided into the following sections:

- Overview
- KSDSs, real and simulated
- Using the simulation
- Migrating a mainframe KSDS
- Conclusion

Overview

The simulation comprises a workspace which may be copied into an application workspace to provide means of operating simulated VSAM key sequenced datasets (KSDSs) which reside on PC.

The workspace includes a set of public functions corresponding to the documented AP123 opcodes [VSAPLTUG].

A simulated KSDS is an APL*PLUS II/386 component file containing a mandatory arrangement of control components. Each VSAM data record is stored as a separate component.

Features

A separate APL*PLUS II/386 component file is used for each simulated KSDS.

Alternate indexes are supported. If declared, they are embedded in the component file concerned. A simulated KSDS' base cluster and its alternate indexes may be open and accessed concurrently if desired. Multiple simulated KSDSs may be open concurrently, too.

Only key-sequenced datasets (KSDS) are simulated. VSAM's entry-sequenced and relative-record dataset types are not supported.

The full repertoire of access opcodes offered by mainframe AP123 is implemented. Operations to open, position and close a KSDS are provided. Records may be read, written and erased. Access may be sequential as well as by key. Also programmed is a bogus read-backwards opcode "B".

Each AP123 opcode is simulated by a corresponding APL*PLUS II/386 function. The function arguments and results serve the role of the auxiliary processor's *CTL* and *DAT* shared variables. This makes converting migrated mainframe application code a straightforward clerical task. During our own migration project, we typically spent about two hours per workspace reviewing and recasting AP123 calls.

KSDSs, Real and Simulated

What is a KSDS?

From the application standpoint, a KSDS is a matrix (perhaps with a ragged right-hand edge) with a keyfield stripe running from top to bottom of it. If the KSDS has any alternate indexes, they are just extra keyfield stripes down this imaginary matrix. Each row of the matrix is a separate record, and access is on an individual record basis.

Records can be read both sequentially and directly. When you read a record directly, you present a keyfield value and receive the record which bears that value in its keyfield. The applicable keyfield is established when the KSDS is allocated: if you allocate the base cluster, the primary keyfield is active, and if you allocate an alternate index, the relevant alternative keyfield applies.

When you read sequentially, the records are returned to you one by one in ascending sequence per the applicable keyfield.

A simulated KSDS

A simulated KSDS is an APL*PLUS II/386 component file. The first ten components are reserved for control information and documentation, and include some spare components in case of future design extensions. Each subsequent component hosts one KSDS data record.

When adding a new record, any component released by a prior record deletion will be re-used; if there are no such components, a new component is appended to the file. Record ordering is therefore roughly chronological, with some inconsistencies.

The first ten components are: (1) freeform documentation, (2) the master index relating keys to component numbers, (3) a standard character vector identifying the file as a simulated KSDS and checked by the simulation at open time, and (4-10) empty vector spares.

The Master Index Component

The master index component keeps the component number for each record key. Access is always via the master index, even when an application is accessing the simulated KSDS sequentially and is not supplying keys.

The component is a five-item nested vector. The first two items map the record keys to the corresponding components, being respectively a text matrix of keys and a vector of component numbers. The third and fourth items identify the positions of keyfields within the matrix of keys and within the records themselves. The fifth item contains padding and is used to buffer the effects of recordcount fluctuations upon the size of the master index component.

Buffering the effects of a fluctuating record count

As users of component filing systems will know, the "expanding replace" is a phenomenon which can waste a lot of space in a component file. An expanding replace occurs when an enlarged component is refilled. Because the component is too large to fit into the space it previously occupied, it is written elsewhere, within the file's free space, and the space which it previously occupied becomes orphaned until a file reorganisation is instigated by the user.

When a simulated KSDS is growing, new keys are being added to the master index component. This component is refilled after each addition. If the component were larger, an expanding replace would occur. Eventually, the component file would reach an enormous size, although much of the space it contained would be orphaned.

It is to prevent this that the master index component is overallocated by including a padding item. By adjusting the size of the padding item, the overall size of the master index component is mostly maintained at a constant figure. The effect is to buffer the effects of recordcount fluctuations, minimising the number of expanding replaces occurring when the master index component is refilled.

A manifest constant held in a global variable specifies the overallocation percentage required. For example, imagine we create a simulated KSDS with 100 records and specify a 10% overallocation. We get a padding item which is 10%

the size of the rest of the master index. We can add ten more records before all the padding is used up. When we add the eleventh record, padding sufficient for eleven more records (10% of 111 records) is added and the first expanding replace occurs.

Similarly, if the simulated KSDS were to shrink through record deletion, the padding item would expand until the number of records fell below 90, at which time it would be reduced commensurately. Space thus released would be reclaimable via `□FDUP`.

In a file with a steadily growing recordcount, a large overallocation percentage gives advantages. In a file which is oscillating about some mean recordcount, a well-chosen percentage can completely eliminate expanding replaces of the master index component.

Internal consistency of the various filed components is not jeopardised by `□FDUP`.

Record components

Record components contain character vectors. Applications may deem the records to be composed of fields, and may deem those fields to contain many forms of data. However, the VSAM simulation merely stores the bytes and leaves their interpretation to the applications.

From the application viewpoint, record fields can hold data in any of the mainframe formats — EBCDIC text, packed decimal, binary fullword, etc. This is unlikely to present any additional problem to a migrated mainframe application, provided that the records have been migrated correctly. Routines which encoded and decoded non-EBCDIC data on the mainframe should still work, in converted form, on the PC. Of course, this is not a VSAM-related issue: the same points apply to any dataset migrated from the mainframe regardless of access method.

Using the Simulation

Functions

The functions supplied in the simulation workspace are classed as public or private. The public functions are identifiable by their naming convention. The private functions operate directly upon temporary global variables and should not be used explicitly by applications.

The simulation provides a corresponding public function for each AP123 opcode. These functions are named in the form "VSKS..", where ".." is the AP123 opcode. For example, mainframe opcode RU (Read for Update) becomes function *VSKSRU* in the APL*PLUS II/386 context.

In addition to the opcode functions, public functions *VSAMchk*, *VSKSB*, and *VSKSSF* are provided. The first may be used to filter returncodes, the second is a pseudo-opcode "B" (for Backwards read), and the last enables the application to switch between open simulated KSDS indexes.

The public functions are:

FUNCTION	PURPOSE	α-ARG	-ARG
VSAMchk	- Check rc	acceptable rcs	rc to check
VSKSB	- Backwards read		generic key *
VSKSC	- Close current view		
VSKSE	- Erase record		recordkey
VSKSKF	- Key Feedback		
VKSOC	- Open & Clear	indexno (1=base)	libno filename
VKSOR	- Open for Read	indexno (1=base)	libno filename
VKSOU	- Open for Update	indexno (1=base)	libno filename
VKSOW	- Open for Write	indexno (1=base)	libno filename
VKSPO	- Position		generic key **
VKSRR	- Read		recordkey *
VKSRU	- Read for Update		recordkey *
VKSSF	- Switch File		fileno
VKSW	- Write		record

* when arg is empty, read next record in indicated direction.

** when arg is empty, position to first record in view.

Programming considerations

Because the concept of pre-allocating files does not exist in the PC context, the openfile operations (AP123's OC OR OU OW opcodes) take a component filename specification instead of a DDname. The spec must be in the form "libno filename", which means library numbers must be predefined in the APL*PLUS II/386 configuration file used at APL*PLUS II/386 startup.

Shared variables are not available in the APL*PLUS II/386 environment. Since there is no AP123 to record the state of play, temporary global variables are used instead. There is no *SV0* analogue: instead the open is simply issued (by running the function corresponding to the desired opcode), specifying APL*PLUS II/386 library number and component filename as function arguments. A file-number is returned from the OC OR OU OW services and must be retained for the simulation to use in indexing the temporary global variables.

The simulation uses the temporary global variables to host control-data for the simulated KSDSs which are currently open. It is not necessary explicitly to pre-create these variables. They are automatically created when the first simulated KSDS is opened. They are automatically expunged when the last simulated KSDS is closed.

In mainframe AP123 programming, services are accessed by specifying one- or two-letter opcodes via the shared control variable, parameter information being preassigned to the shared data variable. The choice of shared variable pair determines which KSDS (whether base-cluster or alternate-index) of those currently open is accessed.

In the simulation, there are no shared variables. Instead, the opcode is implied by the application's choice of public function, and the parameter information is specified via that function's arguments. The current "view" is always accessed.

Every open base-cluster or alternate-index is termed a view, and every view has a unique file-number which was assigned when it was opened. The application should capture this number (returned by the open routines *VSKSOC*, *VSKSOR*, *VSKSOU* and *VSKSOW*).

One view (by default, the latest opened) is deemed current, and it is upon the current view that all I/O operations are transacted. To make another view current, the application specifies its number to the *VSKSSF* "Switch File" function.

When a simulated KSDS is first opened, its master index component is read into the workspace, and some additional view-related information is built. If another of that simulated KSDS' indexes is subsequently opened, the same copy of the master index is re-used, and another set of view-related information is built. This approach enables an application to modify a simulated KSDS via any of its open indexes without the risk of conflicting updates. Logic to synchronise view-related information is included.

Converting a migrated mainframe workspace

Conversion is a manual process.

First, all TSO ALLOC commands and *SV0s* are removed completely.

Next, each use of the AP123 opcodes is replaced by the equivalent simulation function. Assignments to *DAT* become function args, and assignments from *DAT* are now taken from function results. *CTL* returncodes are also taken from

function results. If multiple pairs of shared variables were used, the *VSKSSF* function is added to select the appropriate file-number.

Next, each open function call is recoded to supply a libno and a filename (instead of supplying a DDname). An additional integer argument designates the index to be used. The indexes are numbered with the origin-1 indices of their respective keyfields in the master index component. The base cluster is always index number 1.

Finally, paired calls to functions *VSKSSF* and *VSKSC* are coded in place of every *SVR* for which no prior close was issued, and all *SVRs* are removed.

Returncodes

Where appropriate, the returncodes documented for AP123 are used [VSAPLTUG]. Additional codes are provided for attempts to access nonexistent indexes or views. The extra codes form a new series associated with a new major returncode by which they are differentiated from the standard repertoire.

Size

The simulation requires 127K of *WA* when no simulated KSDSs have yet been opened.

Because the master index component of each open KSDS is held in global variables, the operating space requirement depends on the number of KSDSs open, their keylengths and their recordcounts, plus some additional penalty for each open alternate index. In short, the simulation trades space for speed.

For one application which we migrated, opening the base cluster on a 7,459-record simulated KSDS with 32-byte keys, took 593K of *WA*. Going on to open that KSDS' sole alternate index, which featured 8-byte keys, took an extra 119K of *WA*.

Another similarly configured simulated KSDS contained 7,487 records and yielded proportionately similar results. Because the PC had 16MB of RAM (the limit for that particular machine but not, of course, for APL*PLUS II/386) and the application workspace amounted to only 815K including all simulations, there was no hint of a *WA* problem.

Migrating a Mainframe KSDS

Migrating a KSDS to the PC

The records should first be dumped to a flat sequential file via the Access Method Services REPRO utility. The sequential file should then be transferred to PC, using a product such as IRMA or PC3270. Finally, APL*PLUS II/386 must be used to build a component file containing the correct arrangement of components.

If the KSDS records contain only EBCDIC text, translation to ASCII should be selected during transfer of the sequential file to the PC. If, however, the KSDS records contain numeric fields or APL expressions, translation must not be selected there. Instead, translation must be performed on a field by field basis when the records are loaded into the simulated KSDS.

Treating numerical quantities (packed decimal, binary fullword, and so on) as if they were EBCDIC text and translating their bytes to ASCII is definitely a mistake.

Matters can also get interesting if the records actually contain executable APL statements. Translation from EBCDIC to ASCII is then inappropriate. Translation from APL2's `⎕AV` to APL*PLUS II/386's `⎕AV` is required instead.

Writing a KSDS builder

Writing a function to build a simulated KSDS from a sequential file is conceptually quite straightforward.

First, the function should write the standard ten preliminary components, the second being a dummy as placeholder for the master index.

Second, the records should be read, processed, and appended as individual components. Processing a record entails translating any EBCDIC or APL fields within it, and then noting its key (or keys, if the KSDS includes alternate indexes).

Third, the master index component should be constructed from the accumulated key data, filed in the position occupied by its placeholder, and the component file reorganised via `⎕FDUP`.

As ever, the devil is in the detail. When a KSDS contains multiple record types, each incorporating many fields and using many data formats, the record-processing logic will be extensive.

Conclusions

The migration of a given mainframe KSDS is a one-time affair. Given good record layout documentation, writing the record conversion logic is merely a tedious matter of coding the right field indices against the record vector. This is true for all types of file, not just VSAM.

This is a comprehensive simulation and its internals are necessarily complex. However, its external interface is systematic and largely preserves the look of its mainframe precursor. This keeps the work of converting migrated mainframe logic to a minimum.

Performance was not quantified, but it was certainly adequate on the top-end 486 machines employed.

Next: simulating Defined Operators.

Reference

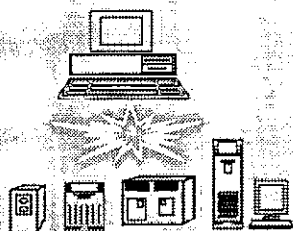
[VSAPLTUG] . VSAPL for TSO Terminal User's Guide, IBM, SH20-9180.

APL ★ Plus II/386 and III Compatible
Dyalog APL/W Compatible

DB+

by TMT-Team Oy

Database Interface



To: Btrieve, dBase, Excel, IBM DB2, INFORMIX, INGRES, Novell's Netware SQL, Tandem NonStop SQL, Oracle, OS/2 DBM, AS/400, Paradox, PROGRESS, Microsoft SQL Server, Sybase, Gupta SQLBase, XDB...

UITW - A complete User interface toolkit for APLers

Demos Spreadsheets Aform samples

Demos Tboxes Forms Tel SS SS2 SS3 Flds Aform YTS

Background, BMP
Iconic button...

	93	94	Futd
PC	123,00		
- 9.x	456,00		
- 10.x	123,00		
- 11	456,00		
II/386	123,00		
- 4.x	456,00		
- 5.x	123,00		
III	456,00		
- 1.0	123,00		

Integer... 1 234 567 890
2-Digit... 123 456 789,55

Date [Windows]... 1993-12-24
Time HHMM... 12:15
Time HHMMSS... 12:15:30
Progress... 57%
UITW is a high-level UI toolkit for multiple plat-

Password...
Char Mask... 45, gf - Hfgh
EditCombo... Your choice
Bitmap, BMP

APL Add-ons and Consulting, TMT-Team Oy, Finland
Tel: 010-358-0-804051* and 80405276, fax 80405222

Index to Advertisers

Dyadic Systems Ltd	8
TMT-Team Oy	142
MicroAPL	80
Soliton Associates	2

All queries regarding advertising in VECTOR should be made to Gill Smith, at 04393-385.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Anthony Camacho,
11 Auburn Road, Redland,
BRISTOL, BS6 6LS
Tel: 0272-730036

Authors wishing to use Windows Write or Word for Windows should contact Vector Production for a copy of the Vector APL TrueType font and Vector APL typebox.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK. Tel: 04393-385 (any time).

BAA: Membership Application Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____
 Address Line 1: _____
 Address Line 2: _____
 Address Line 3: _____
 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category (please tick box): 93/94

(these rates also apply to renewals)

UK private membership	£12	☐
Overseas private membership	£20	☐
Airmail supplement (not needed for Europe)	£8	☐
Corporate membership	£100	☐
Corporate membership overseas	£155	☐
Sustaining membership	£430	☐
Non-voting UK member (student/OAP/unemployed only) £6		☐

PAYMENT — in Sterling only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard or Visa number.

I authorise you to debit my Visa/Mastercard account

Number: _____ Expiry date: ____|____

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

*Data Protection Act:
 The information supplied may be
 stored on computer and processed
 in accordance with the registration
 of the British Computer Society.*

(please tick the required option above)

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON, E3 5HU

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1993/94 Committee

Chairman:	David Eastwood 071-922 8866 microapl@applelink.apple.com	MicroAPL Ltd., South Bank Technopark, 90 London Road, LONDON SE1 6LN
Secretary:	Duncan Pearson 0904-603510 100265.1564@compuserve.com	143 Hull Road, YORK YO1 3JX
Treasurer:	Nicholas Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Anthony Camacho 0272-730036 acamacho@cix.compulink.co.uk	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Activities:	Vacant Post	
Education:	Dr Alan Mayer 0792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Technical:	Jonathan Barman 0488-648575 100116.1030@compuserve.com	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Projects:	George MacLeod 0442-878065	Greymantle Associates Ltd., Bartrum House, Ravens Lane, BERKHAMSTED, Herts HP4 2DY
Publicity:	Misha Jovanovic 0753-853141	99 Oxford Road, WINDSOR, Berks SL4 5DX
Recruitment:	Mark Harris 0438-310155 kestrel@apl.demon.co.uk	Kestrel Consulting, Business & Technology Centre, Bessemer Drive, STEVENAGE, Herts SG1 2DX
Administration:	Rowena Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Anthony Camacho	0272-730036
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (04393-385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (04393-385)
Support Team:	Jonathan & Bridget Barman (0488648-575), Ray Cannon (0252-874697), Richard Weber (0302-539761), Sylvia Camacho, Duncan Pearson, John Searle, David Ziemann (071-267 8032), Jon Sandles	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" — an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Compass R&D Ltd
15 Frederick Sanger Road
Surrey Research Park
GUILDFORD, Surrey GU2 5YD
Tel: 0483-302249
Fax: 0483-302279

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE
Hants. RG24 0AL
Tel: 0256-811125
Fax: 0256-811130

HMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 071-353 4212
Fax: 071-353 3325

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel: 071-922 8866
Fax: 071-928 1006

Soliton Associates Ltd
Groot Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel: +31 20 570 8733
Fax: +31 20 570 8758
Email: ljh@soliton.com

Kestrel Consulting
Business & Technology Centre
Bessemer Drive
STEVENAGE, Herts SG1 2DX
Tel: 0438-310155
Fax: 0438-310131
Email: kestrel@apl.demon.co.uk

Manugistics
2115 East Jefferson St
Rockville
MARYLAND 20852
Tel: (301) 984-5412
Fax: (301) 984-5094
USA

Dutch APL Association
Postbus 1341
3430BH Nieuwegein
Netherlands
Tel: 03474-2337

Insight Systems ApS
Nordre Strandvej 119A
DK-3150 Hellebæk
Denmark
Tel: +45 42 10 70 22
Fax: +45 42 10 75 74